



COMPUTER SCIENCE FIGHTING GAME PROJECT



Contents

Analysis.....	4
What the problem is and why its computational.....	4
Stakeholders.....	5
[StakeHolder] (Stakeholder 1).....	5
[StakeHolder2] (Stakeholder 2).....	5
[StakeHolder3] (Stakeholder 3).....	5
How Stakeholders will be used.....	5
Examples of previous attempts	6
Super Smash Brothers Series.....	6
Granblue Fantasy Versus.....	7
Guilty Gear Strive.....	7
DNF Duel.....	8
Street Fighter V.....	9
Footsies	10
What are the Essential Features?.....	11
What are the Limitations of this implementation?.....	11
Requirements of the Solution.....	12
Hardware Requirements.....	12
Software Requirements.....	12
General Requirements.....	12
Success Criteria.....	15
Design.....	16
Decomposition.....	16
How is each problem solved?	17
Main Menu.....	17
Initialise Players.....	17
Initialise Health.....	18
Mapping Controls.....	18
Usability Features.....	21
Buttons.....	21
Clear Graphical Interface.....	21
Colour Scheme.....	22
Key Data Structures.....	22

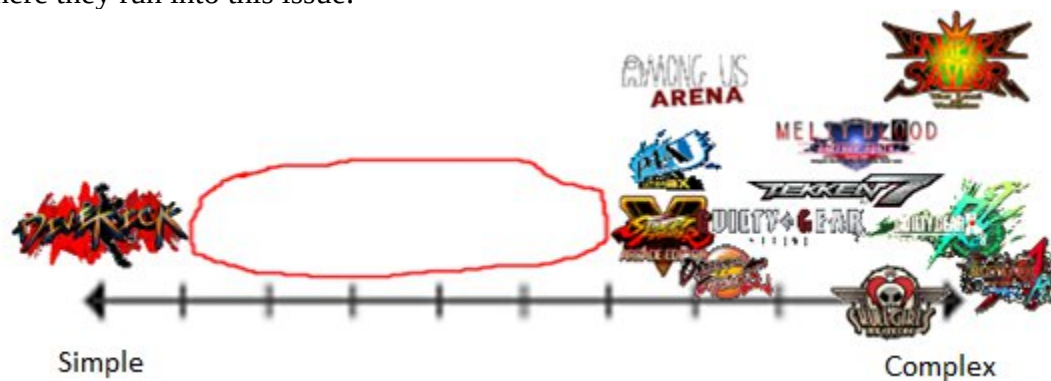
Test Data	24
Post Development Survey	27
Development	29
Prototype 1 - Walking	29
Constructor Method	30
Update Method	30
Animation Handling	31
Left and Right Walk Methods	33
Build.py	34
Alpha Testing	34
Prototype 2 - Jumping	35
Correcting Direction Faced Method:	35
Crouch Status	36
Jumping	38
Alpha Testing	39
Prototype 3 – Hitboxes & Hurtboxes	40
Forward and Back Dashes	40
Hitbox Hurtbox System	42
Alpha Testing	45
Bugs Found	45
Prototype 4 – Health Bars, Blocking, & Specials	47
Blocking System	48
Heavy Attacks and Specials	49
Adding pushback on normals	51
Healthbar	52
Custom Controls	55
Stakeholder response	55
Alpha Testing	55
Bugs found	56
Prototype 5 – Menus	57
Main Menu	57
Pause Menu	58
Stakeholder Response	60
Alpha Testing	60
Bugs found	61
Prototype 6 – Training mode & Settings	62
Settings	62

Training Mode.....	65
Stakeholder Response	67
Alpha Testing.....	67
Current bugs found.....	68
Prototype 7 – Adding a second character.....	69
Css.py.....	70
Animations.py	71
Stakeholder Response	72

Analysis

What the problem is and why its computational

Fighting games are a notoriously hard genre to get into for casual players, both due to their public perception of difficulty, and their actual difficulty. There is a noticeable gap in the fighting game market. On one end, there is the joke fighting game Divekick, which only has 2 buttons, Dive and Kick. This makes it incredibly easy to understand for casuals but has a major lack of depth. When someone who has gotten into fighting games through Divekick, the natural question is “What’s next?”. This is where they run into this issue:



Every other popular fighting game is exponentially harder to get into than Divekick. All of these work together to create a much larger power gap between those who have played the game and learnt all these mechanics, and those who haven't, compared to Divekick. This gap in results due to skill should exist, but it's overwhelming size may cause many casuals to simply just leave fighting games once they've had their fun with Divekick, not truly bringing them *into* the genre.

A particularly large issue for many casuals is doing motion inputs, such as a Dragon Punch motion, which most fighting games lock special moves behind. Special moves are often a core part of a character's playstyle, and their consistent execution is incredibly important to achieving intentionality with your gameplay, meaning the transition from using moves without a reason to using moves for specific reasons consistently in neutral.

Another big issue is the difficulty of performing your first combo within a fighting game in many games, such as the Street Fighter series. This creates the perception of fighting games requiring a lot of lab time for casuals, which very often pushes people away before they even play the game.

The goal of this project is to create a fighting game that can strike a balance with simple for casuals to get into, and a fighting game with some depth – a fighting game able to plug the gap between Divekick and Street Fighter.

This problem is computational as the genre of fighting games as a genre are by their nature electronic games. This means that this problem can only be solved via computational means.

There are various game states that require coding:

- The blocking state (including the calculations of blockstun – the amount of time you are unable to move after a move hits your block, and the coding of when to go into preblock – where your character changes animation, about to enter the block state while the opponent is close) is computational as the calculations relating to blockstun must be automatically calculated, as well as the scenarios where the character goes into preblock must be automatically checked.
- The usage of attacks (such as normal moves and specials) is computational as all code relating to attacking, such as the changing of sprites, the damage dealt by a move, the hitbox size of the attacks, and the blockstun it deals, as well as other move properties, must all be done automatically by the videogame
- The movement of the player (including introducing a player controller) is computational as the interfacing between the keyboard and the videogame must work without the need for the end-user to fiddle around with controls in order to get the game to work

Furthermore the interfacing of real life peripherals such as a fightstick or a controller requires computational methods, such as the usage of libraries.

Stakeholders

The target group for this project is fairly wide, as its anyone with a passing interest in fighting games, which can apply to many people. This is because the project is intended to be an easy to play fighting game. This project is therefore going to ask for the opinions of many people, from young children, to fighting game veterans.

I will therefore have 3 main stakeholders:

[StakeHolder] (Stakeholder 1)

[StakeHolder] is a 17-year-old who rarely plays fighting games but has some experience with video games in general. He is useful as a stakeholder as he will show whether the game is engaging to the average demographic for games in general, while still being a beginner in fighting games.

[StakeHolder2] (Stakeholder 2)

[StakeHolder2] is an elderly man whose only experience with a video game was Wii sports on the Wii. He will be the greatest test to see if the game is accessible to those of all ages, as those who are older generally have issues using technology, especially in understanding video games.

[StakeHolder3] (Stakeholder 3)

[StakeHolder3] is a 9-year-old student in primary school. Having him as my stakeholder as he will be able to show if the game is simple enough for a young child to pick up.

How Stakeholders will be used

Stakeholders will be used throughout the process and will be core to the design decisions of the game. They will be consulted initially for essential features and requirements of the game, using a questionnaire. I will also continuously show off builds of the game in progress in order to gauge their opinions, retroactively changing requirements as required by the stakeholders' wants and needs. They will have a say on mechanics and presentation, and the majority opinion of them will be consulted for any major decisions. The stakeholders are important for this project as they are part of the audience the project is attempting to reach and satisfy, and their enjoyment of the game is one of the main goals chosen for this project.

Examples of previous attempts

There are many attempts to create a simple fighting game. Many have succeeded, while even more have failed.

Super Smash Brothers Series



The Super Smash Bros. Series, created by Nintendo, Bandai Namco, and Sora Ltd. is a fighting game created initially to cater to more casual audiences. This was initially spurred on by the lead developer of the series (Masahiro Sakurai) having the experience of seeing how harder fighting games like King of Fighters leave casual audiences feeling. The Smash series removes motion inputs entirely. It also elects to have a percentage and knockback-based system as opposed to a health-based system that traditional fighting games such as Street Fighter or Fatal Fury have.

These 2 major changes along with many other significant changes such as the way blocking is handled (removing high-low or crossup mixups entirely from the game), or the introduction of Directional Influence in Super Smash Bros. Melee (which makes combos much less guaranteed than in traditional fighting games). These changes this series have created an entire subgenre of fighting games called platform fighters, due to the series and the games it inspired being wildly different from traditional fighters. Super Smash Bros. therefore achieves the goal of being accessible with flying colours, but it does not act as a gateway to other traditional fighters very well, due to its large number of differences.

What can be taken from this game

One of the major things that can be taken from this game is the lack of motion inputs, instead having a special move button that uses directional inputs to choose the specials. This is much more intuitive for the end-user, and a large part of why people

are so easily able to get into Super Smash Bros. as a series. This lack of motion inputs may be useful as an inspiration for this fighting game project.

Granblue Fantasy Versus



Granblue Fantasy Versus is a fighting game released in 2020, created by Arc System Works. Unlike most fighting games, it contains both a guard button and a skill button. The Skill button eliminates the need for a motion input to perform a special, instead only requiring a direction and the Skill button, like Smash Brothers. What Granblue does that is unlike most fighting games, is introduce a slight cooldown on specials. This cooldown depletes faster when the motion input for a special is done instead of the Skill button, which emphasises the use of motion inputs once you know them. This allowed players to pick whether to do the easy input or the harder inputs, which let them quickly reach intentionality with their special use, while not alienating other players.

The introduction of a guard button, however, removes a large amount of complexity from the offense of GBVS, removing high-low mixups, a major part of offense in most fighting games, while also removing an important balance lever for specials and normal (such as making a slower move an overhead, or making an especially strong move a mid, meaning it can be blocked both high and low). The guard button also removes crossups, which is a similarly large reduction to mixups, specifically destroying ambiguous crossups.

As a whole, the game achieves its goals of simplifying the genre successfully, while retaining motion inputs as a harder but more rewarding option.

What can be taken from this game

One major thing that can be taken from the game is the slight cooldown on specials. This is a very interesting mechanic unique to this game that provides an advantage to the motion input specials within the game. This approach works well if we choose to retain motion inputs.

Guilty Gear Strive



Guilty Gear Strive is a fighting game released in 2021, created by Arc System Works. The Guilty Gear series is generally known for its difficult execution, especially on combos, and its large quantity of system mechanics to watch out for, such as Instant Block, Roman Cancel, Blitz, Danger Time, and Faultless Defence. This made older guilty gears such as XRD and +r very unapproachable for beginners to the genre, with most combos being very hard to pull off, and gatlings (the normal moves a move can cancel into) varying heavily by character. This made the series sell fairly poorly, despite its major positives in its incredible visual style.

Guilty Gear Strive bucks this trend of low sales through the simplification of the system mechanics to a smaller subset than in XRD or +r, removing mechanics such as blitz and danger time, and making gatlings much more uniform, allowing people to switch characters easier. The reduction of complexity in combos, along with the introduction of Strive's Roman Cancel system, which is much more intuitive, and slows time to give you more time to get follow-ups off of Roman Cancels; reduced the barrier of entry for newcomers to the series, resulting in landmark sales for the series, becoming one of the largest current fighting games, with a playerbase on par with Street Fighter in size.

The biggest change from previous games in strive however is the damage each individual move does. Each move deals significantly more damage compared to XRD and +r in general, making shorter combos and moves that do not convert significantly more useful than in other fighters with lower damage, while allowing comebacks to occur much more frequently than in +r or XRD. This change is most likely the largest single reduction in skill gap in Strive.

Overall, Strive succeeds with flying colours at bringing a fairly niche fighting game into mainstream relevance as one of the biggest current fighting games compared to its previous entries.

What can be taken from this game

The combo system of Guilty Gear Strive has proven incredibly easy for fighting game newcomers to understand, and along with its high damage, evens the playing field out between beginners and advanced players, as comebacks are much more likely. This kind of dynamic with combos is one that would be great to achieve in this project, to provide the same feeling of learning a basic combo in Guilty Gear.

DNF Duel



DNF Duel is a fighting game released in 2022, created by Arc System Works. This game is similar to Granblue Versus in its mechanics, however its balance overall creates a significantly different feeling game. Its introduction of incredibly large buttons in neutral that are often able to convert into full combos that can often nearly 1 touch. This combined with the restrictive movement makes it incredibly hard for beginners to properly get in against those buttons, while on the other side not really teaching the one pressing these normal attacks how to properly play neutral, such as how to whiff punish or space. These already create a game where although the execution is incredibly simple, the neutral game is incredibly hard for newer players to properly deal with, as the lack of freedom in movement can make everything much harder.

This is combined with a mechanic known as Awakening, a comeback mechanic that activates at low health, allows characters to easily touch of death the opponent in a single combo at low health, which can very easily cause frustration for newcomers, if they are winning and suddenly get hit once and die. Overall, this frustration with mechanics in DNF Duel has caused the game's community to fade out nearly as fast as it appeared.

What can be taken from this game

This game, for all its flaws, has a couple of things that can be taken and understood from the game. One specific thing that can be taken is its freeform special cancel system, which allows for interesting combo extension, although its reliance on grey health makes it significantly less available than it otherwise could be though.

Street Fighter V



Street Fighter V was released in 2016 by Capcom. This game comes directly after the incredibly successful Ultra Street Fighter 4, which is very much known for its tight “one frame link” (meaning the combo has a window of a single frame to press the button for the combo to be true) combos, which is not realistically doable by

newcomers at all, leaving characters who are more reliant on one frame links much weaker in the hands of newcomers. Street Fighter V fixes this through the introduction of buffer. This buffer is 3 frames long, meaning that when a move is inputted while another move is active, the game remembers the input for 3 frames more than it does in Street Fighter 4, a game without buffer. This makes the game's combos more accessible.

What can be taken from this game

A major thing that can be taken from the game is how important the introduction of buffer is into the game for ease of use for lower level players, as it heavily decreased the skill floor for combos for many characters, especially ones who relied heavily on one frame links in USF4.

Footsies



Footsies is a very simple fighting game. It has only 1 button to attack, which has differing attacks depending on whether you hold a direction or not, and whether you hold the button or not (creating 2 special attacks and 2 normal attacks). Its simple nature makes it work on effectively any platform, including phones. This simple nature (lacking even jumping or throws, something nearly every other fighting game has) places it closer to Divekick than any other fighter, while teaching fighting game concepts such as spacing, whiff punishing, and shimmies. However, its simplicity means that it faces the same problem Divekick has. For one, the lack of jumping means you will never learn how to deal with jump-ins in Footsies, making it fail as a gateway to other fighting games, as knowing how to antiair is a very basic thing in a fighting game. Nevertheless, this is probably the closest game to the vision of the proposed fighting game.

What could be taken from this game

The simplicity of this game is a very suitable approach for this project, its simple nature making it instantly incredibly understandable for everyone, while teaching important concepts such as spacing and whiff punishing. Thus, a lot of design concepts for this project may be lifted from Footsies.

What are the Essential Features?

One indispensable feature of the solution would be being able to have a health bar displayed on screen, for players to know how much life there is remaining for both their character and the opponent's character. This is important as it is a core part of strategy for players to base their gameplay on, as well as being a staple for fighting games as a whole, fulfilling the purpose of this project to be a gateway into general fighting games.

Another feature that this game cannot go without would be the ability to use buttons to move, jump, attack, and block. This is to allow players to play the fighting game, as these are core parts of the fighting game genre.

Another very important feature will be the introduction of a training mode, in order to test combos and movement easily by yourself. This is very important as it streamlines the process of improving by providing an environment to easily test out situations. Furthermore, its existence is a staple in modern fighting games.

The final vital feature for this game will be the addition of the ability to play online, via delay based netcode. This is because for many individuals, the social aspect of fighting games is one they cannot as easily access in person, and in order to get games, they must play it online. This therefore removes the limitation offline only games face in terms of their reach.

What are the Limitations of this implementation?

The first limitation of this approach is the choice of delay based netcode over rollback based netcode. Rollback accommodates for many more types of connections and allows for those connections between clients to feel smoother. However, its introduction to this project would take up vast amounts of development time, due to its incredibly complex nature regarding actions such as rollback frames, often requiring a large, dedicated team to work on rollback netcode.

Another major limitation of this project will be the lack of choice in characters. Like footsies, this project is intended to be a fairly simple game to play with casuals, and the development time surrounding having multiple characters, along with their issues regarding balancing, will mean that this project will have solely one character.

A third limitation will be the platforms that it is on. It will only be accessible on computer, due to the control scheme it that it employs. This will reduce its reach compared to a game that is available on all platforms such as mobile or console.

Requirements of the Solution

Hardware Requirements

Number	Requirement	Justification
--------	-------------	---------------

1	Must be able to run on most modern computers.	this game is meant to be aimed towards casual gamers, and most casual gamers lack strong PCs, often having underpowered laptops.
2	Must be able to be controlled by a keyboard	Everyone with a computer will have a keyboard to play the game with, preventing the game from being kept from users who lack a controller.
3	Must be able to run on windows	That is the most popular platform

Software Requirements

Number	Requirement	Justification
1	Must be able to run smoothly on most computers	This is because poor performance can cause stakeholders to be unable to play the game properly.
2	Must be able to run parallel with other applications	This is because people want to be able to use other aspects of their computer rather than just playing the game.
3	The netcode must work between different players on differing networks smoothly when they have good connection	This is in order to provide a smooth and fun experience to stakeholders, as a bad connection can make an otherwise fun game unbearable, especially in the fighting game genre.

General Requirements

In order to figure out the general requirements of the solution, I made a questionnaire to find out what they wanted.

1. How often do you play fighting games?

Never ☐ Rarely ☐ Sometimes ☐ Often ☐

Please respond with how much you agree with these statements:

2. I find blocking in fighting games difficult to get used to

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

3. I often struggle with combos in fighting games

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

4. I find being stuck on defence enjoyable

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

5. A simple fighting game makes it easier for me to engage with the gameplay

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

6. Playing fighting games online is important to me

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

7. Being able to do a flashy combo feels cool to me

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

I shared this questionnaire with my shareholders, and placed their responses within this table:

Question	[StakeHolder]	[StakeHolder2]	[StakeHolder3]
1	Rarely	Never	Sometimes
2	Disagree	Heavily Agree	Agree
3	Agree	Heavily Agree	Agree
4	Disagree	Heavily Disagree	Disagree
5	Disagree	Heavily Agree	Agree
6	Agree	Disagree	Agree
7	Agree	Agree	Heavily Agree

Based on these results from the shareholders, I came up with a list of general requirements to meet their needs:

Number	Requirement	Justification
1	Super Moves, tied to a meter that fills up as the game goes on, and can be activated by pressing two buttons at once.	My stakeholders all agreed that flashy combos were important to feeling cool within a fighting game, however, they also had noticeable problems with executing combos within a fighting game. Therefore, an easy solution to this seemingly counterintuitive issue is to balance flashiness with ease of use, by introducing super moves. When tied to a meter, super moves will give stakeholders the ability do technically simple combos that are able to look flashy with a super attack. This will provide the catharsis of a flashy combo, while allowing any player to do the combo.
2	Simple Controls, with only a few buttons and a lack of motion inputs.	My stakeholders generally seem to believe a simpler fighting game execution-wise will allow for them to get to grips understanding fighting game concepts quicker. As this game is intended to act as a basic gateway to fighting games as a genre, reaching intentionality in gameplay early on is an important aspect of the design of the game.
3	Reversal attacks that can reset the situation, inputted with a singular button	My stakeholders seem to generally dislike being forced into a defensive position for extended periods of time and find blocking tedious. As the goal of this game is to prepare players for harder fighting games, the removal of blocking is not a direction the game will be able to go in, however, introducing reversals gives the player a way to end the opponent's offence. This introduces an easy but risky way to stop offence from the opponent, which meets the Stakeholders issues with blocking and being stuck in defence.
4	A basic tutorial page, explaining the mechanics of the game	My stakeholder's age range is very large, and as this game aims to get people into the fighting game genre, it cannot assume that basic fighting game concepts, like spacing or blocking are understood. Therefore, a comprehensive tutorial for how to play the game is a must, to reach the needs of stakeholders new to fighting games.
5	The ability to get online matches easily	Stakeholders must be able to get a match with a person they know very easily, through the sharing of an ID. This is as for many groups of people who are not used to playing online, anything more complex than that may end up being confusing and tedious.
6	The ability to access all	Stakeholders must be able to use the menus easily and efficiently.

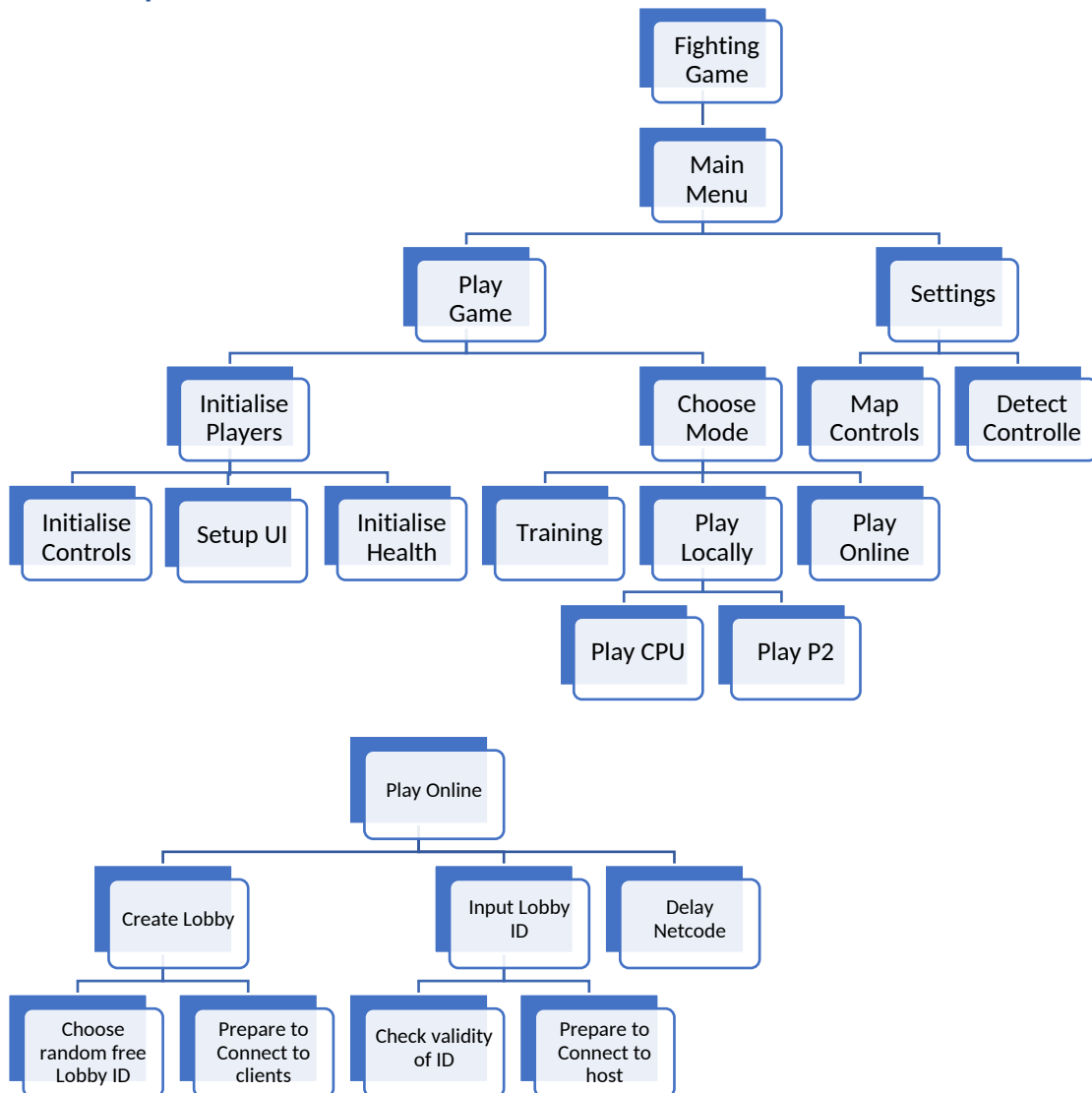
	content easily	
--	----------------	--

Success Criteria

Number	Criteria	Justification
1	When the HP hits 0 for either side, a winner is declared.	These are the conditions for winning or losing a fighting game, and thus must be within this project
2	There should be a menu including options for playing offline, playing online, and playing in training mode.	The players should be able to choose whether to play with the person next to them, play online, or practice in training
3	The game must be playable for both players 1 and 2, with both having keys binded for their respective control schemes.	This is required for the fighting game to introduce the genre to players, as every fighting game has both sides able to play the game.
4	The game must respond to key presses for relevant keys	This is required for the game to feel responsive and not buggy
5	You must be able to have a rematch option	This is required as to not break the game flow of playing with another player
6	The game must be playable with delay netcode	This is to allow players who do not have someone to play with at home to play with someone on the internet
7	The game must have a tutorial page	This allows beginners to the genre to understand how the game works
8	The game must be able to run at a locked frame rate on the average computer	This ensures consistency with how the game works, preventing slowdowns or speedups from altering the flow of the game, and providing a worse experience.

Design

Decomposition



Main Menu

The menu the game starts at, and where the player picks options for gameplay. This is needed as users will otherwise be unable to navigate the game without a main menu.

Settings

The menu where you can set your controls and your controller. This is needed as it is required by [Hardware Requirement 2](#), by allowing people with any number of peripherals, from controllers to keyboards, to bind their own controls.

Choose Mode

Submenu that allows you to choose between training mode, local play, and online play. This is required as [General Requirement 6](#) asks for a menu that allows you to

choose between training, local play, and online, as well as **General Requirement 5**, which requires easy to access online play for all users.

Initialise Players

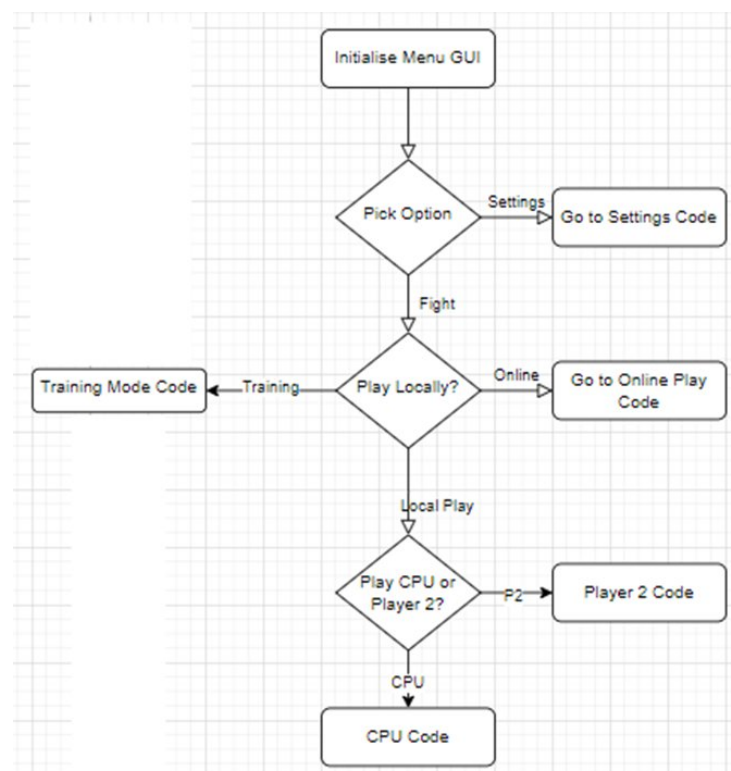
Sets up the game environment when the game starts. This is required for the game to be playable, including methods such as initialising controls.

How is each problem solved?

Pseudo-code and flowcharts will be used to illustrate approximately how each of these broken-down problems will be solved during development.

Main Menu

```
function main_menu:
    chosen_option = init_main_gui()
    #the gui returns a value when an option is picked
    if chosen_option == 0:
        #if the option chosen is option 1, goes into submenu for online
        play_option = init_online_menu_gui() #Online
        if chosen_option == 0: #play online
            online() #Goes to online submenu
        else if chosen_option == 1: #play training
            training()
        else: #local play
            play_option = init_local_menu_gui() #Local
            if chosen_option == 0: #play against another player
                initialise_play(0) #initialises gameplay
            else: #cpu play
                initialise_play(1)
    else:
        settings() #goes into settings
```



Initialise Players

Players will be coded as objects, using OOP. This is in order to reduce the amount of repeated code, allowing easier separation of variables between the players, and introducing character specific methods. Here is an example of how the constructor method for the player class may look

```
class player:
    function __init__(self, player_number):
        self.__p_num = player_number
        self.__mhp = 500
        self.__hp = 0
        self.__button_mapping = default_mapping[player_number - 1]
        self.__hitstun = 0
        self.__blockstun = 0
        self.__attack_type = 0
        self.__is_air = False
        self.__speed_x = 0
        self.__speed_y = 0
        self.__spriteID = 0
```

This only contains some of the class variables that may be needed upon instantiation. Each of these variables are made private, being accessed with getter and setter methods to ensure the variables are not erroneously changed outside of the class.ss

Initialise Health

```
function reset_health(self):
    max_hp = self.get_max_hp()
    self.set_hp(max_hp)
```

This pseudocode method for the player class will be called for both player objects to reset the health at the start of rounds.

Mapping Controls

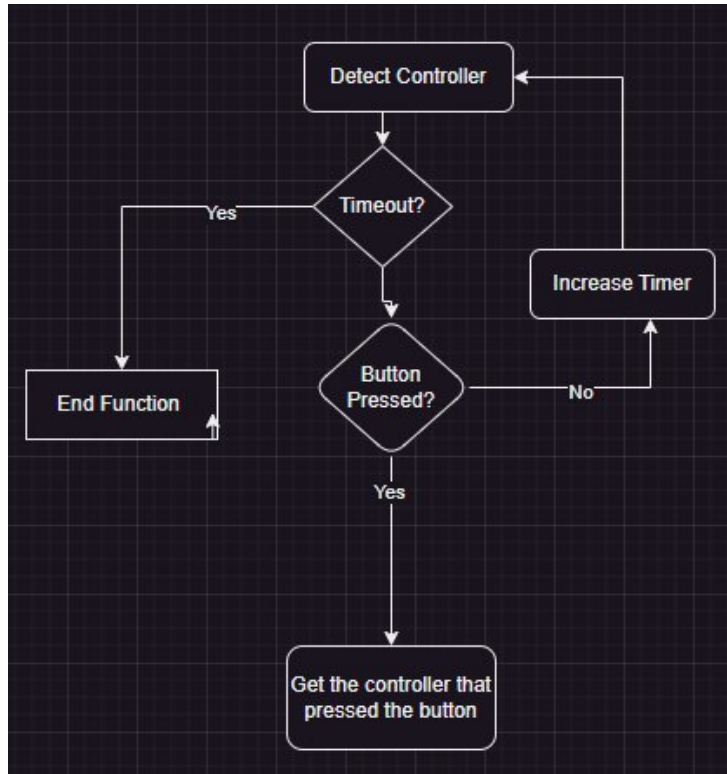
```
function map_controls(self):
    button_mappings = [
        ["Left", ""],
        ["Right", ""],
        ["Up", ""],
        ["Attack", ""],
        ["Special", ""]
    ]
    for i in range(0, len(button_mappings):
        print_next_prompt(button_mappings[i][0] + ": ")
        next_button = ButtonHandler.next_button()
        button_mappings[i][1] = next_button

    self.set_controls(button_mappings)
```

This pseudo-code method explains how button remapping will broadly work. It will iterate through each button in a list, and the next button pressed by the controller will be mapped as the respective mapping for the button. This is a method, so it is specific

to the 2 player objects, which are in the player class. This allows for a quick and simple setup of custom controls, separate for each player.

Detect Controller



```
function detect_controller(self):
    detect_controller_gui()
    timeout_period = 15*30 //Controller detector times out after 15 seconds
    timeout_counter = 0
    while button_pressed != true or timeout_counter >= timeout_period:
        timeout_counter += 1
        wait_frames(2)
    if timeout_counter >= timeout_period:
        return
    self.__controller = ButtonHandler.get_button_owner()
```

What this pseudo-code shows is checking for whether any button is pressed. If a button is pressed either by the keyboard or by a controller, the peripheral that was pressed will be set as the controller for the player. If no button is pressed, it will wait until the timeout.

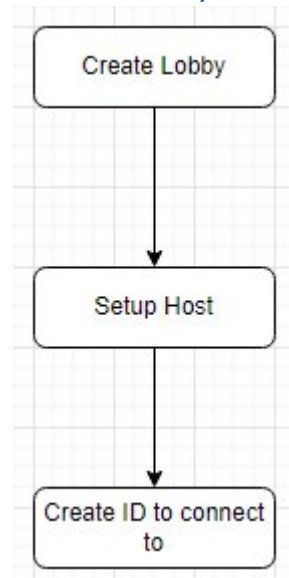
Sprite Handling

```
function set_animation(self, animID):
    for i in range(0, len(sprite_list[animID])):
        self.set_sprite(sprite_list[animID][i])
        wait_frames(1)
    self.set_animation(0) //Recursively calls the set_animation for the idle animation
```

What this pseudo-code represents is how animations will be handled. Each animation has an animID, which is an index in the spritelist (a 2D array) that has the list of the links for all the frames of the animation. The set_animation code iterates through this index in the spritelist in order to change the sprite shown onscreen every frame. After

it is finished, it is called recursively for the idle animation, to ensure that animation loops.

Create Lobby



```
function create_lobby(ip):  
    host = setup_host(ip)  
    id_num = 10000  
    while is_used(id_num) == true:  
        id_num += 1;  
        if id_num > 99999:  
            return false //Used incase all IDs are in use (unlikely)  
    setup_lobby_id(host, id_num)  
    return true
```

This pseudocode shows roughly how lobby IDs will be created, attempting to find the next free ID.

Usability Features

Buttons

Having buttons for the users to press is very accessible, as it makes it very easy for users to provide information to the program of what they want to do.

Clear Graphical Interface

Having a clear graphical interface is incredibly important to the usability of the program, as users need to understand how to use the menu and understand the UI at a glance. Having all stakeholders understand and find the UI easy to grasp is therefore paramount to the project's success. Some mock-ups of the UI are shown below:

Main menu mock-up 1



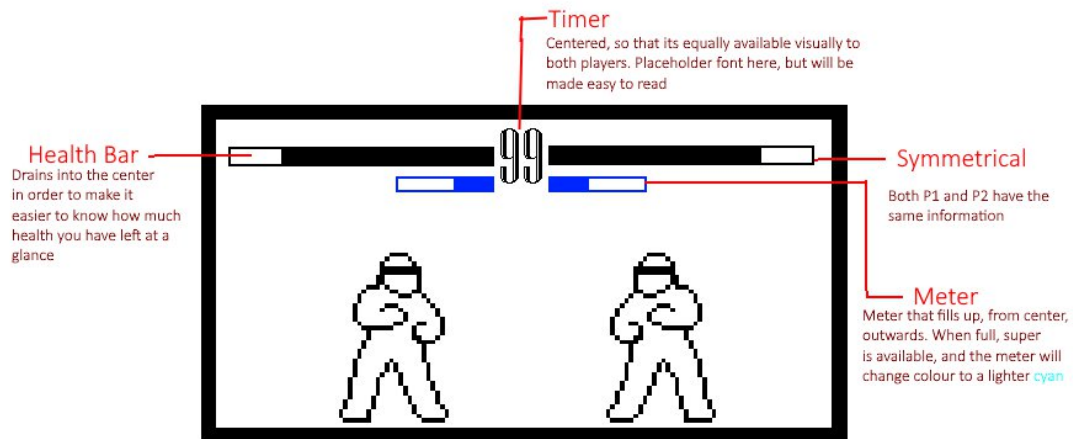
Stakeholders said they found this menu easy to understand but plain. It also causes no issues for the colourblind.

Main Menu mock-up 2



This menu mock-up is inspired by Foolsies' menu, making it easy to understand and visually better looking. Stakeholders said they enjoyed this menu more than mock-up 1.

In-game UI Mockup + Annotations



The in game UI is minimal but functional, containing just the elements required, Health Bar, Meter, and Timer. The UI is simple and easy to understand to stakeholders.

Colour Scheme

Stakeholders expressed interest in a simpler colour scheme, with black and white being a neutral colour scheme that is unaffected by colour blindness or other visual impairments.

Key Data Structures

Key Structure Name	Key Structure Data Type	Justification
self.player_hp	Attribute – Integer	Used to keep track of one player's HP. If it reaches zero, the round ends
self.player_mhp	Attribute – Integer	What the player's HP is set to at the start of each round
self.button_mapping	Attribute – Array	Used to keep track of the button mappings of one player, in order to support custom control schemes.
self.hitstun self.blockstun	Attribute – Integer	Used to determine how long a player stays in the blockstun or hitstun states. When greater than 1, decrements every frame
self.attack_type	Attribute – Integer	Used to determine which attack the character is doing, in order to determine parts of the move such as animation,

		hitbox, total frames, and whether it can cancel into anything
self.is_air	Attribute – Boolean	Used to detect whether in the air, which alters how movement works, and what attacks are done
self.speed_x self.speed_y	Attribute – Float	Used to set the speed the player moves every frame
self.spriteID	Attribute – Integer	Used to keep track of what sprite needs to be displayed on the screen on any given frame
preferences.txt	Text File	Used to keep track of any settings set within game between sessions.
buttons.txt	Text File	Used to keep track of button mappings between sessions.

Test Data

Test Category	Test Number	Test Type	Test Data	Justification	Expected Outcome
Clicking Buttons on main menu	1	Valid	Press centre of all 4 buttons on main menu	To ensure buttons work as intended for regular behaviour	Button is pressed, action occurs based on the button clicked (button config opened or in game opened)
	2	Borderline	Press left edge of each	To ensure even pressing the edges work	Button is pressed, action occurs based on the button clicked (button config opened or in game opened)
	3	Invalid	Press on empty space	To ensure nothing occurs when areas where buttons do not exist	Nothing occurs
	4	Invalid	Press outside of the game window	To ensure that it only checks for mouse clicks when the window is in focus and being clicked on	Nothing occurs
Mapping controls	5	Valid	Press settings button on main menu. Press button config for player 1. Input separate buttons for each button for button mapping. Then go ingame and use every button mapping to ensure it works as intended	To ensure button mapping works as intended	Button mapping finishes, and the characters behave as intended ingame
	6	Borderline	Set the left and right buttons as the same key, then test ingame	to ensure no SOCD (Simultaneous Opposite Directions) bugs occur, such as being able to block both ways	SOCD leads to neither direction being input, and no attacks are blocked
	7	Invalid	Edit the buttons.txt to remove a button binding	To ensure that the game falls back on defaults when the buttons.txt is invalid	The game proceeds with default controls
In-game Movement with controls	8	Valid	Input left and right buttons. Then input up and then down.	To ensure movement works	The character moves left and right, then jumps, then crouches.
	9	Valid	Input an attack button, then a heavy attack button, then a special button	To ensure attacks work	The character attacks, then uses a special
	10	Valid	Input a jump, then attack	To ensure jumping, and jump	The character jumps, then uses

				attacks work	a jump attack
	11	Valid	Hold left and input attack, then hold right and input attack, then hold down and input attack. Hold left and input special, hold right and input special, then hold down and input special.	To ensure directional specials work	The character uses different moves based on the direction held during the attack/special
	12	Borderline	Input left and right at the same time. Then input up and down at the same time	To ensure SOCD cleaning is done at a software level	The character stays within the idle pose
	13	Borderline	Input attack, heavy attack, and special at the same time	To ensure pressing multiple buttons does not cause anything	Nothing happens
	14	Invalid	Press an unbinded key	To ensure unbinded keys do not affect the gamestate	Nothing happens
	15	Valid	Double tap forwards. Then Double tap backwards.	To ensure forward and backdashes work	The player forward and backdashes
Player interactions	16	Valid	Both players attack on the same frame	To ensure trades are handled properly	Both characters enter hitstun
	17	Valid	1 player attacks the other in range of their hurtbox. Then walks backwards and attacks outside of their hurtbox	To ensure attacks collide correctly	The opponent only enters hitstun once
	18	Valid	Player 1 presses crouching light, and player 2 holds down and back. Player 1 presses crouching light again, but Player 2 holds just back. Repeat with standing light and standing heavy. Then Player 1 crosses up player 2 with jumping light, player 2 does not switch direction blocking. Repeat this situation but player 2 does switch direction blocking	To ensure high-low blocking and crossups work as intended	P2 blocks, then gets hit, then blocks s.light twice, then blocks s.heavy while stand blocking but not while crouch blocking. Then P2 gets hit after the first crossup but not the second.
Training mode	19	Valid	Set to frame by frame, press space 4 times. Then hold space for 30 seconds. Then do not touch space for 1 minute. Then	To ensure the game correctly responds to frame by frame	The game progresses 4 frames when space is tapped. Then runs at full speed for 30 seconds while

			hold space again.		space is held. Then stops progressing when space is not pressed
	20	Valid	Set enable hitboxes on, play for 30 seconds. Then set enable hitboxes to false	To ensure the game correctly draws hitboxes and hurtboxes, and correctly disables them when they are disabled	Hitboxes show correctly, then don't show at all when disabled
	21	Valid	Set to block always. Then use crouching light, then standing heavy, then jump over them and use jump heavy.	To ensure the block always setting always blocks everything in all scenarios	The player 2 dummy blocks all attacks
	22	Valid	Set to block after first hit. Then use standing light -> delayed side special	To ensure the block after first hit setting works and has the cpu block after being hit once	The player 2 dummy gets hit by standing light, and then blocks the delayed side special
	23	Valid	Set reversal to dragon punch, and block to none. Then hit with standing light	To ensure the reversal is done out of hitstun for the CPU when block is set to none	The CPU will use DP after being hit
	24	Valid	Set reversal to dragon punch, and block to always. Then hit with standing light	To ensure the reversal is done out of blockstun for the CPU when block is set to none	The CPU will use DP after blocking
	25	Valid	Set reversal to dragon punch, and block to after first hit. Then hit with standing light. After that, use standing light -> delayed side special	To ensure the reversal is only done out of blockstun when the CPU is set to block after first hit	The CPU will only use DP after blocking, not after hitstun
HP Bar	26	Valid	Hit the opponent	To ensure their HP bar goes down	The opponent's HP bar should decrease
	27	Valid	Hit the opponent until their HP hits 0	To ensure the game ends when HP hits 0	The game ends while saying which player wins
Pause Menu	28	Valid	Press escape to open the pause menu, then press escape to close it	To ensure escape correctly opens and closes the menu	The pause menu should open and close
	29	Valid	Press escape to pause, then press light attack to open the movelist. Then press escape to return to ingame	To ensure the movelist works	The pause menu should open and then show the movelist
	30	Borderline	Press escape to pause, then tap down 4 times, then press attack	To ensure that you cannot go further than the third menu option	The pause menu should open, then you should move to the bottom option, and then

					go to the main menu
	31	Valid	Press escape to pause, then tap down once, then press attack	To ensure the go back to game option works	The pause menu should open and close
VS CPU	32	Valid	Play against the computer for 6 matches, with the computer playing as all 3 characters twice. Use the rematch button to play against the same character twice.	To ensure the computer works properly with all characters, and if played repeatedly	The computer should play all 6 matches without getting stuck.

Post Development Survey

In order to prepare to check how effectively this project achieves its goals after development ceases, I have prepared a Post Development Survey for my stakeholders:

Please respond with how much you agree with these statements:

1. Installation was quick and simple

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

2. I was able to navigate the menu easily

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

3. I understood the controls

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

4. I found playing the game enjoyable

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

5. I was able to improve at the game as I kept playing

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

6. I enjoyed the game's artstyle and presentation

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

7. I found executing simple combos easy

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

8. I found the game fair and balanced

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

9. I found the training menu useful

Heavily Disagree ☐

Disagree ☐

Agree ☐

Heavily Agree ☐

10. I found playing online enjoyable

Heavily Disagree ☐

Disagree ☐

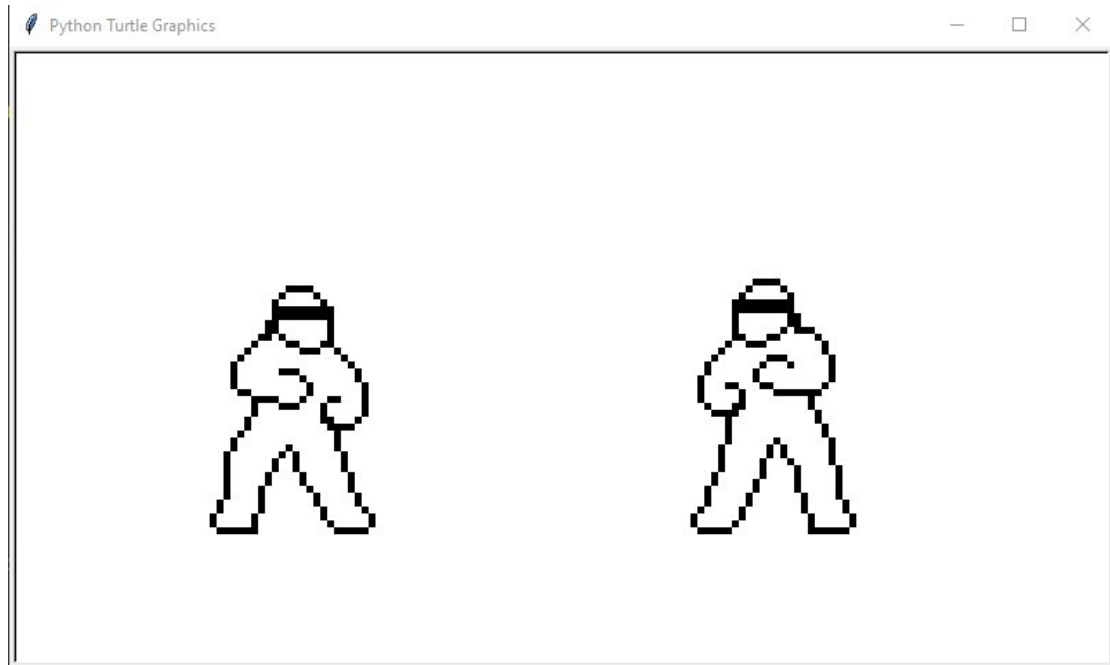
Agree ☐

Heavily Agree ☐



Development

Prototype 1 - Walking



The first prototype's aim was to have 2 controllable characters that can move left and right, with animations. This was achieved through the use of the tkinter, turtle, Pillow, keyboard, os, and time libraries:

```
import turtle
import os
import time
import keyboard
from tkinter import PhotoImage
from turtle import Shape
from PIL import Image, ImageTk
import image_reverser
```

Object Oriented Programming was used in the creation of the players, in order to carry over code from player 1 to player 2, ensuring both worked identically at all times

Constructor Method

```
class player:
    def __init__(self,
        playerNum: int,
        x: int, y: int,
        #sprite, frame, animList,
        controls: list, #Controls expressed as [left, right, up, down, attack, special]
        Left: bool
    ):
        self.playerNum = playerNum
        self.x = x
        self.y = y
        self.isJump = False
        self.isHitstun = False
        self.is_left = Left
        self.moveXThisFrame = 0.0
        self.moveYThisFrame = 0.0

        #Sprite Init
        self.frame = 0
        self.animListID = 0
        self.sprite = "sprites/Idle_0.gif"

        #Turtle Setup
        self.turtle = turtle.Turtle()
        self.turtle.speed(0)
        self.turtle.penup()
        self.turtle.goto(x, y)
        self.turtle.shape(self.sprite)
        self.animate()

        self.controls = controls
```

Used during update method to update position

All 3 used in the animation system

Sets up the turtle

Lowest level of sprite control, usually declared like this in order to update the sprite frame

List containing keybinds, makes it easier later to impl custom controls later on

This constructor method contains most variables relating to each player individually.

Update Method

```
def update(self):
    self.animate()

    if keyboard.is_pressed(self.controls[0]):
        self.left()
    elif (self.animListID == get_anim_ID("WalkL") and self.is_left != False) or (self.animListID == get_anim_ID("WalkR") and self.is_left != True):
        self.set_new_anim_by_ID()

    if keyboard.is_pressed(self.controls[1]):
        self.right()
    elif (self.animListID == get_anim_ID("WalkR") and self.is_left != True) or (self.animListID == get_anim_ID("WalkL") and self.is_left != False):
        self.set_new_anim_by_ID()

    if self.moveXThisFrame != 0 or self.moveYThisFrame != 0:
        x = self.x
        y = self.y
        newVal = x
        newVal = y
        if (x+self.moveXThisFrame < BOX_SIZE[0] or -(x+self.moveXThisFrame) > BOX_SIZE[0]):
            newVal = x+self.moveXThisFrame
        if (y+self.moveYThisFrame > 0):
            newVal = y+self.moveYThisFrame
        #print(f"({newVal}), ({newVal})")
        self.x = newVal
        self.y = newVal
        self.turtle.setpos(self.x, self.y)
        self.moveXThisFrame = 0.0
        self.moveYThisFrame = 0.0

    char_pos[self.playerNum - 1][0] = self.x
    char_pos[self.playerNum - 1][1] = self.y
    #print(char_pos)
```

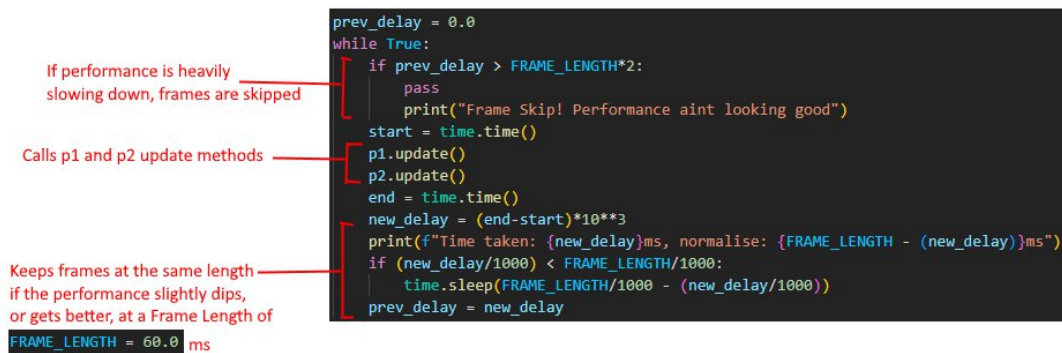
Calls Animation Update method

Usage of keyboard module, determines what happens when "Left Key" and "Right Key" are down, and ends walk animation when they stop walking

Position update system. Done this way to ensure that you move in increments of 1 frame, and you dont move between frames. Also makes it easier to handle positioning

Current pos reported to a global 2D list, used for collisions

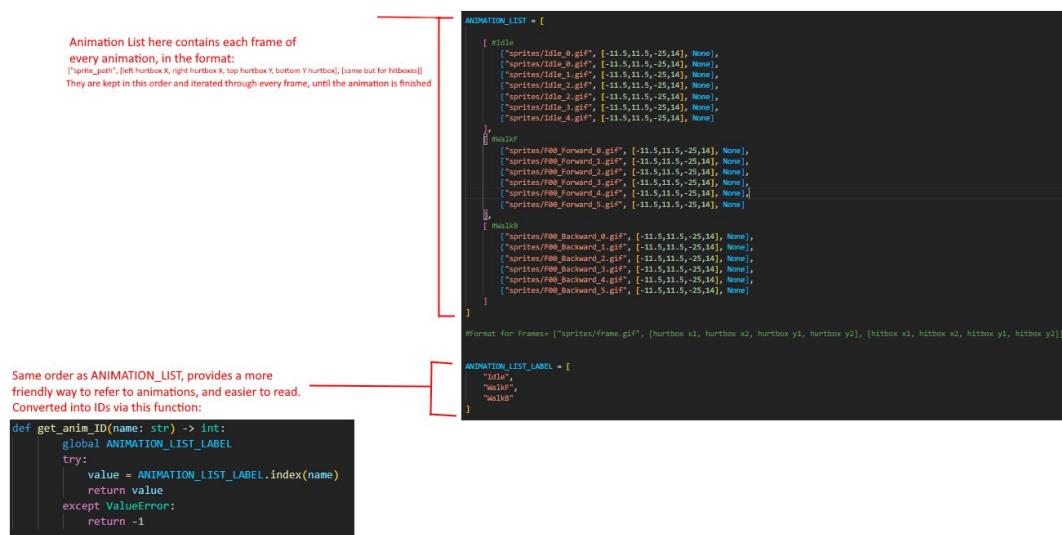
This update method is called every frame, and updates positioning, deals with controls, and calls animation to update. This is called below:



The FRAME_LENGTH compensation ensure it runs at a locked frame rate, which fulfils success criteria 8 (The game must be able to run at a locked frame rate on the average computer).

Animation Handling

Animation is handled through the combination of 2D arrays, lookups, player class methods to iterate through frames, and methods to set animations

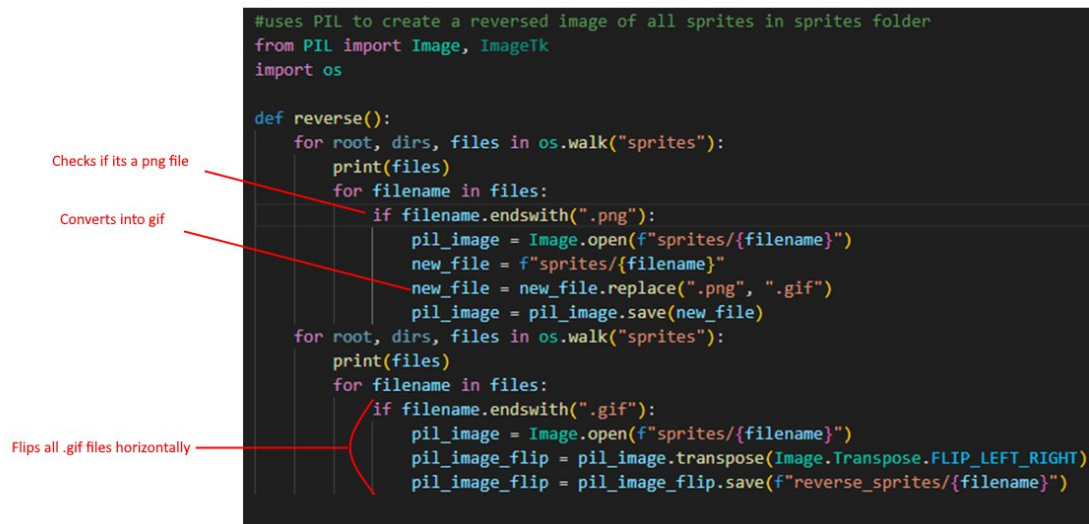


Above is the ANIMATION_LIST, which contains all the data related to sprites and frames within it. This is the core 2D array that is referred to throughout the animation methods. Its simple nature makes it incredibly simple to add animations.



This is the procedure used to pull sprites from the sprites and reverse_sprites folder, and scale them up.

reverse_sprites are generated automatically on startup, thanks to image_reverser.py, which makes use of Pillow to programmatically reverse all .gif files in the sprites folder, and convert .png files into .gif files



The image shows a code editor with Python code for processing sprites. Three red annotations with arrows point to specific lines of code:

- Checks if its a png file**: Points to the line `if filename.endswith(".png"):`.
- Converts into gif**: Points to the line `new_file = new_file.replace(".png", ".gif")`.
- Flips all .gif files horizontally**: Points to the line `pil_image_flip = pil_image.transpose(Image.Transpose.FLIP_LEFT_RIGHT)`.

```
#uses PIL to create a reversed image of all sprites in sprites folder
from PIL import Image, ImageTk
import os

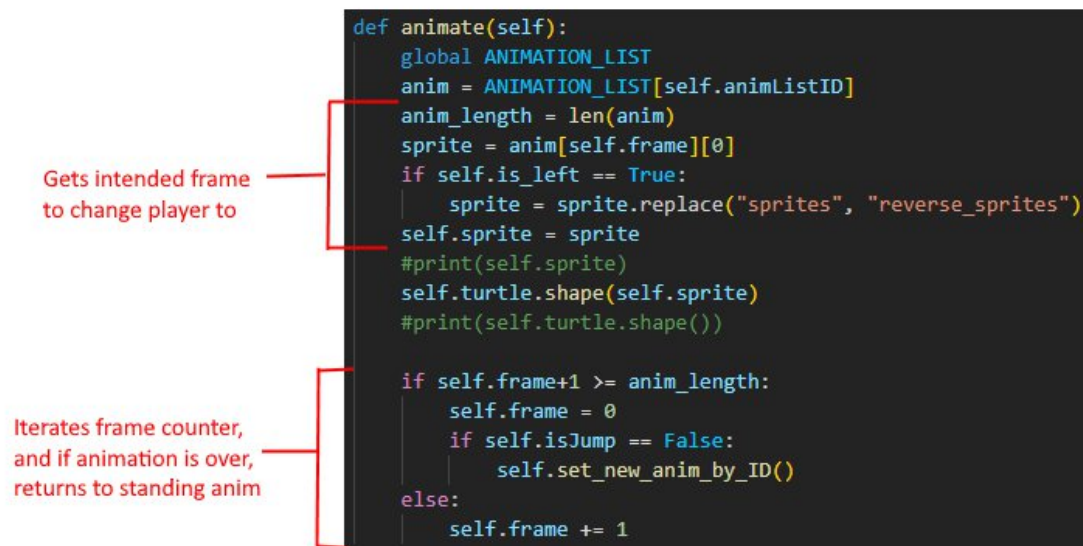
def reverse():
    for root, dirs, files in os.walk("sprites"):
        print(files)
        for filename in files:
            if filename.endswith(".png"):
                pil_image = Image.open(f"sprites/{filename}")
                new_file = f"sprites/{filename}"
                new_file = new_file.replace(".png", ".gif")
                pil_image = pil_image.save(new_file)
    for root, dirs, files in os.walk("sprites"):
        print(files)
        for filename in files:
            if filename.endswith(".gif"):
                pil_image = Image.open(f"sprites/{filename}")
                pil_image_flip = pil_image.transpose(Image.Transpose.FLIP_LEFT_RIGHT)
                pil_image_flip = pil_image_flip.save(f"reverse_sprites/{filename}")
```

This is called within main.py at the start:

```
import image_reverser

image_reverser.reverse()
```

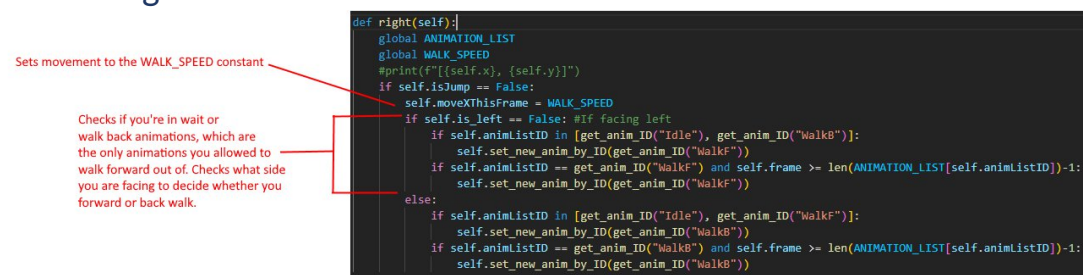
This method is ran every frame, and is what causes the animation to play



This method sets the new animlistID for a new animation, returning -1 if the ID is invalid, to prevent any crashes from invalid animation names.

```
def set_new_anim_by_ID(self, id=0, frame=0):
    if id == -1:
        return
    self.frame = frame
    self.animListID = id
```

Left and Right Walk Methods




```

def left(self):
    global ANIMATION_LIST
    global WALK_SPEED
    if self.isJump == False:
        self.moveXThisFrame = -WALK_SPEED
        if self.is_left == True: #If facing left
            if self.animListID in [get_anim_ID("Idle"), get_anim_ID("WalkB")]:
                self.set_new_anim_by_ID(get_anim_ID("WalkF"))
            if self.animListID == get_anim_ID("WalkF") and self.frame >= len(ANIMATION_LIST[self.animListID])-1:
                self.set_new_anim_by_ID(get_anim_ID("WalkF"))
        else:
            if self.animListID in [get_anim_ID("Idle"), get_anim_ID("WalkF")]:
                self.set_new_anim_by_ID(get_anim_ID("WalkB"))
            if self.animListID == get_anim_ID("WalkB") and self.frame >= len(ANIMATION_LIST[self.animListID])-1:
                self.set_new_anim_by_ID(get_anim_ID("WalkB"))

```

These methods control walking left and right.

Build.py

When the stakeholders were asked on whether they had python installed on their computer, their response was unanimously “what’s python?”. This is a major issue if the game is distributed as a .py file. Therefore, using pyinstaller and py2exe, we can package the codebase into an exe file standalone from python.

Empties folder through
os.unlink and shutil.rmtree

```

from distutils.core import setup
import py2exe
import shutil
import os

def empty_folder(folder):
    for filename in os.listdir(folder):
        file_path = os.path.join(folder, filename)
        try:
            if os.path.isfile(file_path) or os.path.islink(file_path):
                os.unlink(file_path)
            elif os.path.isdir(file_path):
                shutil.rmtree(file_path)
        except Exception as e:
            print('Failed to delete %s. Reason: %s' % (file_path, e))

empty_folder(r'dist')
shutil.copytree(r"sprites", r"dist\sprites")
shutil.copytree(r"reverse_sprites", r"dist\reverse_sprites")

```

sets up py2exe

```

setup(
    options = {'py2exe': {'bundle_files': 1, 'compressed': True}},
    windows = [{'script': "main.py"}],
    zipfile = None,
)

```

This produces an exe file with all dependencies as dll files in this file structure:

lib	28/08/2023 17:15	File folder	
reverse_sprites	28/08/2023 14:11	File folder	
sprites	28/08/2023 15:10	File folder	
libcrypto-1_1.dll	30/08/2021 21:37	Application exten...	3,350 KB
libffi-7.dll	30/08/2021 21:37	Application exten...	33 KB
libssl-1_1.dll	30/08/2021 21:37	Application exten...	679 KB
main.exe	28/08/2023 17:15	Application	11,070 KB
tcl86t.dll	30/08/2021 21:37	Application exten...	1,666 KB
tk86t.dll	30/08/2021 21:37	Application exten...	1,434 KB

Alpha Testing

As this was a very early stage of development, the alpha testing was minimal

Test Number	Test Description	Success?	Details
8	Input left and right buttons	Yes	--
12	Input left and right at the same time.	Yes	--
14	Press an unbinded key	Yes	--

Prototype 2 - Jumping



The second prototype's goal was to have the bases of movement down, with jumping and crouching, as well as all that these entail (such as adding collisions to prevent characters phasing into each other).

Correcting Direction Faced Method:

```
def check_correct_side(self):  
    global ANIMATION_LIST  
    global ACTIONABLE_LIST  
    #print(f"[{self.x}, {self.y}]")  
    if self.isJump == False and self.animListID in ACTIONABLE_LIST:  
        if self.is_left == True and self.x < char_pos[-(self.playerNum)][0]:  
            self.is_left = False  
            self.set_new_anim_by_ID()  
        if self.is_left == False and self.x > char_pos[-(self.playerNum)][0]:  
            self.is_left = True  
            self.set_new_anim_by_ID()
```

Checks if you are in an actionable state

Compare yours and opponents x pos

Sets to default anim with new direction

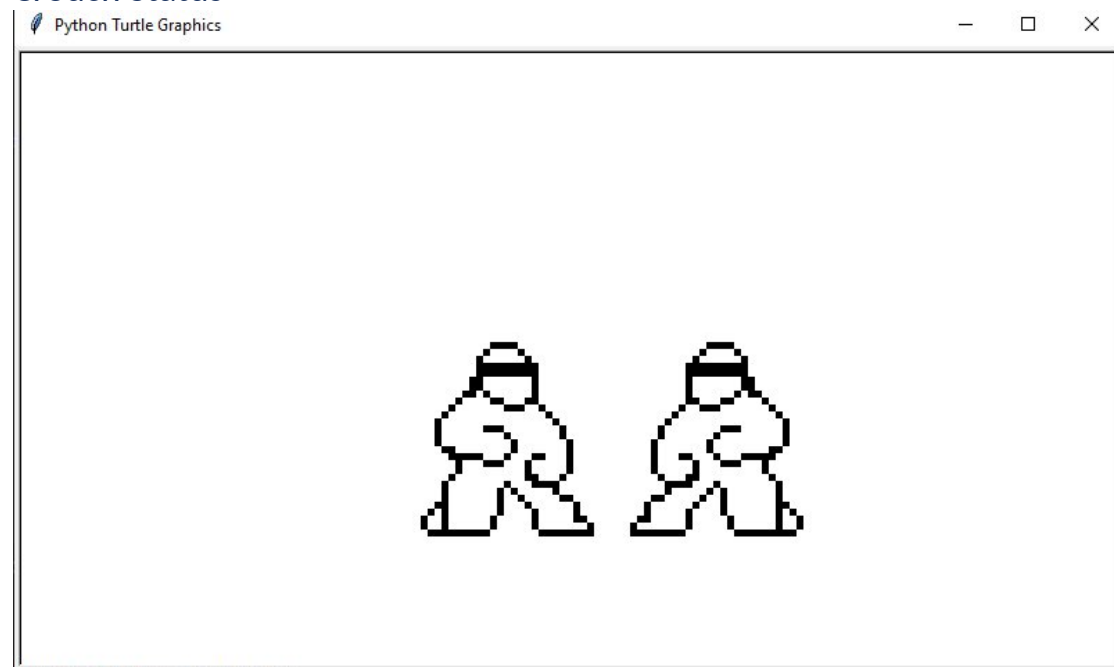
This method checks if the opposing player's position is greater or less than their position, and if it is and the player object is grounded and in an actionable status, they will turn around.

In order to get this to maintain the crouch status when they turn around, the `set_new_anim_by_ID` method had to be slightly rewritten for the default value to default to different animations based on the state.

Default animation code.
sets animation based on their
status, like if they're in the air,
or if they're crouching.

```
def set_new_anim_by_ID(self, id=-2, frame=0):  
    if id == -1:  
        return  
    if id == -2:  
        self.frame = frame  
        if self.isJump:  
            if self.animListID == get_anim_ID("JumpSquat"):  
                self.moveYThisFrame = JUMP_INITIAL  
            self.animListID = get_anim_ID("Air")  
            return  
        elif self.isCrouch:  
            self.animListID = get_anim_ID("CrouchWait")  
            return  
        else:  
            self.animListID = get_anim_ID("Idle")  
            return  
    self.frame = frame  
    self.animListID = id
```

Crouch Status



Crouching by tapping down was introduced in this prototype, and is activated and deactivated by the down key

```
if keyboard.is_pressed(self.controls[1]) and not keyboard.is_pressed(self.controls[2]):  
    self.down()  
elif self.isCrouch:  
    self.uncrouch()
```

Checks for if you're in the correct
animations, that can go into crouch,
and that you're not already in it

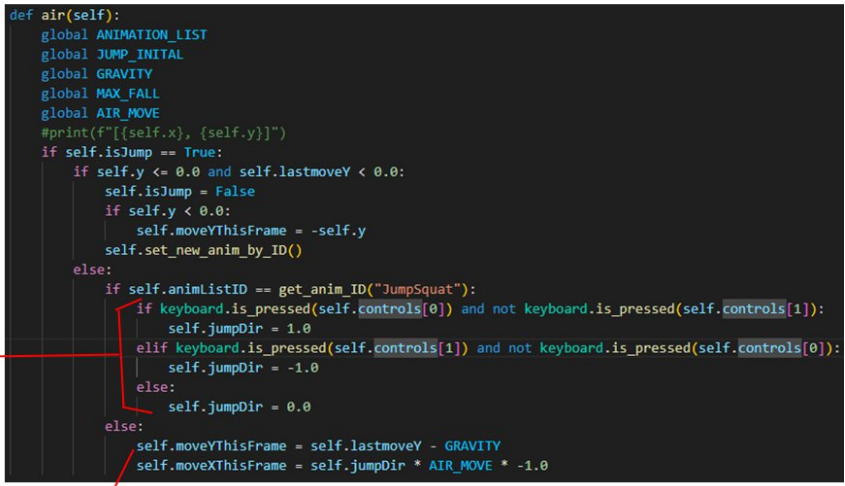
Runs when down is no longer pressed,
controlled by this script:

```
def down(self):  
    global ANIMATION_LIST  
    print(f"({self.x}, {self.y})")  
    if self.isJump == False:  
        if self.isCrouch == False and self.animListID in [get_anim_ID("Idle"), get_anim_ID("Walk"), get_anim_ID("Walk2")]:  
            self.isCrouch = True  
            self.set_new_anim_by_ID(get_anim_ID("Crouch"))  
  
def uncrouch(self):  
    global ANIMATION_LIST  
    print(f"({self.x}, {self.y})")  
    if self.isCrouch == True and self.animListID in [get_anim_ID("CrouchWait")]:  
        self.isCrouch = False  
        self.set_new_anim_by_ID(get_anim_ID("CrouchWait"))
```

These changes fulfill success criteria 3(The game must be playable for both players 1 and 2) and 4 (The game must respond to key presses for relevant keys), and all 3 hardware requirements (Must be able to run on most modern computers, Must be able to be controlled by a keyboard, Must be able to run on Windows) have been fulfilled by this prototype.

Jumping

Jumping is given unique momentum compared to crouching through the use of the air method:



```
def air(self):
    global ANIMATION_LIST
    global JUMP_INITIAL
    global GRAVITY
    global MAX_FALL
    global AIR_MOVE
    #print(f"[{self.x}, {self.y}]")
    if self.isJump == True:
        if self.y <= 0.0 and self.lastmoveY < 0.0:
            self.isJump = False
            if self.y < 0.0:
                self.moveYThisFrame = -self.y
            self.set_new_anim_by_ID()
        else:
            if self.animListID == get_anim_ID("JumpSquat"):
                if keyboard.is_pressed(self.controls[0]) and not keyboard.is_pressed(self.controls[1]):
                    self.jumpDir = 1.0
                elif keyboard.is_pressed(self.controls[1]) and not keyboard.is_pressed(self.controls[0]):
                    self.jumpDir = -1.0
                else:
                    self.jumpDir = 0.0
            else:
                self.moveYThisFrame = self.lastmoveY - GRAVITY
                self.moveXThisFrame = self.jumpDir * AIR_MOVE * -1.0
```

Sets jump direction to left or right

falls faster each frame, caused by the GRAVITY constant. Used to emulate real life gravity, and give the character a natural jump curve

Your jump direction is handled during jumpsquat before you begin jumping, based on whether you are holding left or right.

When you are in the air status, you constantly decelerate from your initial momentum, which is achieved by storing the previous speed on the last frame and subtracting the GRAVITY constant from it.

The initial momentum you get from jump is handled within the animation change script, when you change into the jump animation from jumpsquat, setting vertical momentum to the initial jump speed:

```

def set_new_anim_by_ID(self, id=-2, frame=0):
    if id == -1:
        return
    if id == -2:
        self.frame = frame
        if self.isJump:
            if self.animListID == get_anim_ID("JumpSquat"):
                self.moveYThisFrame = JUMP_INITIAL
            self.animListID = get_anim_ID("Air")
            return
        elif self.isCrouch:
            self.animListID = get_anim_ID("CrouchWait")
            return
        else:
            self.animListID = get_anim_ID("Idle")
            return
    self.frame = frame
    self.animListID = id

```

Alpha Testing

Alpha testing was much the same as prototype 1.

Test Number	Test Description	Success?	Details
8	Input left and right buttons. Then input up and down buttons.	Yes	--
12	Input left and right at the same time. Then input up and down at the same time	Yes	--
14	Press an unbinded key	Yes	--

Prototype 3 – Hitboxes & Hurtboxes

After feedback on movement from stakeholders, their primary concern with Prototype 2 was the jumps feeling way too floaty. This is fixed in this prototype, with adjustments to the JUMP_INITIAL and GRAVITY constants to enable a speedier jump.

The main goal for Prototype 3 was to finish movement, add the ability to attack, and add hitboxes and hurtboxes.

Forward and Back Dashes

Prototype 3 introduces the ability to dash, as moving across the screen was seen as “slightly slow” by stakeholders in previous testing for Prototype 2.

This was achieved through the creation of a “double tap” system to dash:

```
if keyboard.is_pressed(self.controls[0]) and not keyboard.is_pressed(self.controls[1]):
    self.left()
elif (self.animListID == get_anim_ID("WalkF") and self.is_left != False) or (self.animListID == g
    if not self.isHitstun:
        self.set_new_anim_by_ID() #Resets you to standing when left no longer pressed

if keyboard.is_pressed(self.controls[0]) and not keyboard.is_pressed(self.controls[1]):
    self.leftPressed = True
else:
    self.leftPressed = False

if keyboard.is_pressed(self.controls[1]) and not keyboard.is_pressed(self.controls[0]):
    self.right()
elif (self.animListID == get_anim_ID("WalkF") and self.is_left != True) or (self.animListID == ge
    self.set_new_anim_by_ID() #Resets you to standing when right no longer pressed

if keyboard.is_pressed(self.controls[1]) and not keyboard.is_pressed(self.controls[0]):
    self.rightPressed = True #Set so it can be used to check if the button has just been pressed
else:
    self.rightPressed = False
```

```
if self.backdashTimer > 0:
    self.backdashTimer -= 1
if self.forwarddashTimer > 0:
    self.forwarddashTimer -= 1
```

These lines of code are ran in update. The first image sets an attribute to True or False based on whether left or right are pressed. This is effective as these are ran *after* the self.left() and self.right() methods, meaning that if tapped for a frame, the leftPressed

attribute would be shown as false to the self.left() method. This produces the effect of allowing us to check whether the key was pressed on that specific frame rather than held down.

The second image shown above shows a timer that is run down every frame for the leniency of the double tap.

The code that determines when you do go into backdash is contained in the self.left() method:

```
if self.leftPressed == False:
    if self.forwarddashTimer == 0:
        self.forwarddashTimer = DASH_WINDOW
    else:
        self.forwarddashTimer = 0
        self.set_new_anim_by_ID(get_anim_ID("ForwardDash"))
        self.moveXThisFrame = -FDASH_SPEED
```

```
if self.leftPressed == False:
    if self.backdashTimer == 0:
        self.backdashTimer = DASH_WINDOW
    else:
        self.backdashTimer = 0
        self.set_new_anim_by_ID(get_anim_ID("BackDash"))
        self.moveXThisFrame = -BDASH_SPEED
```

These are kept within the logic for forward and back walks respectively. This code checks if you have just pressed forward that frame. If you have and the dash timer is 0, this is counted as your first tap, and the leniency window for a double tap is set. If the forwarddashTimer attribute is above 0 when this is checked, it is assumed this is your second tap, and you are sent into the Dash animation.

Momentum is handled within its own dash method:

```

def dash(self):
    global ANIMATION_LIST
    global ACTIONABLE_LIST
    if self.isJump or self.isCrouch or self.isHitstun:
        return

    side_mul = 1
    if self.is_left:
        side_mul = -1

    if self.animListID == get_anim_ID("ForwardDash"):
        if self.lastmoveX != 0:
            self.moveXThisFrame = self.lastmoveX - (FDASH_DECEL*side_mul)

            if abs(self.lastmoveX) - BDASH_DECEL < 0:
                self.moveXThisFrame = 0

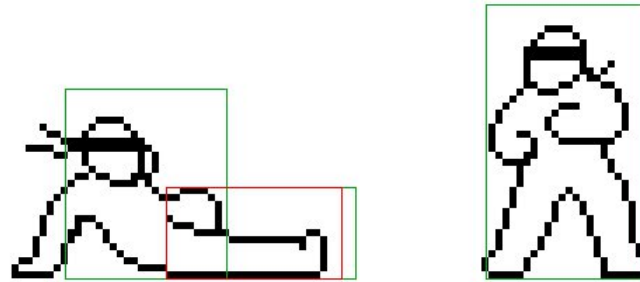
    if self.animListID == get_anim_ID("BackDash"):
        if self.lastmoveX != 0:
            self.moveXThisFrame = self.lastmoveX + (BDASH_DECEL*side_mul)

            if abs(self.lastmoveX) - BDASH_DECEL < 0:
                self.moveXThisFrame = 0

```

The method takes the last frame's speed, and decelerates it by a value until it hits 0

Hitbox Hurtbox System



In order to add attacking opponents, a system of hitboxes and hurtboxes had to be created in order to determine when 1 character hits another.

Hurtboxes and hitbox values are defined for each animation's frame in the ANIMATION_LIST:

```
[ #AttackAir
  ["sprites/F00_AttackAir_0.gif", [-11.5,11.5,-20,12], None, None, None],
  ["sprites/F00_AttackAir_1.gif", [-11.5,11.5,-20,12], None, None, None],
  ["sprites/F00_AttackAir_2.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 3], [1, 1, 4, 15, 0], [-5, 28, -10, 3]],
  ["sprites/F00_AttackAir_2.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 3], [1, 1, 4, 15, 0], [-5, 28, -10, 3]],
  ["sprites/F00_AttackAir_2.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 3], [1, 1, 4, 15, 0], [-5, 28, -10, 3]],
  ["sprites/F00_AttackAir_2.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 3], [1, 1, 4, 15, 0], [-5, 28, -10, 3]],
  ["sprites/F00_AttackAir_2.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 3], [1, 1, 4, 15, 0], [-5, 28, -10, 3]],
  ["sprites/F00_AttackAir_0.gif", [-11.5,11.5,-20,12], None, None, None],
  ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
```

Hurtbox Hitbox Hitbox Properties Hurtbox Extension

These are updated twice every frame (before and after self.animate() method has been ran, in order to ensure hit/hurtboxes dont lag behind by a frame)

```
def update_hit_hurt(self):
    global ANIMATION_LIST
    global JUMP_INITIAL
    anim = ANIMATION_LIST[self.animListID]
    direction_mul = 1.0
    if self.is_left:
        direction_mul = -1.0

    x = self.turtle.xcor()
    y = self.turtle.ycor()

    hurtbox[self.playerNum - 1] = [0,0,0,0]
    hurtbox_pos = [x,x,y,y]
    if anim[self.frame][1] != None:
        hurtbox_pos[0] += anim[self.frame][1][0]*SCALE
        hurtbox_pos[1] += anim[self.frame][1][1]*SCALE
        hurtbox_pos[2] += anim[self.frame][1][2]*SCALE
        hurtbox_pos[3] += anim[self.frame][1][3]*SCALE
        hurtbox[self.playerNum - 1] = hurtbox_pos
    else:
        hurtbox[self.playerNum - 1] = [9990, 10000, 9990, 10000]

    hurtbox_2[self.playerNum - 1] = [0,0,0,0]
    hurtbox2_pos = [x,x,y,y]
    if anim[self.frame][4] != None:
        if direction_mul > 0:
            hurtbox2_pos[0] += anim[self.frame][4][0]*SCALE* direction_mul
            hurtbox2_pos[1] += anim[self.frame][4][1]*SCALE* direction_mul
        else:
            hurtbox2_pos[1] += anim[self.frame][4][0]*SCALE* direction_mul
            hurtbox2_pos[0] += anim[self.frame][4][1]*SCALE* direction_mul
        hurtbox2_pos[2] += anim[self.frame][4][2]*SCALE
        hurtbox2_pos[3] += anim[self.frame][4][3]*SCALE
        hurtbox_2[self.playerNum - 1] = hurtbox2_pos
    else:
        hurtbox_2[self.playerNum - 1] = [999, 1000, 999, 1000]

    hitbox[self.playerNum - 1] = [0,0,0,0]
    hitbox_pos = [x,x,y,y]
    if anim[self.frame][2] != None and not self.hasHit:
        if direction_mul > 0:
            hitbox_pos[0] += (anim[self.frame][2][0] * direction_mul)*SCALE
            hitbox_pos[1] += (anim[self.frame][2][1] * direction_mul)*SCALE
        else:
            hitbox_pos[1] += (anim[self.frame][2][0] * direction_mul)*SCALE
            hitbox_pos[0] += (anim[self.frame][2][1] * direction_mul)*SCALE
        hitbox_pos[2] += anim[self.frame][2][2]*SCALE
        hitbox_pos[3] += anim[self.frame][2][3]*SCALE
        hitbox[self.playerNum - 1] = hitbox_pos
        #print(f"Hitboxes out! Hurtbox: {hurtbox_pos} Hitbox: {hitbox_pos}")
        #print(f"pos [{x}, {y}]")
    else:
        hitbox[self.playerNum - 1] = [9990, 10000, 9990, 10000]
        self.hasHit = False

    hitbox_properties[self.playerNum - 1] = anim[self.frame][3]
```

The hurtbox and hitbox values are sent to a global 2D array, so that they can be easily referred to by the other character object, and the hitbox/hurtbox drawers for the hitbox viewer. The hitboxes and hurtboxes are multiplied by the character scale to ensure that the hurtboxes and hitboxes are accurate. When the hitbox and hurtbox values are labelled as None in the ANIMATION_LIST, the hitboxes and hurtboxes are set out of bounds.

Using these reported values, methods to check whether or not you have hit an opponent, or been hit by an opponent are used to set hitstun, or disable a hitbox.

This method returns True if 2 provided rectangles overlap:

```
def overlap(self, rect1, rect2):
    r1x1 = rect1[0]
    r1x2 = rect1[1]
    r1y1 = rect1[2]
    r1y2 = rect1[3]
    r2x1 = rect2[0]
    r2x2 = rect2[1]
    r2y1 = rect2[2]
    r2y2 = rect2[3]
    if r1x1 > r2x2 or r2x1 > r1x2:
        return False
    if r1y1 > r2y2 or r2y1 > r1y2:
        return False

    print("Overlaps!")
    return True
```

```
def check_is_hurt(self):
    global ANIMATION_LIST
    global ACTIONABLE_LIST
    #print(f"[(self.x), (self.y)]")
    enemy_hitbox = hitbox[-(self.playerNum)]
    my_hurtbox = hurtbox[self.playerNum - 1]
    my_hurtbox2 = hurtbox_2[self.playerNum - 1]

    enemy_properties = hitbox_properties[-(self.playerNum)]
    #if enemy_hitbox[0] < 9000:
    #    print(f"hurtbox [{my_hurtbox[0]}] - {my_hurtbox[1]}", hitbox [{enemy_hitbox[0]}] - {enemy_hitbox[1]}")
    if not enemy_properties == None and enemy_hitbox[0] < 9000 and (self.overlap(enemy_hitbox, my_hurtbox) or self.overlap(enemy_hitbox, my_hurtbox2)):
        print("Hitstun!")
        self.isHitstun = True
        self.hitstunFrames = enemy_properties[2]
        self.moveXThisFrame = enemy_properties[3]
        if self.is_left == True:
            self.moveXThisFrame *= -1

        self.moveYThisFrame = enemy_properties[4]
        if enemy_properties[4] == 0 and self.isJump:
            self.moveYThisFrame = DEFAULT_Y_KNOCKBACK
        self.lastmoveX = self.moveXThisFrame * -1
        self.lastmoveY = self.moveYThisFrame
        self.set_new_anim_by_ID(get_anim_ID("Hitstun"))
```

This method checks if one of the 2 hurtboxes the character object has overlaps the opponent character's hitbox. If it does, you enter hitstun, with your knockback, hitstun frames, and hitstun status changed.

The same is done on the other end with self.check_is_hitting()

```
def check_is_hitting(self):
    global ANIMATION_LIST
    global ACTIONABLE_LIST
    global force_hit_now
    #print(f"[{self.x}, {self.y}]")
    enemy_hurtbox = hurtbox[-(self.playerNum)]
    enemy_hurtbox2 = hurtbox_2[-(self.playerNum)]
    my_hitbox = hitbox[self.playerNum - 1]

    if my_hitbox[0] < 9000 and (self.overlap(enemy_hurtbox, my_hitbox) or self.overlap(enemy_hurtbox2, my_hitbox)):
        if not self.hasHit:
            force_hit_now = True
            self.hasHit = True
```

When true, the character turns off their hitbox so it doesnt hit multiple times.

Hitstun momentum is controlled by this method:

```
def hitstun_movement(self):
    if self.isHitstun:
        self.moveXThisFrame = self.lastmoveX * KB_DECAY_MULTIPLIER
        self.moveYThisFrame = self.lastmoveY * KB_DECAY_MULTIPLIER
        self.lastmoveY = self.moveYThisFrame
        self.lastmoveX = self.moveXThisFrame
```

hitstun frames are counted down in update:

```
if self.hitstunFrames > 0:
    print("still in hitstun")
    self.hitstunFrames -= 1
```

Alpha Testing

Alpha testing had many more tests with hitboxes and hurtboxes

Test Number	Test Description	Success?	Details
8	Input left and right buttons. Then input up and down buttons.	Yes	--
12	Input left and right at the same time. Then input up and down at the same time	Yes	--
9	Input an attack	Yes	--
10	Input a jump, then attack	Yes	--
11	Hold left and input attack, then hold right and input attack, then hold down and input	Yes	--

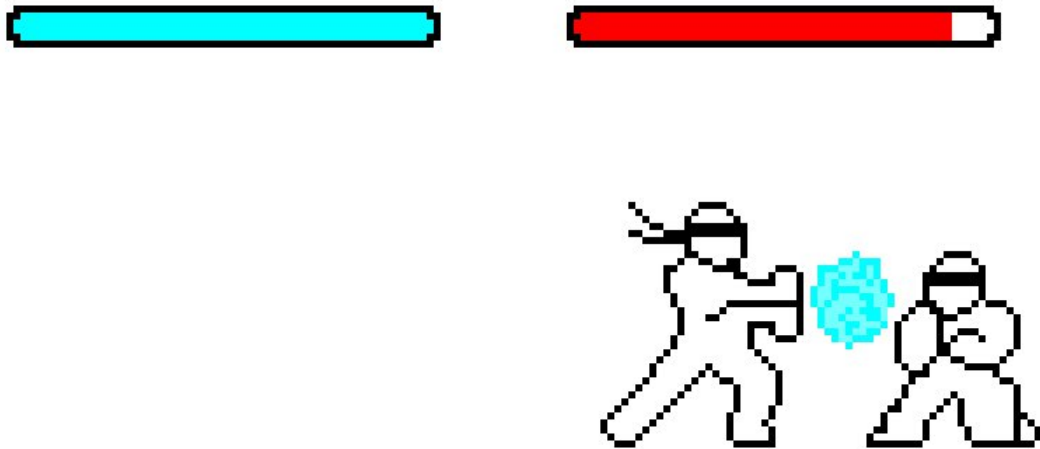
	attack.		
16	Both players attack on the same frame	No	P2 and P1 have a random chance of winning the trade, and not also being put into hitstun
17	1 player attacks the other in range of their hurtbox. Then walks backwards and attacks outside of their hurtbox	No	Opponent player was hit outside of range on certain frames when their extended hurtbox disappeared or the hitbox disappeared
14	Press an unbinded key	Yes	--
15	Double tap forwards. Then Double tap backwards.	Yes	--

Bugs Found

During the creation of the overlap method for the hitbox/hurtbox system, on frames where the hitbox disappeared, an attack would hit from anywhere on screen. This was fixed by adding an additional check to see if the your hitbox was a valid value in the check_is_hitting method and check_is_hurt method.

```
if my_hitbox[0] < 9000 and (self.overlap(enemy_hurtbox, my_hitbox) or self.overlap(enemy_hurtbox2, my_hitbox)):
```


Prototype 4 – Health Bars, Blocking, & Specials



Prototype 4 builds upon the base provided in Prototype 3 and expands upon it to produce a functional fighting game. It adds various new additions:

- Adds ability to deal damage
 - Adds healthbars for the 2 players
 - Adds winscreen when 1 player reaches 0 health
- Adds pushback for hitting light and heavy attacks
- Adds ability to block high and low
 - Adds overheads and low attacks, as well as left right mixups
- Adds Heavy attacks
- Adds Special attacks:
 - Adds special cancelling on hit or on block
 - Adds “Hashogeki” Special:
 - Neutral Special
 - A multihitting attack with a fireball
 - Adds “Tatsumaki” Special:
 - Side Special
 - A 2 hit spinning travelling kick
 - Adds “Shoryuken” Special:
 - Down Special
 - An invulnerable anti-air punch
 - Fulfils General Requirement 3 as a one button reversal

Blocking System

Blocking is set within the check_block method, which runs when an actionable animation or in the blocking animation. It sets the isBlocking attribute to true whenever you are holding away from the opponent, and false whenever you aren't.

```
def check_block(self):
    #if self.isJump:
    #    self.isBlocking = False
    if self.isHitstun:
        return
    if self.hitstunFrames > 0:
        return
    if not (self.animListID in ACTIONABLE_LIST or self.animListID in BLOCKING_LIST):
        self.isBlocking = False
        return

    if not self.is_left:
        if keyboard.is_pressed(self.controls[0]) and not keyboard.is_pressed(self.controls[1]):
            self.isBlocking = True
        else:
            self.isBlocking = False
    else:
        if keyboard.is_pressed(self.controls[1]) and not keyboard.is_pressed(self.controls[0]):
            self.isBlocking = True
        else:
            self.isBlocking = False
```

This blocking attribute is then used by the method did_i_block_that, which is run during the start of hitstun, to check whether to enter hitstun or blockstun.

```
def did_i_block_that(self):
    enemy_properties = hitbox_properties[-(self.playerNum)]
    high_low = enemy_properties[0]
    if self.isHitstun and not self.isBlockstun:
        return False
    if self.isBlocking:
        if high_low == 0:
            return True
        if high_low == -1 and self.isCrouch:
            return True
        if high_low == 1 and not self.isCrouch:
            return True
    return False
```

When an attack is blocked, it is handled similarly to hitstun, except you do not take damage, you are in stun for less time, and you are pushed back less.

```
if self.did_i_block_that():
    self.isBlockstun = True
    self.moveXThisFrame *= BLOCK_KB_MUL
    self.moveYThisFrame = 0
    self.hitstunFrames = int(math.floor(enemy_properties[5]))
    if self.isJump:
        self.hitstunFrames = int(math.floor(enemy_properties[5]*AIR_BLOCK_HITSTUN_MUL))
```

Heavy Attacks and Specials

Heavy attacks and specials were added in a similar way to light attacks, by adding their sprite animations to the ANIMATION_LIST constant, creating buffer attributes and copying how the buffer is handled for light attacks, and adding methods to set which animation to go into.

```
if self.attackBuffer > 0:
    self.attackBuffer -= 1

if self.specialBuffer > 0:
    self.specialBuffer -= 1

if self.jumpBuffer > 0:
    self.jumpBuffer -= 1

if self.heavyBuffer > 0:
    self.heavyBuffer -= 1
```

```
def heavy(self):
    global ANIMATION_LIST
    global ACTIONABLE_LIST
    if not self.animListID in ACTIONABLE_LIST:
        self.heavyBuffer = BUFFER_FRAMES
    else:
        self.heavyBuffer = 0

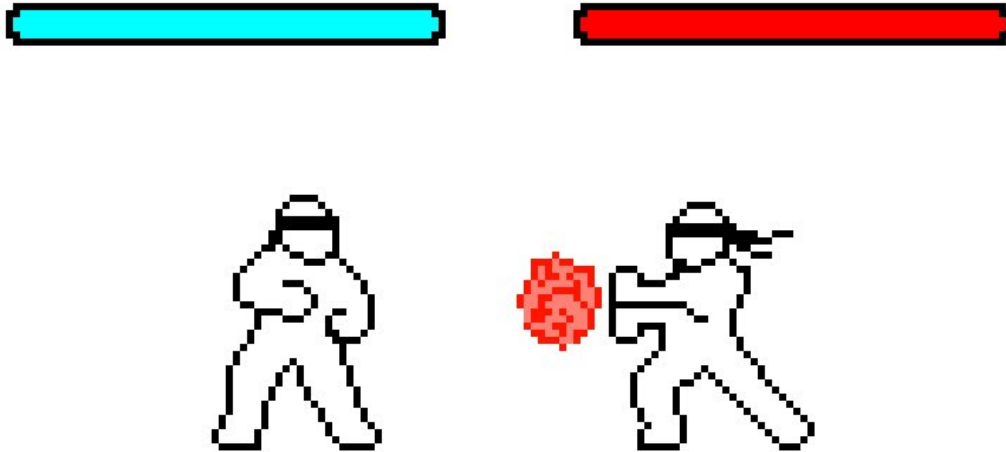
    if not self.isJump:
        if self.animListID in ACTIONABLE_LIST:
            self.doPushback = False
            if keyboard.is_pressed(self.controls[3]) and not keyboard.is_pressed(self.controls[2]):
                self.set_new_anim_by_ID(get_anim_ID("HeavyLw"))
            else:
                self.set_new_anim_by_ID(get_anim_ID("Heavy"))
        else:
            if self.animListID in ACTIONABLE_LIST:
                self.set_new_anim_by_ID(get_anim_ID("HeavyAir"))
```

(heavy method is fundamentally structured the same as attack method)

```
def special(self):
    global ANIMATION_LIST
    global ACTIONABLE_LIST
    global SPECIAL_CANCEL_LIST
    if not self.animListID in ACTIONABLE_LIST and not (self.animListID in SPECIAL_CANCEL_LIST and self.doPushback):
        self.specialBuffer = BUFFER_FRAMES
    else:
        self.specialBuffer = 0
        if self.isJump:
            return

        if keyboard.is_pressed(self.controls[3]) and not keyboard.is_pressed(self.controls[2]):
            self.set_new_anim_by_ID(get_anim_ID("SpecialLw"))
        elif keyboard.is_pressed(self.controls[1]) or keyboard.is_pressed(self.controls[0]):
            self.set_new_anim_by_ID(get_anim_ID("SpecialS"))
        else:
            if self.playerNum == 1:
                self.set_new_anim_by_ID(get_anim_ID("SpecialN1"))
            else:
                self.set_new_anim_by_ID(get_anim_ID("SpecialN2"))
```

The special method is similar, however, it adds an additional check to see if the animation you are currently in is special cancellable, and if it has hit (self.doPushback). Furthermore, there is also a check for left and right key presses for Side Specials. The Special N animations are split by player number, in order to give player 2 a red fireball instead (pictured below)



Special Lw and Special S have unique movement, which is defined in the specialLw and specialS methods:

```

def specialLw(self):
    if self.animListID != get_anim_ID("SpecialLw"):
        return

    direction_mul = 1.0
    if self.is_left:
        direction_mul = -1.0

    if self.frame == 3:
        self.isJump = True
        self.jumpDir = 0
        self.moveYThisFrame = SPECIAL_LW_INIT_Y
        self.moveXThisFrame = SPECIAL_LW_INIT_X*direction_mul
    elif self.frame > 3:
        self.moveYThisFrame = self.lastmoveY - GRAVITY
        self.moveXThisFrame = self.lastmoveX * 0.8

def specialS(self):
    if self.animListID != get_anim_ID("SpecialS"):
        return

    direction_mul = 1.0
    if self.is_left:
        direction_mul = -1.0

    self.moveXThisFrame = SPECIAL_S_X*direction_mul

```

Adding pushback on normals

The doPushback attribute is set to true in the same area where the hasHit attribute is set to true:

```
def check_is_hitting(self):
    global ANIMATION_LIST
    global ACTIONABLE_LIST
    global force_hit_now
    #print(f"[{self.x}, {self.y}]")
    enemy_hurtbox = hurtbox[-(self.playerNum)]
    enemy_hurtbox2 = hurtbox_2[-(self.playerNum)]
    my_hitbox = hitbox[self.playerNum - 1]

    if my_hitbox[0] < 9000 and (self.overlap(enemy_hurtbox, my_hitbox) or self.overlap(enemy_hurtbox2, my_hitbox)):
        if not self.hasHit:
            #print("Yeah")
            force_hit_now = True
            self.hasHit = True

            self.doPushback = True
```

and is set to False whenever a new light or heavy attack is used.

```
def attack(self):
    global ANIMATION_LIST
    global ACTIONABLE_LIST
    if not self.animListID in ACTIONABLE_LIST:
        self.attackBuffer = BUFFER_FRAMES
    else:
        self.attackBuffer = 0

    if not self.isJump:
        if self.animListID in ACTIONABLE_LIST:
            self.doPushback = False
            if keyboard.is_pressed(self.controls[3]) and not keyboard.is_pressed(self.controls[2]):
                self.set_new_anim_by_ID(get_anim_ID("AttackLw"))
            else:
                self.set_new_anim_by_ID(get_anim_ID("Attack"))
        else:
            if self.animListID in ACTIONABLE_LIST:
                self.set_new_anim_by_ID(get_anim_ID("AttackAir"))
```

This doPushback attribute is used within the set_pushback method, which checks whether or not you are in an animation that gets pushback on hit, and sets your X movement to a PUSHBACK constant if you are.

```
def set_pushback(self):
    if self.animListID in [get_anim_ID("AttackLw"), get_anim_ID("Attack"), get_anim_ID("HeavyLw")]:
        side_mul = 1
        if self.is_left:
            side_mul = -1

        if self.doPushback == True:
            self.moveXThisFrame = PUSHBACK * side_mul * -1
```

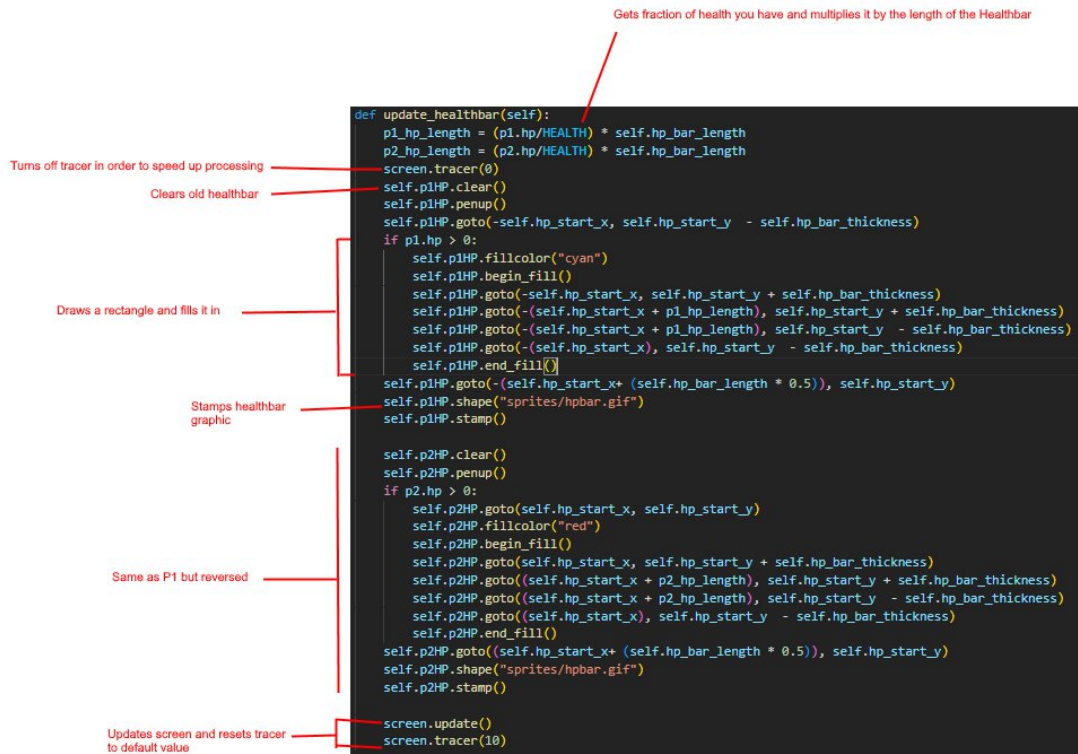

Healthbar

The Healthbar is created with its own battleUI class, which contains 3 turtles, one for each character's HP, and one to display the victory screen:

```
class battleUI:
    def __init__(self):
        self.p1HP = turtle.Turtle()
        self.p2HP = turtle.Turtle()
        self.youWin = turtle.Turtle()
        self.p1HpPos = [-200,150]
        self.p2HpPos = [200,150]
        self.p1HP.penup()
        self.p2HP.penup()
        self.youWin.penup()
        screen.tracer(0)
        self.youWin.hideturtle()
        self.p1HP.hideturtle()
        self.p2HP.hideturtle()
        self.p1HP.goto(self.p1HpPos[0], self.p1HpPos[1])
        self.p2HP.goto(self.p2HpPos[0], self.p2HpPos[1])
        self.youWin.goto(0, 0)
        screen.update()
        screen.tracer(10)

        self.hp_bar_length = 300
        self.hp_start_x = 50
        self.hp_start_y = 175
        self.hp_bar_thickness = 2*SCALE
```

The healthbars are controlled via the update healthbar method:



This method is called in the battleUI update method proper, which contains additional logic for the win screen:

```
def update(self):
    self.update_healthbar()
    if youwin:
        player = 1
        if p2.hp > p1.hp:
            player = 2
        self.youWin.shape(f"sprites/p{player}_win.gif")
        self.youWin.stamp()
```

All of this is then called in the main loop:

```
start = time.time()
turn_order = bool(random.getrandbits(1))
if ((FRAME_STEP and keyboard.is_pressed("space")) or (SPACE_TO_PAUSE and not keyboard.is_pressed("space"))) or (not FRAME_STEP and keyboard.is_pressed("space")):
    ui.update()
    if turn_order:
        p1.update()
        p2.update()
    else:
        p2.update()
        p1.update()
end = time.time()
new_delay = (end-start)*10**3
#print(f"Time taken: {new_delay}ms, normalise: {FRAME_LENGTH - (new_delay)}ms")
if (new_delay/1000) < FRAME_LENGTH/1000:
    time.sleep(FRAME_LENGTH/1000 - (new_delay/1000))
prev_delay = new_delay

p1.hp -= p1.decreaseHp
p2.hp -= p2.decreaseHp
p1.decreaseHp = 0
p2.decreaseHp = 0
if p1.hp <= 0 or p2.hp <= 0:
    youwin = True
```

These changes fulfil success criteria 1 (When the HP hits 0 for either side, a winner is declared.).

Custom Controls

Although not yet easily editable via in game GUI, controls are now read from a text file called “controls.txt”

```
def get_controls_from_txt() -> list:
    f = open("controls.txt")
    controls = f.readlines()
    for x in range(0, len(controls)):
        i = controls[x]
        i = i.replace("\n", "")
        i = i.split(" ")
        while len(i) < 7:
            i.append("_")
        controls[x] = i

    return controls

controlsList = get_controls_from_txt()
p1 = player(
    1,
    -150, 0,
    #["a", "d", "w", "s", "f", "g", "h"],
    controlsList[0],
    False
)
p2 = player(
    2,
    150, 0,
    #["left", "right", "up", "down", ".", "/", "shift"],
    controlsList[1],
```

Stakeholder response

Stakeholders responded very positively to this prototype, being the first prototype that could be considered an actual game. They enjoyed the simplicity of the controls scheme, with only 3 attacking buttons and no motion inputs, which were particularly daunting to Stakeholder 2, who had no experience with fighting games. The simplicity of the controls fulfilled General Requirement 2 (Simple Controls, with only a few buttons and a lack of motion inputs). They particularly enjoyed special cancelling into the special moves to do combos. They did however dislike how safe of an option Neutral Special was on block, as they found it difficult to punish.

Alpha Testing

Alpha testing had tests relating to blocking and HP.

Test Number	Test Description	Success?	Details
9	Input an attack button, then a heavy attack button, then a special button	Yes	--
18	Player 1 presses crouching light, and player 2 holds down and back. Player 1 presses crouching light again, but Player 2 holds just back. Repeat with standing light and standing heavy. Then Player 1 crosses up player 2 with jumping light, player 2 does not switch direction blocking. Repeat this situation but player 2 does switch direction blocking	Yes	--

26	Hit the opponent	Yes	--
27	Hit the opponent until their HP hits 0	Yes	--

Bugs found

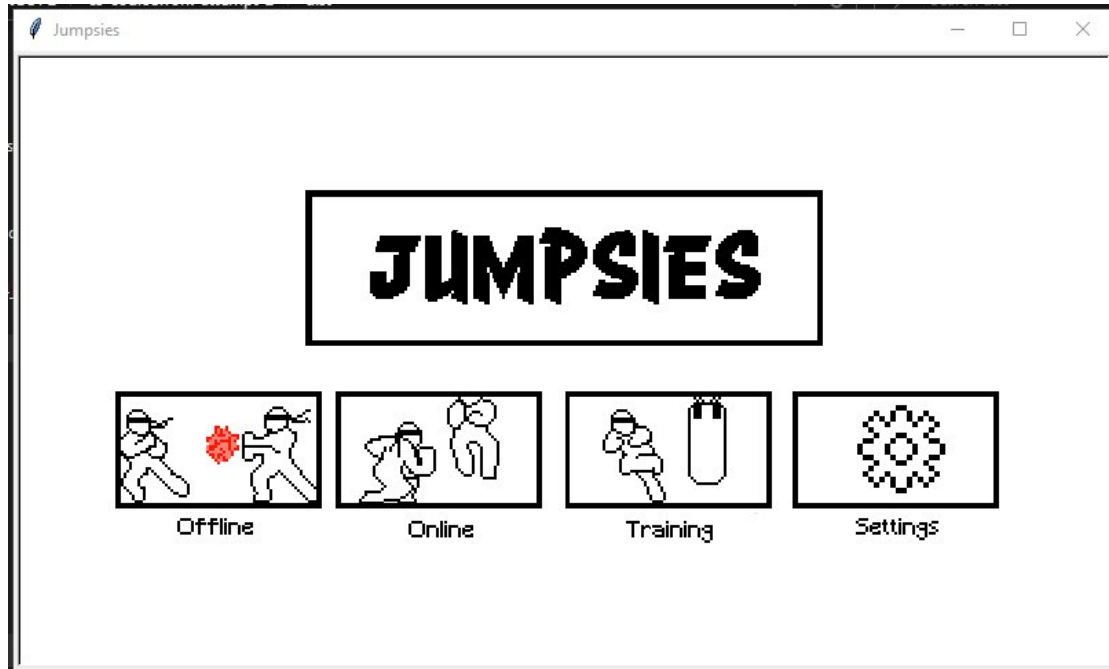
A major bug while developing this prototype was blocking persisting while using a move, allowing a character to block while using a move such as a special. This was fixed through the addition of a check if they were actionable, and setting the `isBlocking` attribute to false:

```
if not (self.animListID in ACTIONABLE_LIST or self.animListID in BLOCKING_LIST):  
    self.isBlocking = False  
return
```

Prototype 5 – Menus

Prototype 5 was focused on adding 2 UI elements, a pause menu, and a main menu.

Main Menu



The main menu was modelled after main menu mock-up 2, due to its positive reception among stakeholders. Each menu button is able to be clicked, however the only button that does anything as of this prototype is the offline button.

The main menu is handled within menu.py, which is separate from the main game, which is run_game.py

```
def check_click_pos(x,y):
    if y < MIN_Y or y > MAX_Y:
        return
    if x >= OFFLINE[0] and x <= OFFLINE[1]:
        try:
            run_game.run()
        except Exception:
            sys.exit()
    elif x >= ONLINE[0] and x <= ONLINE[1]:
        dummy = 0
    elif x >= TRAINING[0] and x <= TRAINING[1]:
        dummy = 0
    elif x >= SETTINGS[0] and x <= SETTINGS[1]:
        dummy = 0

def run(auto=False):
    turtle.TurtleScreen._RUNNING=True
    screen = turtle.Screen()
    screen.setup(SCREEN_WIDTH, SCREEN_HEIGHT)
    screen.title("Jumpsies")
    screen.bgcolor("white")
    screen.addshape("menu/main.gif")
    screen.clearscreen()
    if auto:
        try:
            run_game.run()
        except Exception:
            sys.exit()
    menuUI = turtle.Turtle()
    menuUI.shape("menu/main.gif")
    screen.onscreenclick(check_click_pos)
    screen.mainloop()
```

When you click in the correct X and Y range on the window, it calls the function to run the main gameplay.

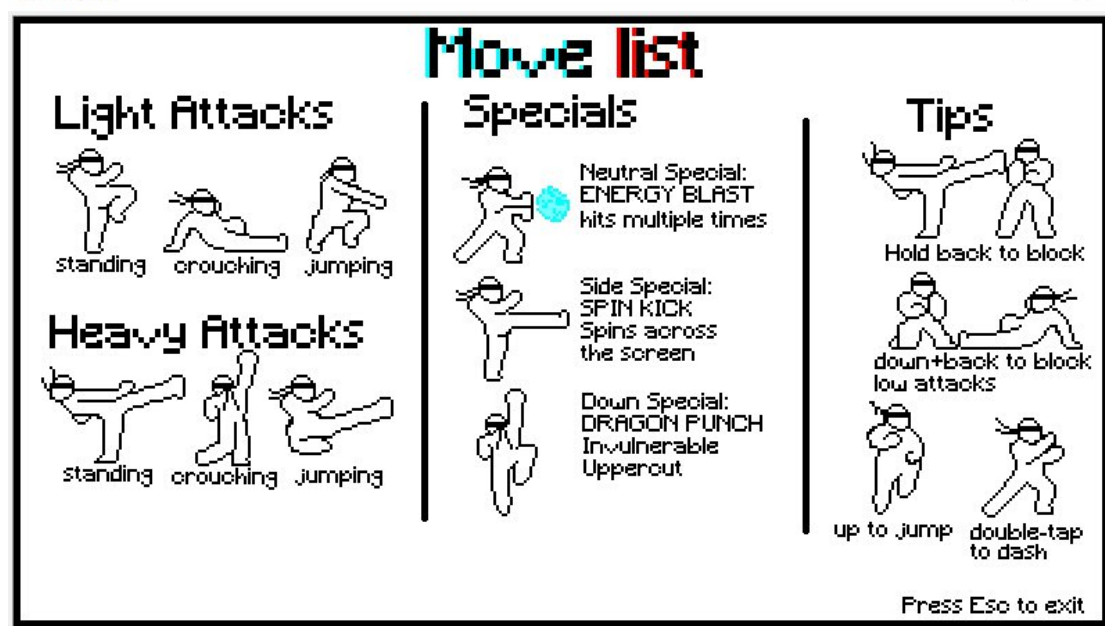
Pause Menu



A Pause menu was also added, accessed by pressing the escape key. It is navigated by pressing up and down, and pressing the light attack button to enter:

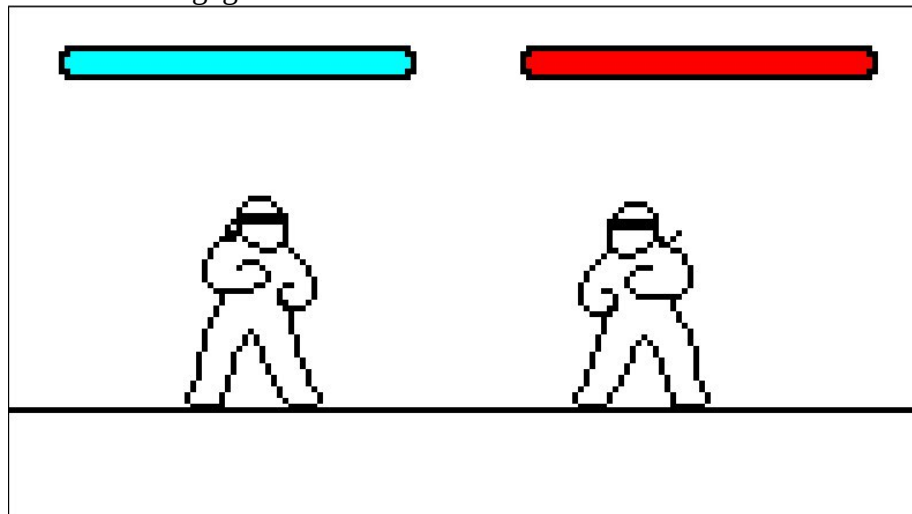
```
def pause_update(pause_turtle, controls):
    shape = pause_turtle.shape()
    up = [controls[0][2], controls[1][2]]
    down = [controls[0][3], controls[1][3]]
    accept = [controls[0][4], controls[1][4]]
    if shape in ["menu/pause_1.gif", "menu/pause_2.gif", "menu/pause_3.gif"]:
        if keyboard.is_pressed(up[0]) or keyboard.is_pressed(up[1]):
            if shape == "menu/pause_2.gif":
                pause_turtle.shape("menu/pause_1.gif")
                time.sleep(0.2)
            if shape == "menu/pause_3.gif":
                pause_turtle.shape("menu/pause_2.gif")
                time.sleep(0.2)
        if keyboard.is_pressed(down[0]) or keyboard.is_pressed(down[1]):
            if shape == "menu/pause_1.gif":
                pause_turtle.shape("menu/pause_2.gif")
                time.sleep(0.2)
            if shape == "menu/pause_2.gif":
                pause_turtle.shape("menu/pause_3.gif")
                time.sleep(0.2)
        if keyboard.is_pressed(accept[0]) or keyboard.is_pressed(accept[1]):
            if shape == "menu/pause_1.gif":
                pause_turtle.shape("menu/movelist.gif")
            if shape == "menu/pause_2.gif":
                return 2
            if shape == "menu/pause_3.gif":
                menu.run()
```

the movelist button shows off a basic tutorial on what the moves are, and what to do:



This fulfils general requirement 4 (A basic tutorial page, explaining the mechanics of the game), by teaching you how to play the game on a single page.

A line was also added to the play scene as the “ground”, in order for the play scene to be easier to engage with.



Stakeholder Response

Stakeholders found the move list incredibly simple to understand and get to grips with, although they noted the lack of a mention of special cancelling, an important mechanic for combos. Stakeholders also enjoyed the main menu’s look and found it easy and simple to understand.

Alpha Testing

This alpha testing was focused on menus.

Test Number	Test Description	Success?	Details
28	Press escape to open the pause menu, then press escape to close it	Yes	--
29	Press escape to pause, then press light attack to open the movelist. Then press escape to return to in game	Yes	--
30	Press escape to pause, then tap down 4 times, then press attack	Yes	--
31	Press escape to pause, then tap down once, then press attack	Yes	--
1	Press centre of all 4 buttons on main menu	Yes	--
2	Press left edge of each main menu button	Yes	--
3	Press on empty space	Yes	--
4	Press outside of the game window	Yes	--

Bugs found

A major issue while developing this prototype was turtle screen class's unwillingness to reset the screen after running mainloop, which was required for onclick methods to work. This was fixed by manually setting the TurtleScreen._RUNNING attribute to true every time the screen was set up:

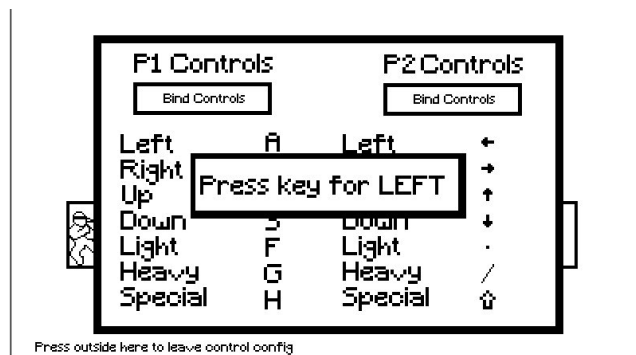
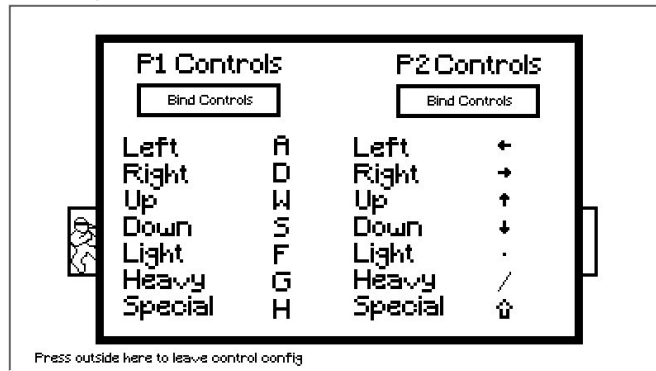
```
turtle.TurtleScreen._RUNNING=True
```

This allowed the screen to persist after mainloop and not throw an error from turtle.terminator

Prototype 6 – Training mode & Settings

This Prototype adds the ability to change controls within the GUI (via the settings button) and adds training mode.

Settings



Settings are handled in menu.py

The settings screen is enabled by setting `is_controls` to true and running the `run_settings_screen()` function

```
def check_click_pos(x,y):
    global is_controls
    global screen_instance
    if not is_controls:
        if y < MIN_Y or y > MAX_Y:
            return
        if x >= OFFLINE[0] and x <= OFFLINE[1]:
            try:
                run_game.run()
            except Exception:
                sys.exit()
        elif x >= ONLINE[0] and x <= ONLINE[1]:
            dummy = 0
        elif x >= TRAINING[0] and x <= TRAINING[1]:
            try:
                run_game.run([True,True,False,0,0])
            except Exception:
                sys.exit()
        elif x >= SETTINGS[0] and x <= SETTINGS[1]:
            is_controls = True
            run_settings_screen()
```


When `is_controls` is set to `true` and the `onscreenclick` method for the screen is called, instead of the default 4 options check, it runs a different check to see if you clicked outside of the settings window or on the 2 control update buttons.

```
else:
    if x > 300 or x < -300 or y < -195 or y > 195:
        run()
    if y < 85 or y > 135:
        return
    if x < -70 and x > -250:
        update_controls(0)
    if x < 250 and x > 70:
        update_controls(1)
```

If you do click Bind Controls, the `update_controls` function is run, which goes through each button and prompts you to press your desired button for it:

```

def update_controls(player):
    global screen_instance
    controls = get_controls_from_txt()
    new_control_set = controls[player]
    printer = turtle.Turtle()
    screen_instance.update()
    screen_instance.tracer(10)
    printer.showturtle()

    keylist = VALID_KEYS
    printer.shape("menu/presskey_left.gif")
    screen_instance.update()
    button = wait_till_button(keylist)
    keylist.remove(button)
    new_control_set[0] = button
    printer.shape("menu/presskey_right.gif")
    screen_instance.update()
    button = wait_till_button(keylist)
    keylist.remove(button)
    new_control_set[1] = button
    printer.shape("menu/presskey_up.gif")
    screen_instance.update()
    button = wait_till_button(keylist)
    keylist.remove(button)
    new_control_set[2] = button
    printer.shape("menu/presskey_down.gif")
    screen_instance.update()
    button = wait_till_button(keylist)
    keylist.remove(button)
    new_control_set[3] = button
    printer.shape("menu/presskey_light.gif")
    screen_instance.update()
    button = wait_till_button(keylist)
    keylist.remove(button)
    new_control_set[4] = button
    printer.shape("menu/presskey_heavy.gif")
    screen_instance.update()
    button = wait_till_button(keylist)
    keylist.remove(button)
    new_control_set[5] = button
    printer.shape("menu/presskey_special.gif")
    screen_instance.update()
    button = wait_till_button(keylist)
    keylist.remove(button)
    new_control_set[6] = button

    controls[player] = new_control_set
    f = open("controls.txt", "w")
    f.write("")
    f.close()
    f = open("controls.txt", "a")
    f.write(" ".join(controls[0]) + "\n")
    f.write(" ".join(controls[1]))
    f.close()
    printer.hideturtle()
    run_settings_screen()

```

After this, it saves these new bindings to controls.txt, and reruns the run_settings_screen() function to refresh the displaying of the controls

The controls are rendered programmatically by iterating through the controls list and stamping the corresponding image using the turtle.shape() and turtle.stamp() methods:

```
printer = turtle.Turtle()
printer.penup()
printer.goto(-70, 55)
for i in controls[0]:
    file = i
    file = file.replace(",", "comma")
    file = file.replace(".", "dot")
    file = file.replace("/", "fslash")
    file = file.replace("UP", "up")
    file = file.replace("RIGHT", "right")
    file = file.replace("Down", "down")
    file = file.replace("LEFT", "left")
    file = file.replace("SHIFT", "shift")
    printer.shape(f"text/{file}.gif")
    printer.stamp()
    printer.goto(-70, printer.ycor() - 33)
printer.goto(200, 55)
for i in controls[1]:
    file = i
    file = file.replace(",", "comma")
    file = file.replace(".", "dot")
    file = file.replace("/", "fslash")
    file = file.replace("UP", "up")
    file = file.replace("RIGHT", "right")
    file = file.replace("Down", "down")
    file = file.replace("LEFT", "left")
    file = file.replace("SHIFT", "shift")
    printer.shape(f"text/{file}.gif")
    printer.stamp()
    printer.goto(200, printer.ycor() - 33)

printer.goto(200, -500)
screen_instance.update()
screen_instance.tracer(10)
```

Training Mode

In order to have training mode settings work, `run_game.py` first had to have all of its logic moved from outside of a function to inside of the `run()` function.

```
def run(training_settings=[False,False,False,0,0]):
    controlsList = get_controls_from_txt()
    turtle.TurtleScreen._RUNNING=True
    screen = turtle.Screen()
    screen.setup(SCREEN_WIDTH, SCREEN_HEIGHT)
    screen.title("Jumpsies")
    screen.bgcolor("white")
```

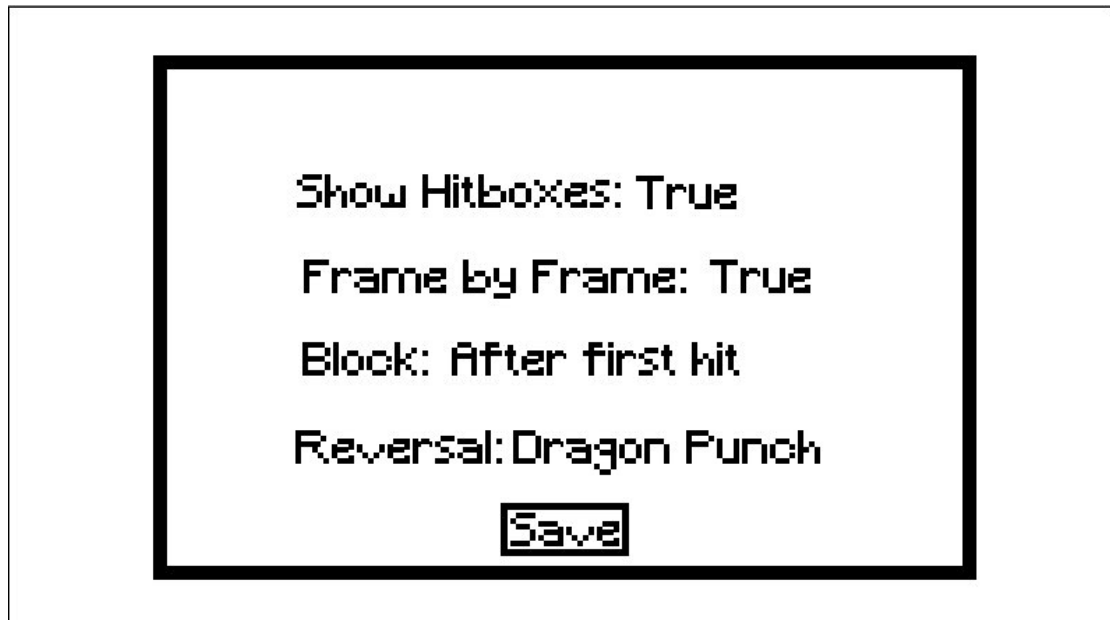
This required replacing global variables used within multiple methods (such as the screen object and the two character objects) into parameters for those methods, such as the constructor method for the battle UI class shown below:

```
class battleUI:
    def __init__(self, screen, p1Turtle, p2Turtle):
        self.screen = screen
        self.p1 = p1Turtle
        self.p2 = p2Turtle
        self.p1HP = turtle.Turtle()
        self.p2HP = turtle.Turtle()
        self.youWin = turtle.Turtle()
        self.p1HpPos = [-200,150]
        self.p2HpPos = [200,150]
        self.p1HP.penup()
        self.p2HP.penup()
        self.youWin.penup()
        screen.tracer(0)
        self.youWin.hideturtle()
        self.p1HP.hideturtle()
        self.p2HP.hideturtle()
        self.p1HP.goto(self.p1HpPos[0], self.p1HpPos[1])
        self.p2HP.goto(self.p2HpPos[0], self.p2HpPos[1])
        self.youWin.goto(0, 0)
        screen.update()
        screen.tracer(10)
```

This allows the main game logic to be called with different parameters to alter how the game works.

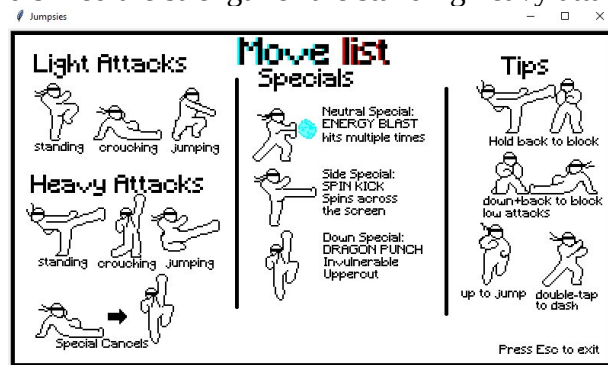
Training mode has options for showing hitboxes, frame by frame (pressing space to advance a frame), blocking settings, and reversal settings.

Each of the settings gets changed by clicking on them, and SAVE calls the run function again, with these new settings.



Stakeholder Response

Stakeholders considered the settings very easy to interact with, and the training mode's settings simple but effective. They also enjoyed the new inclusion of special cancel within the move list. One issue they often had was having issues with being forced to remain blocking. They also disliked the implementation of the fireball, particularly the hurtbox on the fireball, and its lack of movement. Some stakeholders disliked the strength of the standing heavy attack and considered it "cheap".



Alpha Testing

This alpha testing was focused on training mode and button binding.

Test Number	Test Description	Success?	Details
5	Press settings button on main menu. Press button config for player 1. Input separate buttons for each button for button mapping. Then go in game and use every button mapping to ensure it works as intended	Yes	--
6	Set the left and right buttons as the same key, then test in game	Yes	--
7	Edit the buttons.txt to remove a button binding	Yes	--
31	Press escape to pause, then tap down once, then press attack	Yes	--
19	Set to frame by frame, press space 4 times. Then hold space for 30 seconds. Then do not touch space for 1 minute. Then hold space again.	Yes	--
20	Set enable hitboxes on, play for 30 seconds. Then set enable hitboxes to false	Yes	--
21	Set to block always. Then use crouching light, then standing heavy, then jump over them and use jump heavy.	Yes	--
22	Set to block after first hit. Then use standing light -> delayed side special	Yes	--

23	Set reversal to dragon punch, and block to none. Then hit with standing light	Yes	--
24	Set reversal to dragon punch, and block to always. Then hit with standing light	Yes	--
25	Set reversal to dragon punch, and block to after first hit. Then hit with standing light. After that, use standing light -> delayed side special	Yes	--

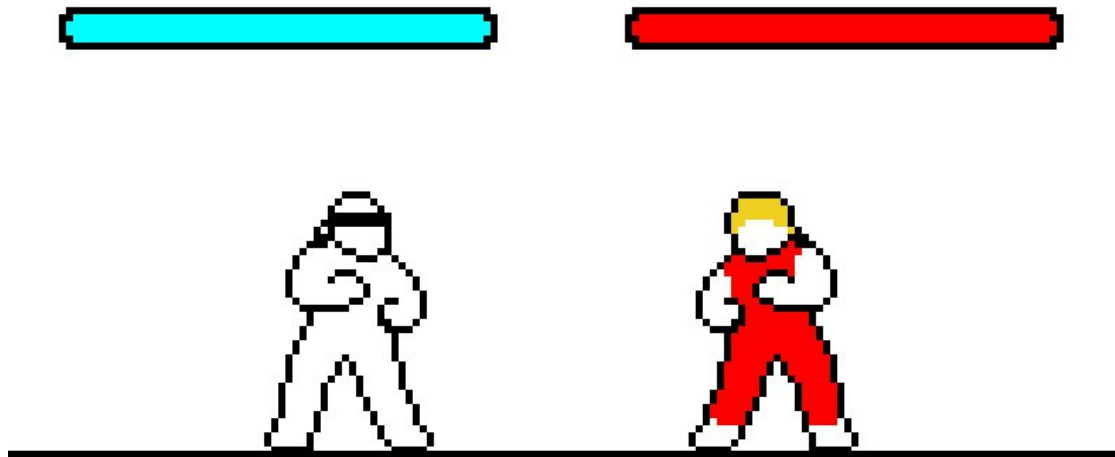
Current bugs found

While developing the training mode, the frame by frame, if left for too long, caused the screen to freeze and not respond. This was fixed by forcing the screen to still update at regular intervals, while not updating the player and UI objects.

While playtesting with stakeholders, a bug was found where being hit in the air after crossing up the opponent caused them to be launched in the wrong direction, and holding jump let them remain facing the wrong way. This bug was addressed by adding JumpSquat to the check_correct_side

```
def check_correct_side(self):
    global ACTIONABLE_LIST
    if (self.isJump == False and self.animListID in ACTIONABLE_LIST) or self.animListID == get_anim_ID("JumpSquat"):
        #Flips you around and puts you into the default animation when crossed up
        if self.is_left == True and self.x < char_pos[-(self.playerNum)][0]:
            self.is_left = False
            self.set_new_anim_by_ID()
        if self.is_left == False and self.x > char_pos[-(self.playerNum)][0]:
            self.is_left = True
            self.set_new_anim_by_ID()
```

Prototype 7 – Adding a second character



Stakeholders expressed an interest in additional characters for the project, and therefore the scope of the project expanded to include at least one additional character.

The addition of a character involved adding a character select screen, as well as moving the `ANIMATION_LIST` constant (as well as making one for the additional character) into its own script “`animations.py`”.

Another addition to the project was the addition of street fighter-like fireballs, as projectiles travelling across the screen.



The Character Select Screen is run from a function that takes in training settings as a parameter and uses that parameter along with the characters you select to run the “run_game.run” function

```
def run(training_settings=[False,False,False,0,0]):
    #Screen setup
    turtle.TurtleScreen._RUNNING=True
    screen = turtle.Screen()
    screen.setup(SCREEN_WIDTH, SCREEN_HEIGHT)
    screen.title("Jumpsies")
    screen.bgcolor("white")
    screen.clearscreen()
    for i in ["sprites", "reverse_sprites"]:
        for root, dirs, files in os.walk(i):
            #print(files)
            for filename in files:
                if filename.endswith(".gif"):
                    #print(filename)
                    #Handles scaling up and stuff
                    larger = PhotoImage(file=f"{i}/{filename}").zoom(SCALE, SCALE)
                    screen.addshape(f"{i}/{filename}", Shape("image", larger))
    for root, dirs, files in os.walk("menu"):
        for filename in files:
            if filename.endswith(".gif"):
                screen.addshape(f"menu/{filename}")
    screen.tracer(0)

    chosen_chars = [None,None]

    bg = turtle.Turtle()
    bg.penup()
```


It uses its own update method ran every frame to move the cursors:

```
def update(turtle, controls, char_pos_x, char_pos_y) -> int:
    if keyboard.is_pressed(controls[0]) and not keyboard.is_pressed(controls[1]): #Similar logic to left/right in run_game.py
        turtle.goto(turtle.xcor()-2, turtle.ycor())

    if keyboard.is_pressed(controls[1]) and not keyboard.is_pressed(controls[0]):
        turtle.goto(turtle.xcor()+2, turtle.ycor())

    if keyboard.is_pressed(controls[2]) and not keyboard.is_pressed(controls[3]):
        turtle.goto(turtle.xcor(), turtle.ycor()+2)

    if keyboard.is_pressed(controls[3]) and not keyboard.is_pressed(controls[2]):
        turtle.goto(turtle.xcor(), turtle.ycor()-2)

    if keyboard.is_pressed(controls[5]): #Cancel button is the heavy button
        return None

    if keyboard.is_pressed(controls[4]): #Attack button is the light button
        x = turtle.xcor()
        y = turtle.ycor()
        for i in range(0, len(char_pos_x)):
            if abs(char_pos_x[i]-x) < 50 and abs(char_pos_y[i]-y) < 50:
                return i #returns char_id based on cursort pos when accept button is pressed

    return -1
```

This update method is called to set the char_id value.

```
p1 = update(p1_cursor, controls[0], char_pos_x, char_pos_y)
p2 = update(p2_cursor, controls[1], char_pos_x, char_pos_y)
screen.update()

if p1 != -1:
    chosen_chars[0] = p1
if p2 != -1:
    chosen_chars[1] = p2
if chosen_chars[0] != None: #If they have a character selected, show that character
    p1_show.showturtle()
    p1_show.shape(f"sprites/F00{chosen_chars[0]}_Idle_0.gif") #We can do this because the sprites are properly named!
    p1_name.showturtle()
    p1_name.shape(f"menu/F00{chosen_chars[0]}_Name.gif")
```

When the char_id is not None, the sprites for the character's idle and their name are shown.

Animations.py

The constants for animation were moved into its own file, imported into run_game, to reduce the clutter in the codebase. They've also been renamed based on their characters.

```
RYU = [
    [ #Idle
        ["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_3.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_4.gif", [-11.5,11.5,-25,14], None, None, None]
    ],
```

This required 3 changes in run_game:

- changing the ANIMATION_LIST constant to contain every character's animation list

```
CHAR_ANIMS = [animations.RYU, animations.KEN]
```

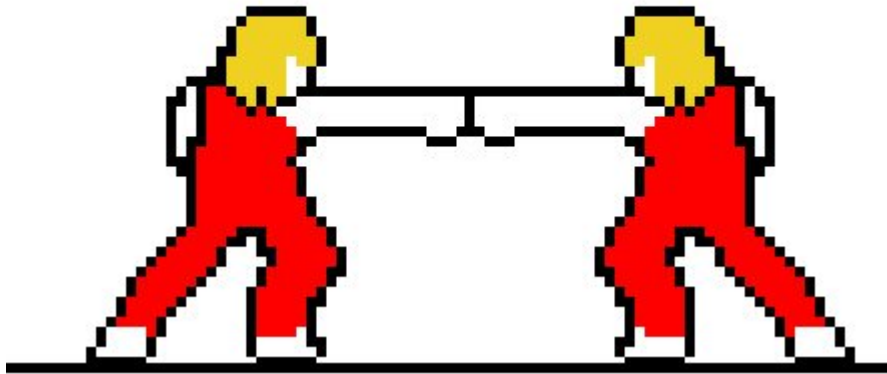
- creating a new attribute using this constant and their char_id

```
#Sprite Init
self.frame = 0
self.animList = CHAR_ANIMS[self.char_id]
self.animListID = 0
self.sprite = f"sprites/F0{self.char_id}_Idle_0.gif"
self.training_settings = Training_settings
self.start_blocking_now = 0
```

- Changing every instance where ANIMATION_LIST is referred to, to instead refer to this new attribute

```
update_sprite(self):
global JUMP_INITIAL
anim = self.animList[self.animListID]
direction_mul = 1.0
if self.is_left:
    direction_mul = -1.0
```

Due to having a fairly modular animation system, very little changes were needed to allow it to have multiple characters' animations.

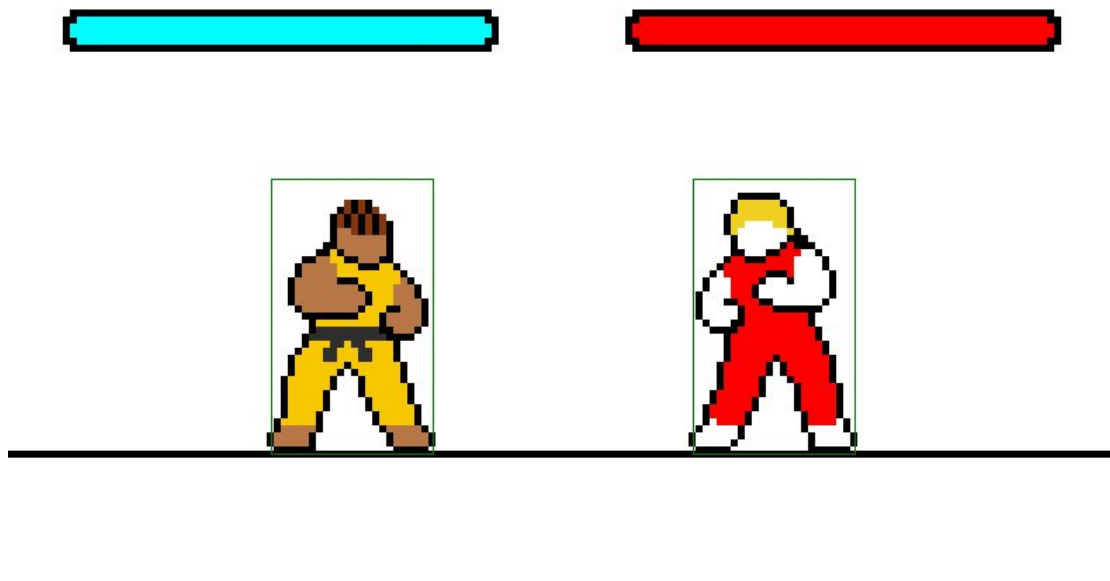


Stakeholder Response

Stakeholders enjoyed this prototype heavily, enjoying the changes made to the first character Rai Yu, particularly the new neutral special. The stakeholders also enjoyed the new character, Ben Casters. One issue stakeholders had was dealing with opponent's blocking all their moves, with one stakeholder suggesting the game add the ability to throw.



Prototype 8 – Third Character & CPU Opponent



This prototype adds a third character, the ability throw opponents, and a computer opponent to fight. During this prototype, a rewrite of the method to check for buttons was written

Button Check

```
if self.readLog:
    self.buttoncheck = self.log[self.logFrame]
    self.logFrame += 1
    if self.logFrame >= self.logLength:
        self.logFrame = 0
elif self.isCpu:
    self.buttoncheck = [False, False, False, False, False, False, False]
else:
    self.buttoncheck = [keyboard.is_pressed(self.controls[0]), keyboard.is_pressed(self.controls[1]),
                        keyboard.is_pressed(self.controls[2]), keyboard.is_pressed(self.controls[3]),
                        keyboard.is_pressed(self.controls[4]), keyboard.is_pressed(self.controls[5]),
                        keyboard.is_pressed(self.controls[6])]
```

Previously, `keyboard.is_pressed` was called an incredibly large amount of times, causing optimisation issues. It is now called once per button per player and stored, and any instances that previously checked for a button now check this list attribute. CPUs have their `buttoncheck` set to `False` so that players are entirely unable to control the CPU.

Throws

```
if (self.buttoncheck[4] and self.buttoncheck[5] and not self.buttoncheck[6]) and not(self.buttoncheck[0] or self.buttoncheck[1]):
    self.throw()
```

Throws are initiated by pressing the light and heavy attacks at the same time. Like the other button checks, this combination check is checked every frame.

```
def throw(self):
    global ACTIONABLE_LIST
    global SPECIAL_CANCEL_LIST
    if self.animListID in ACTIONABLE_LIST and not self.isJump and not self.isCrouch:
        self.set_new_anim_by_ID(get_anim_ID("ThrowWhiff"))

def throw_logic(self):
    global ACTIONABLE_LIST
    global SPECIAL_CANCEL_LIST
    if self.animListID != get_anim_ID("ThrowWhiff") and self.animListID != get_anim_ID("ThrowF"):
        return

    if self.doPushback and not self.hasHit and self.animListID == get_anim_ID("ThrowWhiff"):
        self.set_new_anim_by_ID(get_anim_ID("ThrowF")) # Goes into throw animation if the Grabbox hits
```

throw_logic is ran every frame, while throw is run when the light and heavy buttons are pressed in combination.

```
[
    ["sprites/F02_ThrowWhiff_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowWhiff_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], [0, 20, -15, 0], [2, 0, 13, -45, 45, 0], [0, 24, -25, 0]],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], [0, 20, -15, 0], [2, 0, 13, -45, 45, 0], [0, 24, -25, 0]],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
```

Throws use the high-low value of 2, which is treated as unblockable (as opposed to -1 being low, 0 being mid, and 1 being high). This allows the throw to beat blocking, it cannot be blocked. In order to have the opponent look like they are being thrown, very specific knockback stats were chosen, and the ThrowF animation has a hitbox that covers full screen (as the only scenario that hitbox comes out is if a throw has connected).

Stats.py

In order to differentiate character feel further, each character was given unique walkspeed, dash speed, and health. These constants are stored in stats.py, and called from there along using char_id as an index in run_game.py


```
WALK_SPEED = [10.5,13.5,12.0]
HEALTH = [50,45,48]
FDASH_SPEED = [41.0,42.0,49.0]
FDASH_DECEL = [8.0,8.0,9.5]
BDASH_SPEED = [35.0,35.0,32.0]
BDASH_DECEL = [8.0,8.0,7.5]
```

```
self.moveXThisFrame = stats.WALK_SPEED[self.char_id]
```

Computer Player

The computer player runs on a queue of actions it takes depending on the distance from it and the opponent. What actions get placed on the queue is determined by the `check_movement` procedure:

```

def add_to_queue(object, anim):
    object.cpuQueue.append(anim)

def rng_queue(object, choices):
    Adds a random action to the queue based on a list of choices
    object.cpuQueue.append(random.choice(choices))

def check_movement(object, yourX, theirX, yourY, theirY):
    if object.isJump:
        add_to_queue(object, WAIT)
        return
    distance = int(abs(yourX-theirX)/10) + 10
    distanceY = int(abs(yourY-theirY)/10)
    if distance < 300:
        if distance < 300 < 80:
            if distanceY < 3:
                if they're on the same level or over so slightly above
                rng_queue(object, [DASH_B, 300(object.char_id), 300(object.char_id), GRAB])
            elif distanceY < 25:
                if they're above but not too high to grab
                rng_queue(object, [DASH_B, ANTIAIR(object.char_id), ANTIAIR(object.char_id), EP(object.char_id)])
            else:
                add_to_queue(object, WAIT)
        else:
            if random.randint(0, 100) < 80:
                for _ in range(0, random.randint(1, 10)):
                    add_to_queue(object, WALK_B)
            else:
                for _ in range(0, random.randint(1, 10)):
                    add_to_queue(object, WALK_F)
    elif distance < 300:
        if random.randint(0, 100) < 60:
            if distance > THEIR_POKE_RANGE(object.char_id):
                rng_queue(object, [DASH_B, DASH_B, DASH_F, POKE(object.char_id, DASH_B, DASH_B, DASH_F, POKE2(object.char_id, DASH_B, DASH_B, DASH_F, POKE(object.char_id), BIG(object.char_id))])
            else:
                rng_queue(object, [DASH_B, DASH_B, DASH_F, POKE(object.char_id), POKE2(object.char_id), POKE(object.char_id), DASH_B, DASH_B, DASH_F, POKE(object.char_id), POKE2(object.char_id), POKE(object.char_id), BIG(object.char_id)])
        else:
            if random.randint(0, 100) < 65:
                for _ in range(0, random.randint(1, 15)):
                    add_to_queue(object, WALK_B)
            else:
                for _ in range(0, random.randint(1, 15)):
                    add_to_queue(object, WALK_F)
    elif distance < PROJECTILE_RANGE:
        if random.randint(0, 100) < 60:
            rng_queue(object, [DASH_F, DASH_F, DASH_F, DASH_F, POKE(object.char_id), POKE2(object.char_id), PROJECTILE(object.char_id), PROJECTILE(object.char_id), TATSU(object.char_id), BIG(object.char_id)])
        else:
            if random.randint(0, 100) < 50:
                for _ in range(0, random.randint(1, 15)):
                    add_to_queue(object, WALK_B)
            else:
                for _ in range(0, random.randint(1, 15)):
                    add_to_queue(object, WALK_F)
    elif distance >= PROJECTILE_RANGE:
        if random.randint(0, 100) < 60:
            if distance > THEIR_PROJECTILE_RANGE(object.char_id):
                rng_queue(object, [DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, PROJECTILE(object.char_id), TATSU(object.char_id)])
            else:
                rng_queue(object, [DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, PROJECTILE(object.char_id), PROJECTILE(object.char_id), TATSU(object.char_id)])
        else:
            if random.randint(0, 100) < 50:
                for _ in range(0, random.randint(1, 15)):
                    add_to_queue(object, WALK_B)
            else:
                for _ in range(0, random.randint(1, 15)):
                    add_to_queue(object, WALK_F)

```

The procedure takes in the computer object, and the positions of both players to make a decision. As it needs to be unpredictable, random choices are included at every stage, similar to how you dont know exactly what a player is thinking.

This queue is processed through the `runCpuQueue` method in the `player` class, which changes the animations and position of the computer based on the queue and whether or not the next instruction is doable by the rules of the game:


```

def runCpuQueue(self):
    if self.isHitstun:
        self.cpuQueue = []
        return

    if len(self.cpuQueue) < 1:
        if self.animListID in ACTIONABLE_LIST and not self.isLeaveBlockstun:
            cpu_data.check_movement(self, players_x[self.playerNum - 1], players_x[self.playerNum], players_y[self.playerNum - 1], players_y[self.playerNum])
            if self.cpuQueue[0] in [get_anim_ID("ForwardDash"), get_anim_ID("Backdash")]:
                self.cpuQueue.insert(0, cpu_data.WAIT)
                self.cpuQueue.insert(0, cpu_data.WAIT)
                self.cpuQueue.insert(0, cpu_data.WAIT)
            else:
                if self.animListID in [get_anim_ID("WalkF"), get_anim_ID("WalkB")]:
                    self.set_new_anim_by_ID()
                    return

        if self.animListID == get_anim_ID("WalkF"):
            if self.is_left:
                self.moveXThisFrame = -stats.WALK_SPEED[self.char_id]
            else:
                self.moveXThisFrame = stats.WALK_SPEED[self.char_id]
        if self.animListID == get_anim_ID("WalkB"):
            if self.is_left:
                self.moveXThisFrame = stats.WALK_SPEED[self.char_id]
            else:
                self.moveXThisFrame = -stats.WALK_SPEED[self.char_id]

    next_command = self.cpuQueue[0]
    if next_command == self.animListID:
        self.cpuQueue.pop(0)
        return

```

```

    if self.animListID in SPECIAL_CANCEL_LIST and self.doPushback:
        other = self.accessOtherPlayer[self.playerNum - 1]
        if not other.isBlockstun:
            if random.randint(0, 100) < 30:
                if not self.isJump:
                    self.set_new_anim_by_ID(cpu_data.SPECIAL_CANCEL[self.char_id])
                else:
                    self.set_new_anim_by_ID(get_anim_ID("SpecialS"))
            else:
                rand_val = random.randint(0, 100)
                if rand_val < 5:
                    if not self.isJump:
                        self.set_new_anim_by_ID(cpu_data.SPECIAL_CANCEL_SAFE[self.char_id])
                    else:
                        self.set_new_anim_by_ID(get_anim_ID("SpecialS"))
                elif rand_val < 8:
                    if not self.isJump:
                        self.set_new_anim_by_ID(cpu_data.SPECIAL_CANCEL[self.char_id])
                    else:
                        self.set_new_anim_by_ID(get_anim_ID("SpecialS"))

    if self.animListID in ACTIONABLE_LIST and ((not self.isJump and next_command in cpu_data.GROUND_MOVE) or (self.isJump and next_command in cpu_data.AIR_MOVE)):
        if next_command == get_anim_ID("ForwardDash"):
            self.set_new_anim_by_ID(next_command)
            self.cpuQueue.pop(0)
            if self.is_left:
                self.moveXThisFrame = -stats.FDASH_SPEED[self.char_id]
            else:
                self.moveXThisFrame = stats.FDASH_SPEED[self.char_id]
        elif next_command == get_anim_ID("BackDash"):
            self.set_new_anim_by_ID(next_command)
            self.cpuQueue.pop(0)
            if self.is_left:
                self.moveXThisFrame = stats.BDASH_SPEED[self.char_id]
            else:
                self.moveXThisFrame = -stats.BDASH_SPEED[self.char_id]
        elif next_command == get_anim_ID("WalkF"):
            self.set_new_anim_by_ID(next_command)
            self.cpuQueue.pop(0)
            if self.is_left:

```

```

                self.moveXThisFrame = stats.WALK_SPEED[self.char_id]
            else:
                self.moveXThisFrame = -stats.WALK_SPEED[self.char_id]
        else:
            self.set_new_anim_by_ID(next_command)
            self.cpuQueue.pop(0)
        return

```

Whenever it is done with an instruction, it pops it out and waits until the next frame to check the next instruction. Whenever the queue is empty, it calls check_movement for the next decision. It clears the queue whenever the computer enters hitstun.

The computer also has logic relating to blocking moves. Within the did_i_block_that method, there is now a CPU selection block that rolls RNG to check if the computer should block that attack:

```

if self.isCpu:
    if random.randint(0, 100) < cpu_data.BLOCK_CHANCE and (self.animListID in ACTIONABLE_LIST or self.animListID in BLOCKING_LIST):
        self.isBlocking = True
        if high_low == -1:
            self.isCrouch = True
        return True

```

In addition, the computer also has logic to determine what it should do after blocking an attack, in a similar way to how the training mod reversal option was coded:

```

if self.isCpu:
    if self.isBlockstun:
        self.isLeaveBlockstun = True
    elif self.isLeaveBlockstun and self.animListID in ACTIONABLE_LIST and not self.isJump:
        rng = random.randint(0, 100)
        if rng < 35:
            self.set_new_anim_by_ID(cpu_data.JAB[self.char_id])
        elif rng < 45:
            self.set_new_anim_by_ID(cpu_data.DP[self.char_id])
        elif rng < 55:
            self.set_new_anim_by_ID(cpu_data.DP2[self.char_id])
        elif rng < 60:
            self.set_new_anim_by_ID(cpu_data.GRAB)
        elif rng < 62:
            self.set_new_anim_by_ID(get_anim_ID("BackDash"))
            if self.is_left:
                self.moveXThisFrame = stats.BDASH_SPEED[self.char_id]
            else:
                self.moveXThisFrame = -stats.BDASH_SPEED[self.char_id]

        self.isLeaveBlockstun = False

```

Alpha Testing

This alpha testing session focused solely on the computer opponent.

Test Number	Test Description	Success?	Details
32	Play against the computer for 6 matches, with the computer playing as all 3 characters twice. Use the rematch button to play against the same character twice.	Yes	--

Stakeholder Response

Stakeholders had a very positive reaction to all additions, enjoying the new character Shawn's moves, and the ability to grab blocking opponents. Stakeholders found the computer opponent challenging but enjoyable to beat.

Code

Here is the full code for the game, separated by python file

main.py

```
#Basically just a launcher
import menu

menu.run()
```

menu.py

```
import os
import time
import sys
import run_game
import turtle
import keyboard
import css
from util import get_controls_from_txt
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 450

MAX_Y = -18
MIN_Y = -104
P2 = [-330,-180]
CPU = [-170,-20]
TRAINING = [0,150]
SETTINGS = [160,310]
VALID_KEYS = [
    "a", "b", "c", "d", "e", "f", "g",
    "h", "i", "j", "k", "l", "m", "n",
    "o", "p", "q", "r", "s", "t", "u",
    "v", "w", "x", "y", "z", "right", "left",
    "up", "down", "shift", "/", ".", ",",
]

screen_instance = 0
is_controls = False

def check_click_pos(x,y):
    global is_controls
    global screen_instance
    if not is_controls:
        if y < MIN_Y or y > MAX_Y: #If outside of the Y area where you'd click buttons just
```

```

dont run this
    return
    if x >= P2[0] and x <= P2[1]: #Checks if your X value is in the button area, no need
to check Y as its already done above
        try:
            css.run([False,False,False,0,0]) #Runs the CSS with arguments that the CSS
will pass onto the run_game module
        except Exception:
            sys.exit() #Closes the game on errors so that it doesnt hang
    elif x >= CPU[0] and x <= CPU[1]:
        try:
            css.run([False,False,False,0,0],True)
        except Exception:
            sys.exit()
    elif x >= TRAINING[0] and x <= TRAINING[1]:
        try:
            css.run([True,True,False,0,0])
        except Exception:
            sys.exit()
    elif x >= SETTINGS[0] and x <= SETTINGS[1]:
        is_controls = True
        run_settings_screen()
    else: #If you're in controls, changes your on click behaviour
        if x > 300 or x < -300 or y < -195 or y > 195:
            run() #Leaves controls
        if y < 85 or y > 135: #If not at the controls button height ignore
            return
        if x < -70 and x > -250: #Player 1 controls
            update_controls(0)
        if x < 250 and x > 70: #P2 controls
            update_controls(1)

def update_controls(player):
    global screen_instance
    controls = get_controls_from_txt()
    new_control_set = controls[player]
    printer = turtle.Turtle()
    screen_instance.update()
    screen_instance.tracer(10)
    printer.showturtle()
    #Goes through each key and gets your binding for it
    keylist = VALID_KEYS
    printer.shape("menu/presskey_left.gif")
    screen_instance.update()
    button = wait_till_button(keylist)
    keylist.remove(button)
    new_control_set[0] = button

```

```

printer.shape("menu/presskey_right.gif")
screen_instance.update()
button = wait_till_button(keylist)
keylist.remove(button)
new_control_set[1] = button
printer.shape("menu/presskey_up.gif")
screen_instance.update()
button = wait_till_button(keylist)
keylist.remove(button)
new_control_set[2] = button
printer.shape("menu/presskey_down.gif")
screen_instance.update()
button = wait_till_button(keylist)
keylist.remove(button)
new_control_set[3] = button
printer.shape("menu/presskey_light.gif")
screen_instance.update()
button = wait_till_button(keylist)
keylist.remove(button)
new_control_set[4] = button
printer.shape("menu/presskey_heavy.gif")
screen_instance.update()
button = wait_till_button(keylist)
keylist.remove(button)
new_control_set[5] = button
printer.shape("menu/presskey_special.gif")
screen_instance.update()
button = wait_till_button(keylist)
keylist.remove(button)
new_control_set[6] = button

controls[player] = new_control_set
f = open("controls.txt", "w")
f.write("")
f.close()
f = open("controls.txt", "a")
f.write(" ".join(controls[0]) + "\n")
f.write(" ".join(controls[1]))
f.close()
printer.hideturtle()
run_settings_screen()

def wait_till_button(key_set):
    global screen_instance
    while True:
        for i in key_set:
            if keyboard.is_pressed(i):
                return i

```

```

        screen_instance.update() #Screen must continually be updated to prevent turtle
freezing!
        time.sleep(0.2)

def run_settings_screen():
    global is_controls
    global screen_instance
    controls = get_controls_from_txt()
    screen_instance.clearscreen()
    screen_instance.tracer(0)
    menuUI = turtle.Turtle()
    menuUI.shape("menu/main.gif")
    controlsUI = turtle.Turtle()
    controlsUI.shape("menu/controls_menu.gif")
    printer = turtle.Turtle()
    printer.penup()
    printer.goto(-70, 55)
    #Sets up settings screen
    for i in controls[0]: #Prints out the control schemes for P1 and P2 by creating turtles
for each key that needs to be shown! Awesome
        file = i
        file = file.replace(",", "comma")
        file = file.replace(".", "dot")
        file = file.replace("/", "fslash")
        file = file.replace("UP", "up")
        file = file.replace("RIGHT", "right")
        file = file.replace("Down", "down")
        file = file.replace("LEFT", "left")
        file = file.replace("SHIFT", "shift")
        printer.shape(f"text/{file}.gif")
        printer.stamp()
        printer.goto(-70, printer.ycor() - 33)
    printer.goto(200, 55)
    for i in controls[1]:
        file = i
        file = file.replace(",", "comma")
        file = file.replace(".", "dot")
        file = file.replace("/", "fslash")
        file = file.replace("UP", "up")
        file = file.replace("RIGHT", "right")
        file = file.replace("Down", "down")
        file = file.replace("LEFT", "left")
        file = file.replace("SHIFT", "shift")
        printer.shape(f"text/{file}.gif")
        printer.stamp()
        printer.goto(200, printer.ycor() - 33)

```

```

printer.goto(200, -500)
screen_instance.update()
screen_instance.tracer(10)
screen_instance.onscreenclick(check_click_pos)
screen_instance.mainloop()

def run(auto=False, chars=[], cpu=False):
    global is_controls
    global screen_instance
    is_controls = False
    turtle.TurtleScreen._RUNNING=True
    screen = turtle.Screen()
    screen_instance = screen
    screen.setup(SCREEN_WIDTH, SCREEN_HEIGHT)
    screen.title("Shotos")
    screen.bgcolor("white")
    for root, dirs, files in os.walk("menu"):
        for filename in files:
            if filename.endswith(".gif"):
                screen.addshape(f"menu/{filename}")
    for root, dirs, files in os.walk("text"):
        for filename in files:
            if filename.endswith(".gif"):
                screen.addshape(f"text/{filename}")
    screen.clearscreen()
    if auto:
        try:
            run_game.run([False, False, False, 0, 0], chars, cpu)
        except Exception:
            sys.exit()
    menuUI = turtle.Turtle()
    menuUI.shape("menu/main.gif")
    screen.onscreenclick(check_click_pos)
    screen.mainloop()

```

stats.py

```
WALK_SPEED = [10.5,13.5,12.0]
HEALTH = [50,45,48]
FDASH_SPEED = [41.0,42.0,49.0]
FDASH_DECEL = [8.0,8.0,9.5]
BDASH_SPEED = [35.0,35.0,32.0]
BDASH_DECEL = [8.0,8.0,7.5]
```

util.py

```
#Contains constants and functioned used between many py files
```

```
ANIMATION_LIST_LABEL = [
```

```
    "Idle",
    "WalkF",
    "WalkB",
    "Crouch",
    "CrouchWait",
    "CrouchRv",
    "JumpSquat",
    "ForwardDash",
    "BackDash",
    "Air",
    "Attack",
    "AttackLw",
    "AttackAir",
    "Heavy",
    "HeavyLw",
    "HeavyAir",
    "Hitstun",
    "Guard",
    "CrouchGuard",
    "GuardAir",
    "SpecialLw",
    "SpecialN",
    "SpecialNEmpty",
    "SpecialS",
    "ThrowWhiff",
    "ThrowF"
```

```
]
```

```
def get_anim_ID(name: str) -> int:
    global ANIMATION_LIST_LABEL
    try:
        value = ANIMATION_LIST_LABEL.index(name)
```



```

        return value
    except ValueError:
        return -1

def get_controls_from_txt() -> list:
    f = open("controls.txt")
    controls = f.readlines()
    for x in range(0, len(controls)):
        i = controls[x]
        i = i.replace("\n", "")
        i = i.split(" ")
        while len(i) < 7:
            i.append("_")
        controls[x] = i
    return controls

```

css.py

```

import os
import time
import sys
import run_game
import turtle
import keyboard
import image_reverser
import menu
from tkinter import PhotoImage
from turtle import Shape
import random
from util import get_controls_from_txt

SCREEN_WIDTH = 800
SCREEN_HEIGHT = 450
FRAME_LENGTH = 5.0
SCALE = 5

CHARACTER_AMOUNT = 3

image_reverser.reverse()

def update(turtle, controls, char_pos_x, char_pos_y) -> int:
    if keyboard.is_pressed(controls[0]) and not keyboard.is_pressed(controls[1]): #Similar
    logic to left/right in run_game.py
        turtle.goto(turtle.xcor()-2, turtle.ycor())

    if keyboard.is_pressed(controls[1]) and not keyboard.is_pressed(controls[0]):

```

```

    turtle.goto(turtle.xcor()+2, turtle.ycor())

if keyboard.is_pressed(controls[2]) and not keyboard.is_pressed(controls[3]):
    turtle.goto(turtle.xcor(), turtle.ycor()+2)

if keyboard.is_pressed(controls[3]) and not keyboard.is_pressed(controls[2]):
    turtle.goto(turtle.xcor(), turtle.ycor()-2)

if keyboard.is_pressed(controls[5]): #Cancel button is the heavy button
    return None

if keyboard.is_pressed(controls[4]): #Attack button is the light button
    x = turtle.xcor()
    y = turtle.ycor()
    for i in range(0, len(char_pos_x)):
        if abs(char_pos_x[i]-x) < 50 and abs(char_pos_y[i]-y) < 50:
            return i #returns char_id based on cursor pos when accept button is pressed

    return -1

def run(training_settings=[False,False,False,0,0], cpu=False):
    #Screen setup
    turtle.TurtleScreen._RUNNING=True
    screen = turtle.Screen()
    screen.setup(SCREEN_WIDTH, SCREEN_HEIGHT)
    screen.title("Shotos")
    screen.bgcolor("white")
    screen.clearscreen()
    for i in ["sprites", "reverse_sprites"]:
        for root, dirs, files in os.walk(i):
            #print(files)
            for filename in files:
                if filename.endswith(".gif"):
                    #print(filename)
                    #Handles scaling up and stuff
                    larger = PhotoImage(file=f"{i}/{filename}").zoom(SCALE, SCALE)
                    screen.addshape(f"{i}/{filename}", Shape("image", larger))
    for root, dirs, files in os.walk("menu"):
        for filename in files:
            if filename.endswith(".gif"):
                screen.addshape(f"menu/{filename}")
    screen.tracer(0)

    chosen_chars = [None, None]

    bg = turtle.Turtle()
    bg.penup()

```

```

bg.shape(f"menu/css.gif")

char_turtle = []
char_pos_x = []
char_pos_y = []
for i in range(0, CHARACTER_AMOUNT):
    f = turtle.Turtle()
    f.penup()
    f.goto(-100 + ((i/(CHARACTER_AMOUNT-1))*200), 25) #Programatically prints out
their heads with equal spacing
    f.shape(f"sprites/F0{i}_Head.gif")
    char_turtle.append(f)
    char_pos_x.append(char_turtle[i].xcor())
    char_pos_y.append(char_turtle[i].ycor())
screen.tracer(10)

p1_show = turtle.Turtle()
p1_show.penup()
p1_show.goto(-200,-50)
p1_show.hideturtle()

p1_name = turtle.Turtle()
p1_name.penup()
p1_name.goto(-200,-200)
p1_name.hideturtle()

p2_show = turtle.Turtle()
p2_show.penup()
p2_show.goto(200,-50)
p2_show.hideturtle()

p2_name = turtle.Turtle()
p2_name.penup()
p2_name.goto(200,-200)
p2_name.hideturtle()

p1_cursor = turtle.Turtle()
p1_cursor.penup()
p1_cursor.goto(-100,-50)
p1_cursor.shape("menu/P2_Cursor.gif")
p2_cursor = turtle.Turtle()
p2_cursor.penup()
p2_cursor.goto(100,-50)
p2_cursor.shape("menu/P1_Cursor.gif")

controls = get_controls_from_txt()

if cpu: #If its a CPU randomly choose their character already!

```

```

    rand_val = random.randint(0, CHARACTER_AMOUNT-1)
    print(rand_val)
    chosen_chars[1] = rand_val
    p2_cursor.goto(char_pos_x[rand_val], char_pos_y[rand_val]-20)
screen.update()
prev_delay = 0
while True:
    #try:
        if prev_delay > FRAME_LENGTH*2:
            pass
            #print("Frame Skip! Performance aint looking good")

    start = time.time()
    p1 = update(p1_cursor, controls[0], char_pos_x, char_pos_y)
    p2 = update(p2_cursor, controls[1], char_pos_x, char_pos_y)

    if p1 != -1:
        chosen_chars[0] = p1
    if p2 != -1:
        chosen_chars[1] = p2
    if chosen_chars[0] != None: #If they have a character selected, show that
character
        p1_show.showturtle()
        p1_show.shape(f"sprites/F0{chosen_chars[0]}_Idle_0.gif") #We can do this
because the sprites are properly named!
        p1_name.showturtle()
        p1_name.shape(f"menu/F0{chosen_chars[0]}_Name.gif")
    else:
        p1_show.hideturtle()
        p1_name.hideturtle()
    if chosen_chars[1] != None:
        p2_show.showturtle()
        p2_show.shape(f"reverse_sprites/F0{chosen_chars[1]}_Idle_0.gif")
        p2_name.showturtle()
        p2_name.shape(f"menu/F0{chosen_chars[1]}_Name.gif")
    else:
        p2_show.hideturtle()
        p2_name.hideturtle()
    if None not in chosen_chars:
        screen.update()
        cancel = True
        for i in range(0,10):
            time.sleep(0.1)
            if keyboard.is_pressed(controls[0][5]) or
keyboard.is_pressed(controls[1][5]):
                cancel = False
                break
        if cancel:

```

```

        break
    screen.update()
    if keyboard.is_pressed("escape"):
        menu.run()
    end = time.time()
    new_delay = (end-start)*10**3
    #print(f"Time taken: {new_delay}ms, normalise: {FRAME_LENGTH -
(new_delay)}ms")
    if (new_delay/1000) < FRAME_LENGTH/1000:
        time.sleep(FRAME_LENGTH/1000 - (new_delay/1000))
    prev_delay = new_delay
    #except Exception as exc:
    #print(exc)
    #sys.exit()
run_game.run(training_settings, chosen_chars, cpu)

```

cpu_data.py

```

from util import get_anim_ID
import random
#Ranges for CPU to be wary of
JAB_RANGE = 25

POKE_RANGE = 35

THEIR_POKE_RANGE = [26,28,40]

PROJECTILE_RANGE = 93

THEIR_PROJECTILE_RANGE = [92,53,130]

#Moves types
JAB = [get_anim_ID("Attack"),get_anim_ID("AttackLw"),get_anim_ID("Attack")]
POKE = [get_anim_ID("AttackLw"),get_anim_ID("Attack"),get_anim_ID("AttackLw")]
POKE2 = [get_anim_ID("AttackLw"),get_anim_ID("Heavy"),get_anim_ID("AttackLw")]
ANTIAIR =
[get_anim_ID("HeavyLw"),get_anim_ID("HeavyLw"),get_anim_ID("HeavyLw")]
BIG = [get_anim_ID("Heavy"),get_anim_ID("SpecialN"),get_anim_ID("Heavy")]
DP = [get_anim_ID("SpecialLw"),get_anim_ID("SpecialLw"),get_anim_ID("SpecialLw")]
DP2 = [get_anim_ID("SpecialLw"),get_anim_ID("SpecialS"),get_anim_ID("SpecialLw")]
TATSU = [get_anim_ID("SpecialS"),get_anim_ID("SpecialS"),get_anim_ID("SpecialS")]
PROJECTILE =
[get_anim_ID("SpecialN"),get_anim_ID("SpecialN"),get_anim_ID("SpecialN")]

SPECIAL_CANCEL = [get_anim_ID("SpecialS"), get_anim_ID("SpecialN"),
get_anim_ID("SpecialS")]

```

```

SPECIAL_CANCEL_SAFE = [get_anim_ID("None"), get_anim_ID("SpecialS"),
get_anim_ID("SpecialS")]

WALK_F = get_anim_ID("WalkF")
WALK_B = get_anim_ID("WalkB")
DASH_F = get_anim_ID("ForwardDash")
WAIT = get_anim_ID("Idle")
DASH_B = get_anim_ID("BackDash")
JUMP = get_anim_ID("JumpSquat")
GRAB = get_anim_ID("ThrowWhiff")

#Move Categories
GROUND_MOVE = [get_anim_ID("Attack"), get_anim_ID("AttackLw"),
get_anim_ID("Heavy"), get_anim_ID("HeavyLw"),
get_anim_ID("SpecialN"), get_anim_ID("SpecialLw"),
get_anim_ID("SpecialS"), get_anim_ID("ThrowWhiff"),
get_anim_ID("WalkF"), get_anim_ID("WalkB"), get_anim_ID("ForwardDash"),
get_anim_ID("BackDash"), get_anim_ID("JumpSquat"), get_anim_ID("Idle")
]
AIR_MOVE = [get_anim_ID("AttackAir"), get_anim_ID("HeavyAir"),
get_anim_ID("SpecialS")]

#Parameters
BLOCK_CHANCE = 65

def add_to_queue(object, anim): #Adds an action to the queue
    object.cpuQueue.append(anim)

def rng_queue(object, choices): #Adds a random action to the queue based on a list of
choices
    object.cpuQueue.append(random.choice(choices))

def check_movement(object, yourX, theirX, yourY, theirY):
    if object.isJump:
        add_to_queue(object, WAIT)
        return
    distance = int(abs(yourX-theirX)/10) + 10
    distanceY = int(abs(yourY-theirY)/10)
    if distance < JAB_RANGE: #Up Close
        if random.randint(0, 100) < 80:
            if distanceY < 3: #If theyre on the same level or ever so slightly above
                rng_queue(object, [DASH_B, JAB[object.char_id], JAB[object.char_id],
GRAB])
            elif distanceY < 25: #If theyre above but not too high to antiair
                rng_queue(object, [DASH_B, ANTI-AIR[object.char_id],
ANTI-AIR[object.char_id], DP[object.char_id]])
        else:
            add_to_queue(object, WAIT)

```

```

else:
    if random.randint(0, 100) < 80:
        for _ in range(0, random.randint(1, 10)):
            add_to_queue(object, WALK_B)
    else:
        for _ in range(0, random.randint(1, 10)):
            add_to_queue(object, WALK_F)
elif distance < POKE_RANGE: #Midrange spacing, prime footsies area
    if random.randint(0, 100) < 68:
        if distance > THEIR_POKE_RANGE[object.char_id]:
            rng_queue(object, [DASH_B, DASH_B, DASH_F,
POKE[object.char_id], DASH_B, DASH_B, DASH_F,
POKE2[object.char_id], DASH_B, DASH_B, DASH_F,
POKE[object.char_id], BIG[object.char_id]])
        else:
            rng_queue(object, [DASH_B, DASH_B, DASH_F, POKE[object.char_id],
POKE2[object.char_id], POKE[object.char_id], DASH_B, DASH_B, DASH_F,
POKE[object.char_id], POKE2[object.char_id],
POKE[object.char_id], DASH_B, DASH_B, DASH_F,
POKE[object.char_id], POKE2[object.char_id], POKE[object.char_id],
BIG[object.char_id]])
    else:
        if random.randint(0, 100) < 65:
            for _ in range(0, random.randint(1, 15)):
                add_to_queue(object, WALK_B)
        else:
            for _ in range(0, random.randint(1, 15)):
                add_to_queue(object, WALK_F)
elif distance < PROJECTILE_RANGE: #Within projectile spacing, far range
    if random.randint(0, 100) < 68:
        rng_queue(object, [DASH_F, DASH_F, DASH_F, DASH_F,
POKE[object.char_id], POKE2[object.char_id], PROJECTILE[object.char_id],
PROJECTILE[object.char_id], TATSU[object.char_id], BIG[object.char_id]])
    else:
        if random.randint(0, 100) < 50:
            for _ in range(0, random.randint(1, 15)):
                add_to_queue(object, WALK_B)
        else:
            for _ in range(0, random.randint(1, 15)):
                add_to_queue(object, WALK_F)
elif distance >= PROJECTILE_RANGE: #Very far range, no normals are really hitting
over here
    if random.randint(0, 100) < 68:
        if distance > THEIR_PROJECTILE_RANGE[object.char_id]:
            rng_queue(object, [DASH_F, DASH_F, DASH_F, DASH_F, DASH_F,
DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F,
PROJECTILE[object.char_id], TATSU[object.char_id],
TATSU[object.char_id]

```

```

    ])
    else:
        rng_queue(object, [DASH_F, DASH_F, DASH_F, DASH_F, DASH_F,
DASH_F, DASH_F, DASH_F, DASH_F, DASH_F, DASH_F,
PROJECTILE[object.char_id], PROJECTILE[object.char_id],
PROJECTILE[object.char_id], TATSU[object.char_id]
        ])
    else:
        if random.randint(0, 100) < 50:
            for _ in range(0, random.randint(1, 15)):
                add_to_queue(object, WALK_B)
        else:
            for _ in range(0, random.randint(1, 15)):
                add_to_queue(object, WALK_F)

```

animations.py

```

#Animations

RYU = [
    [ #Idle
        ["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_3.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_4.gif", [-11.5,11.5,-25,14], None, None, None]
    ],
    [ #WalkF
        ["sprites/F00_Forward_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Forward_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Forward_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Forward_3.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Forward_4.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Forward_5.gif", [-11.5,11.5,-25,14], None, None, None]
    ],
    [ #WalkB
        ["sprites/F00_Backward_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Backward_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Backward_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Backward_3.gif", [-11.5,11.5,-25,14], None, None, None],

```



```

["sprites/F00_Backward_4.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_Backward_5.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #Crouch
["sprites/F00_Crouch_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F00_Crouch_1.gif", [-11.5,11.5,-25,2], None, None, None]
],
[ #CrouchWait
["sprites/F00_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F00_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F00_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F00_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F00_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F00_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F00_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F00_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None]
],
[ #CrouchRv
["sprites/F00_Crouch_1.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F00_Crouch_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #JumpSquat
["sprites/F00_JumpSquat_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_JumpSquat_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #ForwardDash
["sprites/F00_ForwardDash_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_ForwardDash_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_ForwardDash_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_ForwardDash_3.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #BackDash
["sprites/F00_BackwardDash_0.gif", None, None, None, None],
["sprites/F00_BackwardDash_0.gif", None, None, None, None],
["sprites/F00_BackwardDash_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_BackwardDash_2.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #Air
["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_Air_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_Air_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_Air_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F00_Air_1.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #AttackN
["sprites/F00_Attack_2.gif", [-11.5,11.5,-25,14], [8, 18, -11, 1], [0, 1, 3, 6, 0, 1], [5,

```

```

22, -12, 1]],
    ["sprites/F00_Attack_2.gif", [-11.5,11.5,-25,14], [8, 18, -11, 1], [0, 1, 3, 6, 0, 1], [5,
22, -12, 1]],
    ["sprites/F00_Attack_1.gif", [-11.5,11.5,-25,14], None, None, [5, 22, -12, 1]],
    ["sprites/F00_Attack_4.gif", [-11.5,11.5,-25,14], None, None, [5, 22, -12, 1]],
    ["sprites/F00_Attack_5.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Attack_5.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #AttackLw
    ["sprites/F00_AttackLw_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F00_AttackLw_1.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F00_AttackLw_2.gif", [-11.5,11.5,-25,2], [3, 25, -25, -12], [-1, 1, 4, 15, 0,
1], [3, 28, -25, -12]],
    ["sprites/F00_AttackLw_2.gif", [-11.5,11.5,-25,2], [3, 25, -25, -12], [-1, 1, 4, 15, 0,
1], [3, 28, -25, -12]],
    ["sprites/F00_AttackLw_2.gif", [-11.5,11.5,-25,2], None, None, [3, 28, -25, -12]],
    ["sprites/F00_AttackLw_3.gif", [-11.5,11.5,-25,2], None, None, [3, 28, -25, -12]],
    ["sprites/F00_AttackLw_4.gif", [-11.5,11.5,-25,2], None, None, [3, 28, -25, -12]],
    ["sprites/F00_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F00_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None]
],
[ #AttackAir
    ["sprites/F00_AttackAir_0.gif", [-11.5,11.5,-23,12], None, None, None],
    ["sprites/F00_AttackAir_1.gif", [-11.5,11.5,-23,12], [3, 15, -12, 3], [1, 1, 3, 10, 15, 3],
[2, 16, -13, 4]],
    ["sprites/F00_AttackAir_1.gif", [-11.5,11.5,-23,12], [3, 15, -12, 3], [1, 1, 3, 10, 15, 3],
[2, 16, -13, 4]],
    ["sprites/F00_AttackAir_1.gif", [-11.5,11.5,-23,12], [3, 15, -12, 3], [1, 1, 3, 10, 15, 3],
[2, 16, -13, 4]],
    ["sprites/F00_AttackAir_2.gif", [-11.5,11.5,-23,12], None, None, None],
    ["sprites/F00_AttackAir_2.gif", [-11.5,11.5,-23,12], None, None, None],
    ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #HeavyN
    ["sprites/F00_Heavy_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Heavy_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Heavy_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Heavy_3.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Heavy_4.gif", [-11.5,7,-25,14], None, None, None],
    ["sprites/F00_Heavy_4.gif", [-11.5,7,-25,14], None, None, None],
    ["sprites/F00_Heavy_5.gif", [-11.5,7,-25,14], [6, 30, -6, 5], [1, 3.5, 8, 45, 45, 1], [6,
25, -10, 18]],
    ["sprites/F00_Heavy_5.gif", [-11.5,7,-25,14], [6, 30, -6, 5], [1, 3.5, 8, 45, 45, 1], [6,
25, -10, 18]],
    ["sprites/F00_Heavy_5.gif", [-11.5,7,-25,14], [6, 30, -6, 5], [1, 3.5, 8, 45, 45, 1], [6,

```

```

25, -10, 18]],
    ["sprites/F00_Heavy_5.gif", [-11.5,7,-25,14], None, None, [6, 25, -10, 18]],
    ["sprites/F00_Heavy_6.gif", [-11.5,7,-25,14], None, None, None],
    ["sprites/F00_Heavy_6.gif", [-11.5,7,-25,14], None, None, None],
    ["sprites/F00_Heavy_7.gif", [-11.5,7,-25,14], None, None, None],
    ["sprites/F00_Heavy_7.gif", [-11.5,7,-25,14], None, None, None],
    ["sprites/F00_Idle_0.gif", [-11.5,7,-25,14], None, None, None],
],
[ #HeavyLw
    ["sprites/F00_HeavyLw_0.gif", [-11.5,11.5,-25,3], None, None, None],
    ["sprites/F00_HeavyLw_1.gif", [-11.5,11.5,-25,-10], None, None, None],
    ["sprites/F00_HeavyLw_2.gif", [-11.5,11.5,-25,-10], None, None, None],
    ["sprites/F00_HeavyLw_3.gif", [-11.5,8,-25,-10], [3, 13, -2, 17], [0, 2.75, 4, 55, 55,
3], None],
    ["sprites/F00_HeavyLw_3.gif", [-11.5,8,-25,-10], [3, 13, -2, 17], [0, 2.75, 4, 55, 55,
3], None],
    ["sprites/F00_HeavyLw_3.gif", [-11.5,8,-25,-10], [3, 13, -2, 17], [0, 2.75, 4, 55, 55,
3], None],
    ["sprites/F00_HeavyLw_2.gif", [-11.5,11.5,-25,3], None, None, None],
    ["sprites/F00_HeavyLw_2.gif", [-11.5,11.5,-25,3], None, None, None],
    ["sprites/F00_HeavyLw_1.gif", [-11.5,11.5,-25,3], None, None, None],
    ["sprites/F00_HeavyLw_0.gif", [-11.5,11.5,-25,3], None, None, None],
],
[ #HeavyAir
    ["sprites/F00_HeavyAir_0.gif", [-11.5,11.5,-20,12], None, None, None],
    ["sprites/F00_HeavyAir_1.gif", [-11.5,11.5,-20,12], None, None, None],
    ["sprites/F00_HeavyAir_2.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 3], [1, 2, 6, 15, 0, 3],
[-5, 30, -10, 3]],
    ["sprites/F00_HeavyAir_2.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 3], [1, 2, 6, 15, 0, 3],
[-5, 30, -10, 3]],
    ["sprites/F00_HeavyAir_2.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 3], [1, 2, 6, 15, 0, 3],
[-5, 30, -10, 3]],
    ["sprites/F00_HeavyAir_2.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 3], [1, 2, 6, 15, 0, 3],
[-5, 30, -10, 3]],
    ["sprites/F00_HeavyAir_2.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 3], [1, 2, 6, 15, 0, 3],
[-5, 30, -10, 3]],
    ["sprites/F00_HeavyAir_0.gif", [-11.5,11.5,-20,12], None, None, None],
    ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #Hitstun
    ["sprites/F00_Hitstun_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Hitstun_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Hitstun_1.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #Guard
    ["sprites/F00_Guard_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Guard_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Guard_1.gif", [-11.5,11.5,-25,14], None, None, None]
]

```

```
[
    [
        ["sprites/F00_CrouchGuard_0.gif", [-11.5,11.5,-25,2], None, None, None],
        ["sprites/F00_CrouchGuard_0.gif", [-11.5,11.5,-25,2], None, None, None],
        ["sprites/F00_CrouchGuard_1.gif", [-11.5,11.5,-25,2], None, None, None]
    ],
    [
        ["sprites/F00_GuardAir_0.gif", [-11.5,11.5,-25,2], None, None, None],
        ["sprites/F00_GuardAir_0.gif", [-11.5,11.5,-25,2], None, None, None],
        ["sprites/F00_GuardAir_0.gif", [-11.5,11.5,-25,2], None, None, None]
    ],
    [
        ["sprites/F00_SpecialLw_0.gif", [-11.5,11.5,-25,4], None, None, None],
        ["sprites/F00_SpecialLw_1.gif", [-11.5,11.5,-25,4], None, None, None],
        ["sprites/F00_SpecialLw_2.gif", None, [7, 28, -10, 5], [0, 3, 10, 20, 100, 3], None],
        ["sprites/F00_SpecialLw_3.gif", None, [7, 18, -8, 25], [0, 3, 10, 20, 100, 3], None],
        ["sprites/F00_SpecialLw_3.gif", None, [7, 18, -8, 25], [0, 3, 10, 20, 100, 3], None],
        ["sprites/F00_SpecialLw_3.gif", [-11.5,11.5,-22,25], None, None, None],
        ["sprites/F00_SpecialLw_4.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_SpecialLw_5.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_SpecialLw_6.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_SpecialLw_6.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Air_0.gif", [-11.5,11.5,-25,14], None, None, None]
    ],
    [
        ["sprites/F00_SpecialN_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_SpecialN_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_SpecialN8.gif", [-11.5,11.5,-25,14], [17, 27, -4, 2], [0, 1, 3, 0, -1, 2],
None],
        ["sprites/F00_SpecialN9.gif", [-11.5,11.5,-25,14], [30, 40, -4, 2], [0, 0.75, 3, 0, -1,
2], None],
        ["sprites/F00_SpecialN10.gif", [-11.5,11.5,-25,14], [43, 53, -4, 2], [0, 0.75, 3, 0, -1,
2], None],
        ["sprites/F00_SpecialN11.gif", [-11.5,11.5,-25,14], [56, 66, -4, 2], [0, 0.5, 3, 0, -1,
2], None],
        ["sprites/F00_SpecialN12.gif", [-11.5,11.5,-25,14], [69, 79, -4, 2], [0, 0.5, 3, 0, -1,
2], None],
        ["sprites/F00_SpecialN13.gif", [-11.5,11.5,-25,14], [82, 92, -4, 2], [0, 0.5, 3, 0, -1,
```

```

2], None],
    ["sprites/F00_SpecialN_6.gif", [-11.5,11.5,-25,14], [82, 92, -4, 2], [0, 0.5, 3, 0, -1,
2], None],
    ["sprites/F00_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_7.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #SpecialNEmpty
    ["sprites/F00_SpecialN_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialN_7.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #SpecialS
    ["sprites/F00_SpecialS_0.gif", [-11.5,11.5,-25,-10], None, None, None],
    ["sprites/F00_SpecialS_0.gif", [-11.5,11.5,-25,-10], None, None, None],
    ["sprites/F00_SpecialS_0.gif", [-11.5,11.5,-25,-10], None, None, None],
    ["sprites/F00_SpecialS_1.gif", [-11.5,11.5,-25,14], [3, 25, -6, 3], [0, 2, 10, 45, 50, 1],
[3, 26, -10, 3]],
    ["sprites/F00_SpecialS_1.gif", [-11.5,11.5,-25,14], [3, 25, -6, 3], [0, 2, 10, 45, 50, 1],
[3, 26, -10, 3]],
    ["sprites/F00_SpecialS_5.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialS_5.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialS_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialS_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialS_4.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialS_4.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialS_1.gif", [-11.5,11.5,-25,14], [3, 25, -6, 3], [0, 0.5, 5, 45, 50,
1], [3, 26, -10, 3]],
    ["sprites/F00_SpecialS_1.gif", [-11.5,11.5,-25,14], [3, 25, -6, 3], [0, 0.5, 5, 45, 50,
1], [3, 26, -10, 3]],
    ["sprites/F00_SpecialS_3.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialS_3.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialS_3.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F00_SpecialS_3.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #ThrowWhiff
    ["sprites/F00_ThrowWhiff_0.gif", [-11.5,11.5,-25,14], None, None, None],

```

```

        ["sprites/F00_ThrowWhiff_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], [0, 20, -15, 0], [2, 0, 13, -45,
45, 0], [0, 24, -25, 0]],
        ["sprites/F00_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], [0, 20, -15, 0], [2, 0, 13, -45,
45, 0], [0, 24, -25, 0]],
        ["sprites/F00_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
        ["sprites/F00_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
        ["sprites/F00_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
        ["sprites/F00_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
    ],
    [ #ThrowF
        ["sprites/F00_ThrowF_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_ThrowF_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_ThrowF_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_ThrowF_1.gif", [-11.5,11.5,-25,14], [-250, 250, -100, 100], [2, 2, 10,
40, 45, 0], None],
        ["sprites/F00_ThrowF_1.gif", [-11.5,11.5,-25,14], [-250, 250, -100, 100], [2, 2, 10,
40, 45, 0], None],
        ["sprites/F00_ThrowF_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_ThrowF_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_ThrowF_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_ThrowF_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_ThrowF_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F00_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
    ]
]

KEN = [
    [ #Idle
        ["sprites/F01_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F01_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F01_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F01_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F01_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F01_Idle_3.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F01_Idle_4.gif", [-11.5,11.5,-25,14], None, None, None]
    ],
    [ #WalkF
        ["sprites/F01_Forward_0.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F01_Forward_1.gif", [-11.5,11.5,-25,14], None, None, None],
        ["sprites/F01_Forward_2.gif", [-11.5,11.5,-25,14], None, None, None],

```



```

["sprites/F01_Forward_3.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Forward_4.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Forward_5.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #WalkB
["sprites/F01_Backward_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Backward_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Backward_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Backward_3.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Backward_4.gif", [-11.5,11.5,-25,14], None, None, None]#,
#["sprites/F01_Backward_5.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #Crouch
["sprites/F01_Crouch_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F01_Crouch_1.gif", [-11.5,11.5,-25,2], None, None, None]
],
[ #CrouchWait
["sprites/F01_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F01_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F01_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F01_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F01_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F01_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F01_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F01_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None]
],
[ #CrouchRv
["sprites/F01_Crouch_1.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F01_Crouch_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #JumpSquat
["sprites/F01_JumpSquat_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_JumpSquat_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #ForwardDash
["sprites/F01_ForwardDash_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_ForwardDash_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_ForwardDash_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_ForwardDash_3.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #BackDash
["sprites/F01_BackwardDash_0.gif", None, None, None, None],
["sprites/F01_BackwardDash_0.gif", None, None, None, None],
["sprites/F01_BackwardDash_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_BackwardDash_2.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #Air

```

```

["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Air_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Air_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Air_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Air_1.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #AttackN
["sprites/F01_Attack_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Attack_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Attack_2.gif", [-11.5,11.5,-25,14], [8, 28, -18, -1], [0, 1, 4, 30, 0, 1], [8,
29, -18, 2]],
["sprites/F01_Attack_2.gif", [-11.5,11.5,-25,14], [8, 28, -18, -1], [0, 1, 4, 30, 0, 1], [8,
29, -18, 2]],
["sprites/F01_Attack_2.gif", [-11.5,11.5,-25,14], [8, 28, -18, -1], [0, 1, 4, 30, 0, 1], [8,
29, -18, 2]],
["sprites/F01_Attack_2.gif", [-11.5,11.5,-25,14], [8, 28, -18, -1], [0, 1, 4, 30, 0, 1], [8,
29, -18, 2]],
["sprites/F01_Attack_3.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Attack_3.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #AttackLw
["sprites/F01_AttackLw_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F01_AttackLw_1.gif", [-11.5,11.5,-25,2], [4, 20, -20, -5], [0, 1, 3, 10, 0, 2],
[4, 21, -18, -2]],
["sprites/F01_AttackLw_1.gif", [-11.5,11.5,-25,2], [4, 20, -20, -5], [0, 1, 3, 10, 0, 2],
[4, 21, -18, -2]],
["sprites/F01_AttackLw_2.gif", [-11.5,11.5,-25,2], None, None, None]
],
[ #AttackAir
["sprites/F01_AttackAir_1.gif", [-11.5,11.5,-23,12], [-2, 12, -16, -8], [1, 1, 3, 20, 0,
3], [-4, 14, -18, -10]],
["sprites/F01_AttackAir_1.gif", [-11.5,11.5,-23,12], [-2, 12, -16, -8], [1, 1, 3, 20, 0,
3], [-4, 14, -18, -10]],
["sprites/F01_AttackAir_1.gif", [-11.5,11.5,-23,12], [-2, 12, -16, -8], [1, 1, 3, 20, 0,
3], [-4, 14, -18, -10]],
["sprites/F01_AttackAir_1.gif", [-11.5,11.5,-23,12], [-2, 12, -16, -8], [1, 1, 3, 20, 0,
3], [-4, 14, -18, -10]],
["sprites/F01_AttackAir_1.gif", [-11.5,11.5,-23,12], [-2, 12, -16, -8], [1, 1, 3, 20, 0,
3], [-4, 14, -18, -10]],
["sprites/F01_AttackAir_1.gif", [-11.5,11.5,-23,12], [-2, 12, -16, -8], [1, 1, 3, 20, 0,
3], [-4, 14, -18, -10]],
["sprites/F01_AttackAir_1.gif", [-11.5,11.5,-23,12], [-2, 12, -16, -8], [1, 1, 3, 20, 0,
3], [-4, 14, -18, -10]],
["sprites/F01_AttackAir_0.gif", [-11.5,11.5,-23,12], None, None, None],
["sprites/F01_AttackAir_0.gif", [-11.5,11.5,-23,12], None, None, None],
["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],

```



```

["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #HeavyN
["sprites/F01_Heavy_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Heavy_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Heavy_2.gif", [-11.5,11.5,-25,14], [3, 27, -2, 3], [0, 2.8, 6, 45, 0, 3],
None],
["sprites/F01_Heavy_2.gif", [-11.5,11.5,-25,14], [3, 27, -2, 3], [0, 2.8, 6, 45, 0, 3],
None],
["sprites/F01_Heavy_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Heavy_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Heavy_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Heavy_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Heavy_3.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #HeavyLw
["sprites/F01_HeavyLw_0.gif", [-11.5,11.5,-25,8], None, None, None],
["sprites/F01_HeavyLw_1.gif", [-11.5,11.5,-25,8], None, None, None],
["sprites/F01_HeavyLw_2.gif", [-11.5,11.5,-25,8], None, None, None],
["sprites/F01_HeavyLw_3.gif", [-11.5,8,-25,8], [3, 20, -2, 17], [0, 3, 7, -35, 90, 3],
None],
["sprites/F01_HeavyLw_3.gif", [-11.5,8,-25,8], [3, 20, -2, 17], [0, 3, 7, -35, 90, 3],
None],
["sprites/F01_HeavyLw_3.gif", [-11.5,8,-25,8], [3, 20, -2, 17], [0, 3, 7, -35, 90, 3],
None],
["sprites/F01_HeavyLw_2.gif", [-11.5,11.5,-25,8], None, None, None],
["sprites/F01_HeavyLw_2.gif", [-11.5,11.5,-25,8], None, None, None],
["sprites/F01_HeavyLw_1.gif", [-11.5,11.5,-25,8], None, None, None],
["sprites/F01_HeavyLw_0.gif", [-11.5,11.5,-25,8], None, None, None]
],
[ #HeavyAir
["sprites/F01_HeavyAir_0.gif", [-11.5,11.5,-20,12], None, None, None],
["sprites/F01_HeavyAir_1.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 7], [1, 2, 4, 15, 40,
2], [-5, 29, -13, 10]],
["sprites/F01_HeavyAir_1.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 7], [1, 2, 4, 15, 40,
2], [-5, 29, -13, 10]],
["sprites/F01_HeavyAir_1.gif", [-11.5,11.5,-20,12], [-5, 28, -10, 7], [1, 2, 4, 15, 40,
2], [-5, 29, -13, 10]],
["sprites/F01_HeavyAir_2.gif", [-11.5,11.5,-20,12], None, None, None],
["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #Hitstun
["sprites/F01_Hitstun_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Hitstun_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Hitstun_1.gif", [-11.5,11.5,-25,14], None, None, None]
],

```

```
[ #Guard
    ["sprites/F01_Guard_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Guard_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Guard_1.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #CrouchGuard
    ["sprites/F01_CrouchGuard_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F01_CrouchGuard_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F01_CrouchGuard_1.gif", [-11.5,11.5,-25,2], None, None, None]
],
[ #GuardAir
    ["sprites/F01_GuardAir_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F01_GuardAir_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F01_GuardAir_0.gif", [-11.5,11.5,-25,2], None, None, None]
],
[ #SpecialLw
    ["sprites/F01_SpecialLw_0.gif", [-11.5,11.5,-25,4], None, None, None],
    ["sprites/F01_SpecialLw_1.gif", None, None, None, None],
    ["sprites/F01_SpecialLw_2.gif", None, [7, 23, -10, 5], [0, 5, 10, 0, 60, 3], None],
    ["sprites/F01_SpecialLw_2.gif", None, [7, 23, -10, 5], [0, 5, 10, 0, 60, 3], None],
    ["sprites/F01_SpecialLw_3.gif", None, None, None, None],
    ["sprites/F01_SpecialLw_3.gif", None, [7, 18, -8, 25], [0, 0, 10, 20, 100, 3], None],
    ["sprites/F01_SpecialLw_3.gif", None, [7, 18, -8, 25], [0, 0, 10, 20, 100, 3], None],
    ["sprites/F01_SpecialLw_3.gif", [-11.5,11.5,-22,25], None, None, None],
    ["sprites/F01_SpecialLw_4.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialLw_5.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialLw_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialLw_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Air_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #SpecialN
    ["sprites/F01_SpecialN_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN8.gif", [-11.5,11.5,-25,14], [17, 27, -4, 2], [0, 1.2, 6, 0, 60, 2],
None],
    ["sprites/F01_SpecialN9.gif", [-11.5,11.5,-25,14], [23.5, 33.5, -4, 2], [0, 1.2, 6, 0, 60,
2], None],
    ["sprites/F01_SpecialN10.gif", [-11.5,11.5,-25,14], [30, 40, -4, 2], [0, 1.2, 6, 0, 60,
2], None],
```

```

    ["sprites/F01_SpecialN11.gif", [-11.5,11.5,-25,14], [36.5, 46.5, -4, 2], [0, 1.2, 6, 0,
60, 2], None],
    ["sprites/F01_SpecialN12.gif", [-11.5,11.5,-25,14], [43, 53, -4, 2], [0, 1.2, 6, 0, 60,
2], None],
    ["sprites/F01_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN_7.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #SpecialNEmpty
    ["sprites/F01_SpecialN_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_SpecialN_6.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #SpecialS
    ["sprites/F01_SpecialS_5.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F01_SpecialS_5.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F01_SpecialS_1.gif", None, None, None, None],
    ["sprites/F01_SpecialS_2.gif", None, None, None, None],
    ["sprites/F01_SpecialS_3.gif", None, None, None, None],
    ["sprites/F01_SpecialS_4.gif", None, None, None, None],
    ["sprites/F01_SpecialS_1.gif", None, None, None, None],
    ["sprites/F01_SpecialS_2.gif", None, None, None, None],
    ["sprites/F01_SpecialS_3.gif", None, None, None, None],
    ["sprites/F01_SpecialS_4.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F01_SpecialS_5.gif", [-11.5,11.5,-25,2], None, None, None]
],
[ #ThrowWhiff
    ["sprites/F01_ThrowWhiff_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_ThrowWhiff_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], [0, 20, -15, 0], [2, 0, 13, -45,
45, 0], [0, 24, -25, 0]],
    ["sprites/F01_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], [0, 20, -15, 0], [2, 0, 13, -45,
45, 0], [0, 24, -25, 0]],
    ["sprites/F01_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
    ["sprites/F01_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
    ["sprites/F01_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
    ["sprites/F01_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F01_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #ThrowF

```

```

["sprites/F01_ThrowF_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_ThrowF_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_ThrowF_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_ThrowF_1.gif", [-11.5,11.5,-25,14], [-250, 250, -100, 100], [2, 2, 10,
40, 45, 0], None],
["sprites/F01_ThrowF_1.gif", [-11.5,11.5,-25,14], [-250, 250, -100, 100], [2, 2, 10,
40, 45, 0], None],
["sprites/F01_ThrowF_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_ThrowF_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_ThrowF_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_ThrowF_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_ThrowF_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F01_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
]
]

```

SEAN = [

[#Idle

```

["sprites/F02_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Idle_3.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Idle_4.gif", [-11.5,11.5,-25,14], None, None, None]

```

],

[#WalkF

```

["sprites/F02_Forward_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Forward_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Forward_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Forward_3.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Forward_4.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Forward_5.gif", [-11.5,11.5,-25,14], None, None, None]

```

],

[#WalkB

```

["sprites/F02_Backward_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Backward_1.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Backward_2.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Backward_3.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Backward_4.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Backward_5.gif", [-11.5,11.5,-25,14], None, None, None]

```

],

[#Crouch

```

["sprites/F02_Crouch_0.gif", [-11.5,11.5,-25,2], None, None, None],
["sprites/F02_Crouch_1.gif", [-11.5,11.5,-25,2], None, None, None]

```

],

```

[ #CrouchWait
    ["sprites/F02_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F02_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F02_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F02_CrouchWait_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F02_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F02_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F02_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F02_CrouchWait_1.gif", [-11.5,11.5,-25,2], None, None, None]
],
[ #CrouchRv
    ["sprites/F02_Crouch_1.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F02_Crouch_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #JumpSquat
    ["sprites/F02_JumpSquat_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_JumpSquat_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #ForwardDash
    ["sprites/F02_ForwardDash_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ForwardDash_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ForwardDash_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ForwardDash_3.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #BackDash
    ["sprites/F02_BackwardDash_0.gif", None, None, None, None],
    ["sprites/F02_BackwardDash_0.gif", None, None, None, None],
    ["sprites/F02_BackwardDash_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_BackwardDash_2.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #Air
    ["sprites/F02_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Air_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Air_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Air_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Air_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Air_1.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #AttackN
    ["sprites/F02_Attack_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Attack_1.gif", [-11.5,11.5,-25,14], [7, 23, -2, 3], [0, 1, 3, 6, 0, 1], [7,
25, -4, 5]],
    ["sprites/F02_Attack_1.gif", [-11.5,11.5,-25,14], [7, 23, -2, 3], [0, 1, 3, 6, 0, 1], [7,
25, -4, 5]],
    ["sprites/F02_Attack_1.gif", [-11.5,11.5,-25,14], None, None, [7, 25, -4, 5]],
    ["sprites/F02_Attack_2.gif", [-11.5,11.5,-25,14], None, None, [7, 25, -4, 5]],
    ["sprites/F02_Attack_3.gif", [-11.5,11.5,-25,14], None, None, None]
]

```

```

],
[ #AttackLw
    ["sprites/F02_AttackLw_0.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F02_AttackLw_1.gif", [-11.5,11.5,-25,2], None, None, None],
    ["sprites/F02_AttackLw_2.gif", [-11.5,11.5,-25,2], [3, 29, -25, -15], [-1, 2.5, 4, 20, 0,
1], [3, 30, -25, -10]],
    ["sprites/F02_AttackLw_2.gif", [-11.5,11.5,-25,2], [3, 29, -25, -15], [-1, 2.5, 4, 20, 0,
1], [3, 30, -25, -10]],
    ["sprites/F02_AttackLw_2.gif", [-11.5,11.5,-25,2], [3, 29, -25, -15], [-1, 2.5, 4, 20, 0,
1], [3, 30, -25, -10]],
    ["sprites/F02_AttackLw_2.gif", [-11.5,11.5,-25,2], [3, 29, -25, -15], [-1, 2.5, 4, 20, 0,
1], [3, 30, -25, -10]],
    ["sprites/F02_AttackLw_2.gif", [-11.5,11.5,-25,2], [3, 29, -25, -15], [-1, 2.5, 4, 20, 0,
1], [3, 30, -25, -10]],
    ["sprites/F02_AttackLw_2.gif", [-11.5,11.5,-25,2], [3, 29, -25, -15], [-1, 2.5, 4, 20, 0,
1], [3, 30, -25, -10]],
    ["sprites/F02_AttackLw_2.gif", [-11.5,11.5,-25,2], None, None, [3, 28, -25, -12]],
    ["sprites/F02_AttackLw_3.gif", [-11.5,11.5,-25,2], None, None, [3, 28, -25, -12]],
    ["sprites/F02_AttackLw_4.gif", [-11.5,11.5,-25,2], None, None, [3, 28, -25, -12]],
    ["sprites/F02_Crouch_1.gif", [-11.5,11.5,-25,2], None, None, [3, 28, -25, -12]],
    ["sprites/F02_Crouch_1.gif", [-11.5,11.5,-25,2], None, None, [3, 28, -25, -12]]
],
[ #AttackAir
    ["sprites/F02_AttackAir_0.gif", [-11.5,11.5,-23,12], [3, 17, -3, 3], [1, 1, 3, 10, 15, 3],
[2, 20, -6, 6]],
    ["sprites/F02_AttackAir_0.gif", [-11.5,11.5,-23,12], [3, 17, -3, 3], [1, 1, 3, 10, 15, 3],
[2, 20, -6, 6]],
    ["sprites/F02_AttackAir_0.gif", [-11.5,11.5,-23,12], None, None, None],
    ["sprites/F02_AttackAir_0.gif", [-11.5,11.5,-23,12], None, None, None],
    ["sprites/F02_AttackAir_0.gif", [-11.5,11.5,-23,12], None, None, None],
    ["sprites/F02_AttackAir_1.gif", [-11.5,11.5,-23,12], None, None, None],
    ["sprites/F02_AttackAir_1.gif", [-11.5,11.5,-23,12], None, None, None],
    ["sprites/F02_AttackAir_1.gif", [-11.5,11.5,-23,12], None, None, None],
    ["sprites/F02_Air_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #HeavyN
    ["sprites/F02_Idle_0.gif", [-11.5,7,-25,14], None, None, None],
    ["sprites/F02_Heavy_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Heavy_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Heavy_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Heavy_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Heavy_2.gif", [-11.5,11.5,-25,14], [2, 23, -13, 7], [1, 4, 8, 10, -45, 1],
[2, 25, -13, 8]],
    ["sprites/F02_Heavy_2.gif", [-11.5,11.5,-25,14], [2, 23, -13, 7], [1, 4, 8, 10, -45, 1],
[2, 25, -13, 8]],
    ["sprites/F02_Heavy_2.gif", [-11.5,11.5,-25,14], None, None, [2, 25, -13, 8]],
    ["sprites/F02_Heavy_3.gif", [-11.5,7,-25,14], None, None, None],
    ["sprites/F02_Heavy_3.gif", [-11.5,7,-25,14], None, None, None]
]

```



```

["sprites/F02_Idle_1.gif", [-11.5,7,-25,14], None, None, None],
["sprites/F02_Idle_0.gif", [-11.5,7,-25,14], None, None, None]
],
[ #HeavyLw
["sprites/F02_HeavyLw_1.gif", [-11.5,11.5,-25,-10], None, None, None],
["sprites/F02_HeavyLw_2.gif", [-11.5,11.5,-25,-10], None, None, None],
["sprites/F02_HeavyLw_3.gif", [-11.5,8,-25,-10], [3, 13, -2, 17], [0, 4, 6, 0, 70, 3],
None],
["sprites/F02_HeavyLw_3.gif", [-11.5,8,-25,-10], [3, 13, -2, 17], [0, 4, 6, 0, 70, 3],
None],
["sprites/F02_HeavyLw_3.gif", [-11.5,8,-25,-10], [3, 13, -2, 17], [0, 4, 6, 0, 70, 3],
None],
["sprites/F02_HeavyLw_2.gif", [-11.5,11.5,-25,3], None, None, None],
["sprites/F02_HeavyLw_2.gif", [-11.5,11.5,-25,3], None, None, None],
["sprites/F02_HeavyLw_1.gif", [-11.5,11.5,-25,3], None, None, None],
["sprites/F02_HeavyLw_0.gif", [-11.5,11.5,-25,3], None, None, None]
],
[ #HeavyAir
["sprites/F02_HeavyAir_0.gif", [-11.5,11.5,-20,12], None, None, None],
["sprites/F02_HeavyAir_1.gif", [-11.5,11.5,-20,12], [7, 16, -10, 5], [0, 3, 4, 0, 0, 3],
[7, 19, -10, 5]],
["sprites/F02_HeavyAir_1.gif", [-11.5,11.5,-20,12], [7, 16, -10, 5], [0, 3, 4, 0, 0, 3],
[7, 19, -10, 5]],
["sprites/F02_HeavyAir_1.gif", [-11.5,11.5,-20,12], None, None, None],
["sprites/F02_HeavyAir_3.gif", [-11.5,11.5,-20,12], [7, 16, -10, 17], [0, 2, 8, 0, 60,
3], [7, 22, -10, 18]],
["sprites/F02_HeavyAir_3.gif", [-11.5,11.5,-20,12], [7, 16, -10, 17], [0, 2, 8, 0, 60,
3], [7, 22, -10, 18]],
["sprites/F02_HeavyAir_3.gif", [-11.5,11.5,-20,12], [7, 16, -10, 17], [0, 2, 8, 0, 60,
3], [7, 22, -10, 18]],
["sprites/F02_HeavyAir_3.gif", [-11.5,11.5,-20,12], None, None, None],
["sprites/F02_HeavyAir_3.gif", [-11.5,11.5,-20,12], None, None, None],
["sprites/F02_HeavyAir_1.gif", [-11.5,11.5,-20,12], None, None, None],
["sprites/F02_HeavyAir_1.gif", [-11.5,11.5,-20,12], None, None, None],
["sprites/F02_HeavyAir_1.gif", [-11.5,11.5,-20,12], None, None, None],
["sprites/F02_Air_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #Hitstun
["sprites/F02_Hitstun_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Hitstun_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Hitstun_1.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #Guard
["sprites/F02_Guard_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Guard_0.gif", [-11.5,11.5,-25,14], None, None, None],
["sprites/F02_Guard_1.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #CrouchGuard

```

```
[{"sprites/F02_CrouchGuard_0.gif", [-11.5, 11.5, -25, 2], None, None, None}, {"sprites/F02_CrouchGuard_0.gif", [-11.5, 11.5, -25, 2], None, None, None}, {"sprites/F02_CrouchGuard_1.gif", [-11.5, 11.5, -25, 2], None, None, None}], [{"#GuardAir": {"sprites/F02_GuardAir_0.gif", [-11.5, 11.5, -25, 2], None, None, None}, {"sprites/F02_GuardAir_0.gif", [-11.5, 11.5, -25, 2], None, None, None}, {"sprites/F02_GuardAir_0.gif", [-11.5, 11.5, -25, 2], None, None, None}}, [{"#SpecialLw": {"sprites/F02_SpecialLw_0.gif", [-11.5, 11.5, -25, 4], None, None, None}, {"sprites/F02_SpecialLw_1.gif", [-11.5, 11.5, -25, 4], None, None, None}, {"sprites/F02_SpecialLw_2.gif", None, [7, 16, -10, 5], [0, 4, 10, 45, 100, 3], None}, {"sprites/F02_SpecialLw_3.gif", None, [-12, 12, -8, 25], [0, 4, 10, 45, 100, 3], None}, {"sprites/F02_SpecialLw_3.gif", None, [-12, 12, -8, 25], [0, 4, 10, 45, 100, 3], None}, {"sprites/F02_SpecialLw_3.gif", [-11.5, 11.5, -22, 25], None, None, None}, {"sprites/F02_SpecialLw_4.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_SpecialLw_5.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_SpecialLw_6.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_SpecialLw_6.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_Air_0.gif", [-11.5, 11.5, -25, 14], None, None, None}], [{"#SpecialN": {"sprites/F02_SpecialLw_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_SpecialLw_0.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_SpecialN_1.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_SpecialN_1.gif", [-11.5, 11.5, -25, 14], None, None, None}, {"sprites/F02_SpecialN_2.gif", [-11.5, 11.5, -25, 14], [1, 16, 10, 33], [0, 3.5, 10, 56, 56, 5], None}, {"sprites/F02_SpecialN_3.gif", [-11.5, 11.5, -25, 14], [14, 24, 37, 47], [0, 3.5, 10, 56, 56, 5], None}, {"sprites/F02_SpecialN_4.gif", [-11.5, 11.5, -25, 14], [29, 39, 55, 65], [0, 3.5, 10, 56, 56, 5], None}, {"sprites/F02_SpecialN_5.gif", [-11.5, 11.5, -25, 14], [35, 45, 61, 71], [0, 3.5, 10, 56, 56, 5], None}, {"sprites/F02_SpecialN_6.gif", [-11.5, 11.5, -25, 14], [45, 55, 61, 71], [0, 3.5, 10, 56, 56, 5], None}]
```



```

    ["sprites/F02_SpecialN_7.gif", [-11.5,11.5,-25,14], [63, 73, 65, 75], [0, 3.5, 10, 56,
56, 5], None],
    ["sprites/F02_SpecialN_8.gif", [-11.5,11.5,-25,14], [79, 89, 55, 63], [0, 3.5, 10, 56,
56, 5], None],
    ["sprites/F02_SpecialN_9.gif", [-11.5,11.5,-25,14], [94, 104, 33, 43], [0, 3.5, 10, 56,
56, 5], None],
    ["sprites/F02_SpecialN_10.gif", [-11.5,11.5,-25,14], [105, 115, 12, 22], [0, 3.5, 10,
56, 56, 5], None],
    ["sprites/F02_SpecialN_11.gif", [-11.5,11.5,-25,14], [117, 127, -10, 0], [0, 3.5, 10,
56, 56, 5], None],
    ["sprites/F02_SpecialN_12.gif", [-11.5,11.5,-25,14], [120, 130, -22, -12], [0, 3.5, 10,
56, 56, 5], None]
],
[ #SpecialEmpty
    ["sprites/F02_SpecialLw_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialLw_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialN_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialN_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialN_2_Empty.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialN_2_Empty.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialN_2_Empty.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialN_8_Empty.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialN_8_Empty.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialN_8_Empty.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialN_8_Empty.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialN_8_Empty.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialLw_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialLw_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialLw_1.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #SpecialS
    ["sprites/F02_SpecialLw_1.gif", [-11.5,11.5,-25,-10], None, None, None],
    ["sprites/F02_SpecialLw_1.gif", [-11.5,11.5,-25,-10], None, None, None],
    ["sprites/F02_SpecialLw_0.gif", [-11.5,11.5,-25,-10], None, None, None],
    ["sprites/F02_SpecialS_0.gif", [-11.5,11.5,-25,14], [-5, 28, -10, 3], [1, 4, 10, 45, 50,
4], [-5, 30, -10, 3]],
    ["sprites/F02_SpecialS_0.gif", [-11.5,11.5,-25,14], [-5, 28, -10, 3], [1, 4, 10, 45, 50,
4], [-5, 30, -10, 3]],
    ["sprites/F02_SpecialS_0.gif", [-11.5,11.5,-25,14], [-5, 28, -10, 3], [1, 4, 10, 45, 50,
4], [-5, 30, -10, 3]],
    ["sprites/F02_SpecialS_0.gif", [-11.5,11.5,-25,14], [-5, 28, -10, 3], [1, 4, 10, 45, 50,
4], [-5, 30, -10, 3]],
    ["sprites/F02_SpecialS_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialS_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialS_3.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialS_2.gif", [-11.5,11.5,-25,14], [-5, 28, -10, 7], [1, 2, 4, 15, 40,
2], [-5, 29, -13, 10]],
    ["sprites/F02_SpecialS_2.gif", [-11.5,11.5,-25,14], [-5, 28, -10, 7], [1, 2, 4, 15, 40,

```

```

2], [-5, 29, -13, 10]],
    ["sprites/F02_SpecialS_2.gif", [-11.5,11.5,-25,14], [-5, 28, -10, 7], [1, 2, 4, 15, 40,
2], [-5, 29, -13, 10]],
    ["sprites/F02_SpecialS_3.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_SpecialS_3.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Crouch_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Crouch_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #ThrowWhiff
    ["sprites/F02_ThrowWhiff_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowWhiff_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], [0, 20, -15, 0], [2, 0, 13, -45,
45, 0], [0, 24, -25, 0]],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], [0, 20, -15, 0], [2, 0, 13, -45,
45, 0], [0, 24, -25, 0]],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, [0, 24, -25, 0]],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowWhiff_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
],
[ #ThrowF
    ["sprites/F02_ThrowF_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowF_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowF_0.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowF_1.gif", [-11.5,11.5,-25,14], [-250, 250, -100, 100], [2, 2, 10,
40, 45, 0], None],
    ["sprites/F02_ThrowF_1.gif", [-11.5,11.5,-25,14], [-250, 250, -100, 100], [2, 2, 10,
40, 45, 0], None],
    ["sprites/F02_ThrowF_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowF_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowF_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowF_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_ThrowF_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_2.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_1.gif", [-11.5,11.5,-25,14], None, None, None],
    ["sprites/F02_Idle_0.gif", [-11.5,11.5,-25,14], None, None, None]
]
]
]

```

run_game.py

```
import turtle
```

```
import os
import time
import keyboard
from tkinter import PhotoImage
from turtle import Shape
import image_reverser
import random
import math
import sys
import menu
import cpu_data
import animations
import stats
from util import get_anim_ID
from util import get_controls_from_txt
```

```
image_reverser.reverse()
```

```
#CONSTANTS
```

```
SCREEN_WIDTH = 800
SCREEN_HEIGHT = 450
BOX_SIZE = [325, 360]
FRAME_LENGTH = 60.0
```

```
#Character constants
```

```
SCALE = 5
```

```
DASH_WINDOW = 4
```

```
JUMP_INITIAL = 50.0
GRAVITY = 8.0
MAX_FALL = 40.0
AIR_MOVE = 15.0
```

```
SPECIAL_LW_INIT_Y = 45.0
SPECIAL_LW_INIT_X = 10.0
```

```
SPECIAL_S_X = 20.0
```

```
BUFFER_FRAMES = 1
```

```
#Collisions!
```

```
PUSHBOXES = [11.5,-25,14]
PUSHING_FORCE = 5
char_pos = [[0,0], [0,0]]
```

```
#hitbox hurtbox interactions
```

```

HP_RESTORE_DURATION = 45
BLOCK_TIME = 11
DEFAULT_Y_KNOCKBACK = 20 + GRAVITY
KB_DECAY_MULTIPLIER = 0.8
force_hit_now = False
PUSHBACK = 15
BLOCK_KB_MUL = 0.5
AIR_BLOCK_HITSTUN_MUL = 1.5

hurtbox = [
    [0,0,0,0],
    [0,0,0,0]
]
hurtbox_2 = [
    [0,0,0,0],
    [0,0,0,0]
]

hitbox = [
    [0,0,0,0],
    [0,0,0,0]
]

hitbox_properties = [
    [0,0,0,0,0,0],
    [0,0,0,0,0,0]
]

players_x = [0,0]
players_y = [0,0]

#Format for Frames= ["sprites/frame.gif", [hurtbox x1, hurtbox x2, hurtbox y1, hurtbox
y2]
# , [hitbox x1, hitbox x2, hitbox y1, hitbox y2]
# , [hit_height, damage, hitstun, kb_speed_x, kb_speed_y, blockstun] - hit height is (-1
Low, 1 High, 0 Mid, 2 unblockable)
# , [hurtbox x1, hurtbox x2, hurtbox y1, hurtbox y2] Second set of hurtboxess, used for
hurtbox extensions on moves

#Lists of anims in a certain category
ACTIONABLE_LIST = [
    get_anim_ID("Idle"), get_anim_ID("WalkF"), get_anim_ID("WalkB"),
    get_anim_ID("Crouch"), get_anim_ID("CrouchWait"), get_anim_ID("CrouchRv"),

```

```

    get_anim_ID("Air")
]

SPECIAL_CANCEL_LIST = [
    get_anim_ID("Attack"), get_anim_ID("AttackLw"), get_anim_ID("AttackAir"),
    get_anim_ID("Heavy"), get_anim_ID("HeavyLw"), get_anim_ID("HeavyAir")
]

BLOCKING_LIST = [
    get_anim_ID("Guard"), get_anim_ID("CrouchGuard"), get_anim_ID("GuardAir")
]

#Debug options
FRAME_STEP = False
SPACE_TO_PAUSE = True

#Hitbox/Hurtbox Drawing

def rectangle(turtle, points):
    points = [
        [points[0], points[2]],
        [points[1], points[2]],
        [points[1], points[3]],
        [points[0], points[3]],
        [points[0], points[2]]
    ]
    turtle.clear()
    turtle.penup()
    turtle.speed(0)
    turtle.goto(points[0][0], points[0][1])
    turtle.pendown()
    for i in points:
        turtle.goto(i[0], i[1])
    turtle.penup()

def draw_boxes(screen, p1hurt, p1hurt2, p1hit, p2hurt, p2hurt2, p2hit): #Used for
hurtbox/hitbox display in training mode
    screen.tracer(0)
    rectangle(p1hurt, hurtbox[0])
    rectangle(p2hurt, hurtbox[1])
    rectangle(p1hurt2, hurtbox_2[0])
    rectangle(p2hurt2, hurtbox_2[1])
    rectangle(p1hit, hitbox[0])
    rectangle(p2hit, hitbox[1])

```

```

screen.update()
screen.tracer(10)

CHAR_ANIMS = [animations.RYU, animations.KEN, animations.SEAN]
#print(screen.getshapes())

class player:
    def __init__(self,
        playerNum: int,
        x: int, y: int,
        #sprite, frame, animList,
        controls: list, #Controls expressed as [left, right, up, down, light, heavy, special]
        Left: bool,
        Training_settings: list,
        character: int,
        read_log = False,
        is_cpu = False
    ):
        self.playerNum = playerNum
        self.x = x
        self.y = y
        self.char_id = character
        self.accessOtherPlayer = []
        self.is_left = Left
        self.isCrouch = False
        self.moveXThisFrame = 0.0
        self.moveYThisFrame = 0.0

        #Sprite Init
        self.frame = 0
        self.animList = CHAR_ANIMS[self.char_id]
        self.animListID = 0
        self.sprite = f"sprites/F0{self.char_id}_Idle_0.gif"
        self.training_settings = Training_settings
        self.start_blocking_now = 0

        #Jump Attributes
        self.isJump = False
        self.lastmoveY = 0.0
        self.lastmoveX = 0.0
        self.lastmoveX = 0.0
        self.jumpDir = 0.0 #-1.0 is back, 0.0 is neutral, 1.0 is forwards

        #Attack Stuff
        self.attackPressed = False
        self.specialPressed = False
        self.heavyPressed = False
        self.attackBuffer = 0

```

```

self.jumpBuffer = 0
self.specialBuffer = 0
self.heavyBuffer = 0
self.doPushback = False
self.buttoncheck = [False,False,False,False,False,False,False]

#Hitstun
self.isHitstun = False
self.isBlockstun = False
self.isLeaveBlockstun = False
self.isBlocking = False
self.hitstunFrames = 0
self.maxHitstun = 0
self.hp = stats.HEALTH[self.char_id]
self.decreaseHp = 0

self.leftPressed = False
self.rightPressed = False
self.backdashTimer = 0
self.forwarddashTimer = 0
self.disableJostle = False

#For Hadouken
self.hadouTimer = 0

#Turtle Setup
self.turtle = turtle.Turtle()
self.turtle.speed(0)
self.turtle.penup()
self.turtle.goto(x, y)
self.turtle.shape(self.sprite)
self.animate()

self.isCpu = is_cpu
self.cpuQueue = []

self.controls = controls
self.readLog = read_log
self.logFrame = 0
self.log = []
if self.readLog: #Debug logging stuff, to check if you can take full control of the
player using code rather than buttons
    try:
        f = open("test_log.txt")
        text = f.readlines()
        f.close()
        for i in text:

```

```

        line = i.replace(" ", "")
        line = line.replace("\n", "")
        line = line.replace("[", "")
        line = line.replace("]", "")
        line = line.split(",")
        line = [ele.lower().capitalize() == "True" for ele in line] #Converts strings
into real booleans
        self.log.append(line)
    except Exception:
        print("failed to read log")
        self.log = []
        self.readLog = False

self.logLength = len(self.log)

def update_other_list(self, new): #Used to access opponent's attributes and methods
    self.accessOtherPlayer = new

def update_hit_hurt(self): #Updates hitbox and hurtbox placement
    global JUMP_INITIAL
    anim = self.animList[self.animListID]
    direction_mul = 1.0
    if self.is_left:
        direction_mul = -1.0

    x = self.turtle.xcor()
    y = self.turtle.ycor()

    hurtbox[self.playerNum - 1] = [0,0,0,0]
    hurtbox_pos = [x,x,y,y]
    if anim[self.frame][1] != None:
        hurtbox_pos[0] += anim[self.frame][1][0]*SCALE
        hurtbox_pos[1] += anim[self.frame][1][1]*SCALE
        hurtbox_pos[2] += anim[self.frame][1][2]*SCALE
        hurtbox_pos[3] += anim[self.frame][1][3]*SCALE
        hurtbox[self.playerNum - 1] = hurtbox_pos
    else:
        hurtbox[self.playerNum - 1] = [9990, 10000, 9990, 10000]

    hurtbox_2[self.playerNum - 1] = [0,0,0,0]
    hurtbox2_pos = [x,x,y,y]
    if anim[self.frame][4] != None:
        if direction_mul > 0:
            hurtbox2_pos[0] += anim[self.frame][4][0]*SCALE* direction_mul
            hurtbox2_pos[1] += anim[self.frame][4][1]*SCALE* direction_mul
        else:

```



```

        hurtbox2_pos[1] += anim[self.frame][4][0]*SCALE* direction_mul
        hurtbox2_pos[0] += anim[self.frame][4][1]*SCALE* direction_mul
        hurtbox2_pos[2] += anim[self.frame][4][2]*SCALE
        hurtbox2_pos[3] += anim[self.frame][4][3]*SCALE
        hurtbox_2[self.playerNum - 1] = hurtbox2_pos
    else:
        hurtbox_2[self.playerNum - 1] = [9990, 10000, 9990, 10000]

    hitbox[self.playerNum - 1] = [0,0,0,0]
    hitbox_pos = [x,x,y,y]
    if anim[self.frame][2] != None and not self.hasHit:
        if direction_mul > 0:
            hitbox_pos[0] += (anim[self.frame][2][0] * direction_mul)*SCALE
            hitbox_pos[1] += (anim[self.frame][2][1] * direction_mul)*SCALE
        else:
            hitbox_pos[1] += (anim[self.frame][2][0] * direction_mul)*SCALE
            hitbox_pos[0] += (anim[self.frame][2][1] * direction_mul)*SCALE
        hitbox_pos[2] += anim[self.frame][2][2]*SCALE
        hitbox_pos[3] += anim[self.frame][2][3]*SCALE
        hitbox[self.playerNum - 1] = hitbox_pos
        #print(f"Hitboxes out! Hurtbox: {hurtbox_pos} Hitbox: {hitbox_pos}")
        #print(f"pos [{x}, {y}]")
    else:
        hitbox[self.playerNum - 1] = [9990, 10000, 9990, 10000]
        self.hasHit = False

    hitbox_properties[self.playerNum - 1] = anim[self.frame][3]

def animate(self):
    global JUMP_INITIAL
    anim = self.animList[self.animListID]
    anim_length = len(anim)
    sprite = anim[self.frame][0]
    if self.is_left == True:
        sprite = sprite.replace("sprites", "reverse_sprites") #If facing left, replace them
with their reversed variant
    self.sprite = sprite
    #print(self.sprite)
    self.turtle.shape(self.sprite)
    #print(self.turtle.shape())

    #Report Hitboxes and Hurtboxes
    self.update_hit_hurt()

    if self.frame+1 >= anim_length: #Returns to default anim for state
        self.frame = 0

```

```

        self.set_new_anim_by_ID()
    else:
        self.frame += 1

    def overlap(self, rect1, rect2): #Checks if 2 rectangles in space overlap! If they do
reports as true. Used to check if hitboxes overlap hurtboxes
        r1x1 = rect1[0]
        r1x2 = rect1[1]
        r1y1 = rect1[2]
        r1y2 = rect1[3]
        r2x1 = rect2[0]
        r2x2 = rect2[1]
        r2y1 = rect2[2]
        r2y2 = rect2[3]
        if r1x1 > r2x2 or r2x1 > r1x2:
            return False
        if r1y1 > r2y2 or r2y1 > r1y2:
            return False

        #print("Overlaps!")
        return True

    def did_i_block_that(self): #The blocking checker
        enemy_properties = hitbox_properties[-(self.playerNum)]
        high_low = enemy_properties[0]
        if high_low == 2: #Unblockable! Used for throws
            return False

        if self.isHitstun and not self.isBlockstun: #You can't block while you're in hitstun!
            return False

        if (self.training_settings[3] == 1 or self.start_blocking_now > 0) and (self.animListID
in ACTIONABLE_LIST or self.animListID in BLOCKING_LIST) and self.playerNum == 2:
            if self.training_settings[3] == 2: #Training mode block all or block after first hit
type behaviour
                self.start_blocking_now = BLOCK_TIME

        self.isBlocking = True
        if high_low == -1:
            self.isCrouch = True
            return True
        if self.isCpu: #CPU behaviour
            if random.randint(0, 100) < cpu_data.BLOCK_CHANCE and (self.animListID in
ACTIONABLE_LIST or self.animListID in BLOCKING_LIST):
                self.isBlocking = True
                if high_low == -1:
                    self.isCrouch = True
            return True

```

```

if self.isBlocking: #Regular behaviour
    if high_low == 0:
        return True
    if high_low == -1 and self.isCrouch:
        return True
    if high_low == 1 and not self.isCrouch:
        return True
    return False

def start_hitstun(self):
    #print("start")
    enemy_properties = hitbox_properties[-(self.playerNum)]
    self.hitstunFrames = enemy_properties[2]
    #print(self.hitstunFrames)
    self.moveXThisFrame = enemy_properties[3]
    if self.is_left == True:
        self.moveXThisFrame *= -1

    self.moveYThisFrame = enemy_properties[4]
    if enemy_properties[4] == 0 and self.isJump:
        self.moveYThisFrame = DEFAULT_Y_KNOCKBACK

    if self.did_i_block_that():
        self.isBlockstun = True
        self.moveXThisFrame *= BLOCK_KB_MUL
        self.moveYThisFrame = 0
        self.hitstunFrames = int(math.floor(enemy_properties[5]))
        if self.isJump:
            self.hitstunFrames =
int(math.floor(enemy_properties[5]*AIR_BLOCK_HITSTUN_MUL))

    self.isHitstun = True

    self.lastmoveX = self.moveXThisFrame * -1
    self.lastmoveY = self.moveYThisFrame
    if not self.did_i_block_that():
        self.jumpDir = 0
        if self.training_settings[3] == 2:
            self.start_blocking_now = BLOCK_TIME
        if enemy_properties[4] > 0 and not self.isJump:
            self.isJump = True
            self.lastmoveY = self.moveYThisFrame + GRAVITY

    if self.maxHitstun != self.hitstunFrames:
        self.maxHitstun = self.hitstunFrames

```

```

        #print("Hurting")
        self.decreaseHp = enemy_properties[1]
        self.set_new_anim_by_ID()
    else:
        if self.isCrouch:
            self.set_new_anim_by_ID()
        else:
            self.set_new_anim_by_ID()

def check_is_hurt(self):
    global ACTIONABLE_LIST
    #print(f"[{self.x}, {self.y}]")
    enemy_hitbox = hitbox[-(self.playerNum)]
    my_hurtbox = hurtbox[self.playerNum - 1]
    my_hurtbox2 = hurtbox_2[self.playerNum - 1]

    enemy_properties = hitbox_properties[-(self.playerNum)]
    #if enemy_hitbox[0] < 9000:
        #print(f"hurtbox [{my_hurtbox[0]} - {my_hurtbox[1]}], hitbox [{enemy_hitbox[0]} - {enemy_hitbox[1]}]")
        if not enemy_properties == None and enemy_hitbox[0] < 9000 and (self.overlap(enemy_hitbox, my_hurtbox) or self.overlap(enemy_hitbox, my_hurtbox2)):
            #print("Hitstun!")
            self.start_hitstun()

def hitstun_movement(self):
    if self.isHitstun: #Governs where they be going during hitstun
        self.moveXThisFrame = self.lastmoveX * KB_DECAY_MULTIPLIER
        #Knockback decays at a set rate so that it naturally stops
        self.moveYThisFrame = self.lastmoveY * KB_DECAY_MULTIPLIER
        self.lastmoveY = self.moveYThisFrame
        self.lastmoveX = self.moveXThisFrame

def check_is_hitting(self):
    global ACTIONABLE_LIST
    global force_hit_now
    #print(f"[{self.x}, {self.y}]")
    enemy_hurtbox = hurtbox[-(self.playerNum)]
    enemy_hurtbox2 = hurtbox_2[-(self.playerNum)]
    my_hitbox = hitbox[self.playerNum - 1]

    if my_hitbox[0] < 9000 and (self.overlap(enemy_hurtbox, my_hitbox) or self.overlap(enemy_hurtbox2, my_hitbox)):
        if not self.hasHit:
            #print("Yeah")
            force_hit_now = True

```

```

        self.hasHit = True

        self.doPushback = True

def set_new_anim_by_ID(self, id=-2, frame=0):
    if id == -1:
        return

    self.doPushback = False
    self.hasHit = False

    if id == -2:
        self.frame = frame
        if self.isHitstun:
            if self.isBlockstun:
                if self.isCrouch:
                    self.set_new_anim_by_ID(get_anim_ID("CrouchGuard"))
                elif self.isJump:
                    self.set_new_anim_by_ID(get_anim_ID("GuardAir"))
                else:
                    self.set_new_anim_by_ID(get_anim_ID("Guard"))
            else:
                self.animListID = get_anim_ID("Hitstun")
            return
        if self.isJump:
            if self.animListID == get_anim_ID("JumpSquat"):
                self.moveYThisFrame = JUMP_INITIAL
            self.animListID = get_anim_ID("Air")
            return
        elif self.isCrouch:
            self.animListID = get_anim_ID("CrouchWait")
            return
        else:
            self.animListID = get_anim_ID("Idle")
            return

    global force_hit_now

    force_hit_now = False
    self.frame = frame
    self.animListID = id

def right(self):
    global SPECIAL_CANCEL_LIST

```

```

        #print(f"[{self.x}, {self.y}])
        if self.isJump == False and not self.isHitstun:
            if self.animListID in [get_anim_ID("Idle"), get_anim_ID("WalkF"),
get_anim_ID("WalkB")]:
                self.moveXThisFrame = stats.WALK_SPEED[self.char_id]
            if self.is_left == False: #If facing left
                if self.animListID in [get_anim_ID("Idle"), get_anim_ID("WalkB")]:
                    self.set_new_anim_by_ID(get_anim_ID("WalkF"))
                if self.animListID == get_anim_ID("WalkF") and self.frame >=
len(self.animList[self.animListID])-1:
                    self.set_new_anim_by_ID(get_anim_ID("WalkF"))

            if self.rightPressed == False:
                if self.forwarddashTimer == 0:
                    self.forwarddashTimer = DASH_WINDOW
                else:
                    if self.animListID in ACTIONABLE_LIST:
                        self.forwarddashTimer = 0
                        self.set_new_anim_by_ID(get_anim_ID("ForwardDash"))
                        self.moveXThisFrame = stats.FDASH_SPEED[self.char_id]
                    else:
                        if self.animListID in [get_anim_ID("Idle"), get_anim_ID("WalkF")]:
                            self.set_new_anim_by_ID(get_anim_ID("WalkB"))
                        if self.animListID == get_anim_ID("WalkB") and self.frame >=
len(self.animList[self.animListID])-1:
                            self.set_new_anim_by_ID(get_anim_ID("WalkB"))

            if self.rightPressed == False:
                if self.backdashTimer == 0:
                    self.backdashTimer = DASH_WINDOW
                else:
                    if self.animListID in ACTIONABLE_LIST:
                        self.backdashTimer = 0
                        self.set_new_anim_by_ID(get_anim_ID("BackDash"))
                        self.moveXThisFrame = stats.BDASH_SPEED[self.char_id]

def left(self):
    if self.isJump == False and not self.isHitstun:
        if self.animListID in [get_anim_ID("Idle"), get_anim_ID("WalkF"),
get_anim_ID("WalkB")]:
            self.moveXThisFrame = -stats.WALK_SPEED[self.char_id]
        if self.is_left == True: #If facing left
            if self.animListID in [get_anim_ID("Idle"), get_anim_ID("WalkB")]:
                self.set_new_anim_by_ID(get_anim_ID("WalkF"))
            if self.animListID == get_anim_ID("WalkF") and self.frame >=
len(self.animList[self.animListID])-1:
                self.set_new_anim_by_ID(get_anim_ID("WalkF"))

```

```

        if self.leftPressed == False:
            if self.forwarddashTimer == 0:
                self.forwarddashTimer = DASH_WINDOW
            else:
                if self.animListID in ACTIONABLE_LIST:
                    self.forwarddashTimer = 0
                    self.set_new_anim_by_ID(get_anim_ID("ForwardDash"))
                    self.moveXThisFrame = -stats.FDASH_SPEED[self.char_id]
                else:
                    if self.animListID in [get_anim_ID("Idle"), get_anim_ID("WalkF")]:
                        self.set_new_anim_by_ID(get_anim_ID("WalkB"))
                    if self.animListID == get_anim_ID("WalkB") and self.frame >=
len(self.animList[self.animListID])-1:
                        self.set_new_anim_by_ID(get_anim_ID("WalkB"))

        if self.leftPressed == False:
            if self.backdashTimer == 0:
                self.backdashTimer = DASH_WINDOW
            else:
                if self.animListID in ACTIONABLE_LIST:
                    self.backdashTimer = 0
                    self.set_new_anim_by_ID(get_anim_ID("BackDash"))
                    self.moveXThisFrame = -stats.BDASH_SPEED[self.char_id]

def down(self):
    #print(f"[{self.x}, {self.y}]")
    if self.isJump == False:
        if self.isCrouch == False and self.animListID in [get_anim_ID("Idle"),
get_anim_ID("WalkF"), get_anim_ID("WalkB")]:
            self.isCrouch = True
            self.set_new_anim_by_ID(get_anim_ID("Crouch"))

def uncrouch(self):
    #print(f"[{self.x}, {self.y}]")
    if self.isCrouch == True and self.animListID in [get_anim_ID("CrouchWait")]:
        self.isCrouch = False
        self.set_new_anim_by_ID(get_anim_ID("CrouchRv"))

def up(self):
    #print(f"[{self.x}, {self.y}]")
    if self.isJump == False:
        if self.animListID in [get_anim_ID("Idle"), get_anim_ID("WalkF"),
get_anim_ID("WalkB"), get_anim_ID("CrouchRv")]:
            self.isJump = True
            self.set_new_anim_by_ID(get_anim_ID("JumpSquat")) #Jump time

```

```

else:
    self.jumpBuffer = BUFFER_FRAMES

def air(self):
    global JUMP_INITIAL
    global GRAVITY
    global MAX_FALL
    global AIR_MOVE
    global force_hit_now
    #print(f"[{self.x}, {self.y}]")
    if self.isJump == True:
        if self.y <= 0.0 and self.lastmoveY < 0.0:
            self.isJump = False
            if self.y < 0.0:
                self.moveYThisFrame = -self.y
            self.set_new_anim_by_ID()
        else:
            if self.animListID == get_anim_ID("JumpSquat"):
                if self.buttoncheck[0] and not self.buttoncheck[1]:
                    self.jumpDir = 1.0 #Jump right
                elif self.buttoncheck[1] and not self.buttoncheck[0]:
                    self.jumpDir = -1.0 #Jump left
                else:
                    self.jumpDir = 0.0 #Jump straight

                #makes buffering rising aerals a breeze!
                if self.buttoncheck[4] and not self.buttoncheck[5]:
                    self.attackBuffer = 1
                if self.buttoncheck[5] and not self.buttoncheck[4]:
                    self.heavyBuffer = 1
            else:
                #print("we running")
                self.moveYThisFrame = self.lastmoveY - GRAVITY #Makes them fall
                if not self.isHitstun:
                    self.moveXThisFrame = self.jumpDir * AIR_MOVE * -1.0

def attack(self):
    global ACTIONABLE_LIST
    if not self.animListID in ACTIONABLE_LIST: #Attack buffer
        self.attackBuffer = BUFFER_FRAMES
    else:
        self.attackBuffer = 0

if not self.isJump:
    if self.animListID in ACTIONABLE_LIST:
        self.doPushback = False
        if self.buttoncheck[3] and not self.buttoncheck[2]:

```



```

        self.set_new_anim_by_ID(get_anim_ID("AttackLw"))
    else:
        self.set_new_anim_by_ID(get_anim_ID("Attack"))
    else:
        if self.animListID in ACTIONABLE_LIST:
            self.set_new_anim_by_ID(get_anim_ID("AttackAir"))

def attackLw(self):
    global ACTIONABLE_LIST
    if not self.animListID == get_anim_ID("AttackLw"):
        return

    if self.char_id == 2: #So sean can slide
        direction_mul = 1.0
        if self.is_left:
            direction_mul = -1.0

    if self.frame < 9:
        self.moveXThisFrame = 18 * direction_mul

def heavyN(self):
    global ACTIONABLE_LIST
    if not self.animListID == get_anim_ID("Heavy"):
        return

    if self.char_id == 0: #So ryu can kick
        direction_mul = 1.0
        if self.is_left:
            direction_mul = -1.0

    if self.frame < 6:
        self.moveXThisFrame = 10 * direction_mul

    if self.frame == 6:
        self.moveXThisFrame = 25 * direction_mul

def heavy(self):
    global ACTIONABLE_LIST
    if not self.animListID in ACTIONABLE_LIST:
        self.heavyBuffer = BUFFER_FRAMES
    else:
        self.heavyBuffer = 0

    if not self.isJump:
        if self.animListID in ACTIONABLE_LIST:
            self.doPushback = False
            if self.buttoncheck[3] and not self.buttoncheck[2]:

```

```

        self.set_new_anim_by_ID(get_anim_ID("HeavyLw"))
    else:
        self.set_new_anim_by_ID(get_anim_ID("Heavy"))
    else:
        if self.animListID in ACTIONABLE_LIST:
            self.set_new_anim_by_ID(get_anim_ID("HeavyAir"))

def special(self):
    global ACTIONABLE_LIST
    global SPECIAL_CANCEL_LIST
    if not self.animListID in ACTIONABLE_LIST and not (self.animListID in
SPECIAL_CANCEL_LIST and self.doPushback):
        self.specialBuffer = BUFFER_FRAMES
    else:
        self.specialBuffer = 0
    if self.isJump:
        if self.buttoncheck[1] or self.buttoncheck[0]: #Lets you specifically side special
in the air!
            self.set_new_anim_by_ID(get_anim_ID("SpecialS"))
        return

    if self.buttoncheck[3] and not self.buttoncheck[2]:
        self.set_new_anim_by_ID(get_anim_ID("SpecialLw"))
    elif self.buttoncheck[1] or self.buttoncheck[0]:
        self.set_new_anim_by_ID(get_anim_ID("SpecialS"))
    else:
        self.set_new_anim_by_ID(get_anim_ID("SpecialN"))

def throw(self):
    global ACTIONABLE_LIST
    global SPECIAL_CANCEL_LIST
    if self.animListID in ACTIONABLE_LIST and not self.isJump and not self.isCrouch:
        self.set_new_anim_by_ID(get_anim_ID("ThrowWhiff"))

def throw_logic(self):
    global ACTIONABLE_LIST
    global SPECIAL_CANCEL_LIST
    if self.animListID != get_anim_ID("ThrowWhiff") and self.animListID !=
get_anim_ID("ThrowF"):
        return

    if self.doPushback and not self.hasHit and self.animListID ==
get_anim_ID("ThrowWhiff"):
        self.set_new_anim_by_ID(get_anim_ID("ThrowF")) # Goes into throw animation
if the Grabbox hits

```

```

def specialLw(self):
    if self.animListID != get_anim_ID("SpecialLw"):
        return

    direction_mul = 1.0
    if self.is_left:
        direction_mul = -1.0

    if self.char_id == 0: #all these are character specific behaviours
        if self.frame == 3:
            self.isJump = True
            self.jumpDir = 0
            self.moveYThisFrame = SPECIAL_LW_INIT_Y
            self.moveXThisFrame = SPECIAL_LW_INIT_X*direction_mul
        elif self.frame > 3:
            self.moveYThisFrame = self.lastmoveY - GRAVITY
            self.moveXThisFrame = self.lastmoveX * 0.8
    if self.char_id == 1:
        if self.frame == 4:
            self.isJump = True
            self.jumpDir = 0
            self.moveYThisFrame = SPECIAL_LW_INIT_Y
            self.moveXThisFrame = SPECIAL_LW_INIT_X*direction_mul
        elif self.frame > 3:
            self.moveYThisFrame = self.lastmoveY - GRAVITY
            self.moveXThisFrame = self.lastmoveX * 0.8
    if self.char_id == 2:
        if self.frame == 3:
            self.isJump = True
            self.jumpDir = 0
            self.moveYThisFrame = SPECIAL_LW_INIT_Y*0.9
            self.moveXThisFrame = SPECIAL_LW_INIT_X*direction_mul
        elif self.frame > 3:
            self.moveYThisFrame = self.lastmoveY - GRAVITY
            self.moveXThisFrame = self.lastmoveX * 0.7

def specialS(self):
    if self.animListID != get_anim_ID("SpecialS"):
        if self.char_id == 1:
            self.disableJostle = False
        return

    direction_mul = 1.0
    if self.is_left:
        direction_mul = -1.0

    if self.char_id == 0: #all these are character specific behaviours
        if self.isJump:

```

```

        self.moveXThisFrame = SPECIAL_S_X*direction_mul*1.2
    else:
        self.moveXThisFrame = SPECIAL_S_X*direction_mul
if self.char_id == 1:
    if self.isJump:
        self.moveXThisFrame = SPECIAL_S_X*direction_mul*1.2
    if self.frame < 2:
        self.frame = 2
    if self.frame >= 2 and self.frame < 10:
        self.disableJostle = True
    else:
        self.disableJostle = False
        if self.is_left == True and self.x < char_pos[-(self.playerNum)][0]:
            self.is_left = False
        if self.is_left == False and self.x > char_pos[-(self.playerNum)][0]:
            self.is_left = True
else:
    self.isCrouch = True
    if self.frame >= 2 and self.frame < 10:
        self.moveXThisFrame = SPECIAL_S_X*direction_mul*1.2
        self.disableJostle = True
    else:
        self.disableJostle = False
        if self.is_left == True and self.x < char_pos[-(self.playerNum)][0]:
            self.is_left = False
        if self.is_left == False and self.x > char_pos[-(self.playerNum)][0]:
            self.is_left = True
if self.char_id == 2:
    self.moveXThisFrame = SPECIAL_S_X*direction_mul*1.2

def specialN(self):
    # contains the projectile logic
    if self.animListID != get_anim_ID("SpecialN"):
        self.hadouTimer = 0
    return

    if self.hadouTimer >= 1: # If you've hit the opponent with your projectile, switch to
the version of the anim where the projectile sprite doesnt exist
        self.set_new_anim_by_ID(get_anim_ID("SpecialNEmpty"), self.frame)
    if self.doPushback:
        self.hadouTimer += 1

def set_pushback(self):
    #Prevents these moves having pushback per character
    if self.animListID == get_anim_ID("HeavyN") and self.char_id == 0:
        return
    if self.animListID == get_anim_ID("AttackLw") and self.char_id == 2:
        return

```

```

        if self.animListID == get_anim_ID("HeavyLw") and self.char_id == 2:
            return

        if self.animListID in [get_anim_ID("AttackLw"), get_anim_ID("Attack"),
get_anim_ID("Heavy"), get_anim_ID("HeavyLw"), get_anim_ID("ThrowF")]:
            side_mul = 1
            if self.is_left:
                side_mul = -1

        if self.doPushback == True:
            self.moveXThisFrame = PUSHBACK * side_mul * -1
            if self.animListID == get_anim_ID("ThrowF"):
                self.moveXThisFrame *= 0.7

def dash(self):
    global ACTIONABLE_LIST
    if self.isJump or self.isCrouch or self.isHitstun:
        return

    side_mul = 1
    if self.is_left:
        side_mul = -1

    if self.animListID == get_anim_ID("ForwardDash"):
        if self.lastmoveX != 0:
            self.moveXThisFrame = self.lastmoveX -
(stats.FDASH_DECEL[self.char_id]*side_mul)

            if abs(self.lastmoveX) - stats.FDASH_DECEL[self.char_id] < 0:
                self.moveXThisFrame = 0

    if self.animListID == get_anim_ID("BackDash"):
        if self.lastmoveX != 0:
            self.moveXThisFrame = self.lastmoveX +
(stats.BDASH_DECEL[self.char_id]*side_mul)

            if abs(self.lastmoveX) - stats.BDASH_DECEL[self.char_id] < 0:
                self.moveXThisFrame = 0

def check_correct_side(self):
    global ACTIONABLE_LIST
    if (self.isJump == False and self.animListID in ACTIONABLE_LIST) or
self.animListID == get_anim_ID("JumpSquat"):

```

```

        #Flips you around and puts you into the default animation when crossed up
        if self.is_left == True and self.x < char_pos[-(self.playerNum)][0]:
            self.is_left = False
            self.set_new_anim_by_ID()
        if self.is_left == False and self.x > char_pos[-(self.playerNum)][0]:
            self.is_left = True
            self.set_new_anim_by_ID()

    def check_block(self):
        #if self.isJump:
        #self.isBlocking = False
        if self.isHitstun:
            return
        if self.hitstunFrames > 0:
            return
        if not (self.animListID in ACTIONABLE_LIST or self.animListID in BLOCKING_LIST):
            self.isBlocking = False
            return

        if not self.is_left:
            if self.buttoncheck[0] and not self.buttoncheck[1]:
                self.isBlocking = True
            else:
                self.isBlocking = False
        else:
            if self.buttoncheck[1] and not self.buttoncheck[0]:
                self.isBlocking = True
            else:
                self.isBlocking = False

    def runCpuQueue(self):
        if self.isHitstun:
            self.cpuQueue = []
            return

        if len(self.cpuQueue) < 1: #If queue is empty, try and fill it up again
            if self.animListID in ACTIONABLE_LIST and not self.isLeaveBlockstun:
                cpu_data.check_movement(self, players_x[self.playerNum - 1], players_x[-self.playerNum], players_y[self.playerNum - 1], players_y[-self.playerNum])
                if self.cpuQueue[0] in [get_anim_ID("ForwardDash"), get_anim_ID("Backdash")]:
                    self.cpuQueue.insert(0, cpu_data.WAIT)
                    self.cpuQueue.insert(0, cpu_data.WAIT)
                    self.cpuQueue.insert(0, cpu_data.WAIT)
            else:
                if self.animListID in [get_anim_ID("WalkF"), get_anim_ID("WalkB")]:
                    self.set_new_anim_by_ID()

```

```

        return

    if self.animListID == get_anim_ID("WalkF"): #Ensures they walk properly!
        if self.is_left:
            self.moveXThisFrame = -stats.WALK_SPEED[self.char_id]
        else:
            self.moveXThisFrame = stats.WALK_SPEED[self.char_id]
    if self.animListID == get_anim_ID("WalkB"):
        if self.is_left:
            self.moveXThisFrame = stats.WALK_SPEED[self.char_id]
        else:
            self.moveXThisFrame = -stats.WALK_SPEED[self.char_id]

    next_command = self.cpuQueue[0]
    if next_command == self.animListID: #If the command is repeated just keep on
doing what you been doing
        self.cpuQueue.pop(0)
        return

    if self.animListID in SPECIAL_CANCEL_LIST and self.doPushback:
        other = self.accessOtherPlayer[self.playerNum - 1]
        if not other.isBlockstun:
            if random.randint(0, 100) < 30:
                if not self.isJump:
                    self.set_new_anim_by_ID(cpu_data.SPECIAL_CANCEL[self.char_id])
                else:
                    self.set_new_anim_by_ID(get_anim_ID("SpecialS"))
            else:
                rand_val = random.randint(0, 100)
                if rand_val < 5:
                    if not self.isJump:
                        self.set_new_anim_by_ID(cpu_data.SPECIAL_CANCEL_SAFE[self.c
har_id])
                    else:
                        self.set_new_anim_by_ID(get_anim_ID("SpecialS"))
                elif rand_val < 8:
                    if not self.isJump:
                        self.set_new_anim_by_ID(cpu_data.SPECIAL_CANCEL[self.char_id])
                    else:
                        self.set_new_anim_by_ID(get_anim_ID("SpecialS"))

    if self.animListID in ACTIONABLE_LIST and ((not self.isJump and next_command
in cpu_data.GROUND_MOVE) or (self.isJump and next_command in
cpu_data.AIR_MOVE)):
        if next_command == get_anim_ID("ForwardDash"):
            self.set_new_anim_by_ID(next_command)
            self.cpuQueue.pop(0)
            if self.is_left:

```

```

        self.moveXThisFrame = -stats.FDASH_SPEED[self.char_id]
    else:
        self.moveXThisFrame = stats.FDASH_SPEED[self.char_id]
    elif next_command == get_anim_ID("BackDash"):
        self.set_new_anim_by_ID(next_command)
        self.cpuQueue.pop(0)
        if self.is_left:
            self.moveXThisFrame = stats.BDASH_SPEED[self.char_id]
        else:
            self.moveXThisFrame = -stats.BDASH_SPEED[self.char_id]
    elif next_command == get_anim_ID("WalkF"):
        self.set_new_anim_by_ID(next_command)
        self.cpuQueue.pop(0)
        if self.is_left:
            self.moveXThisFrame = -stats.WALK_SPEED[self.char_id]
        else:
            self.moveXThisFrame = stats.WALK_SPEED[self.char_id]
    elif next_command == get_anim_ID("WalkB"):
        self.set_new_anim_by_ID(next_command)
        self.cpuQueue.pop(0)
        if self.is_left:
            self.moveXThisFrame = stats.WALK_SPEED[self.char_id]
        else:
            self.moveXThisFrame = -stats.WALK_SPEED[self.char_id]
    else:
        self.set_new_anim_by_ID(next_command)
        self.cpuQueue.pop(0)
    return

def update(self): #Once per player frame
    global PUSHBOXES
    global PUSHING_FORCE
    global force_hit_now

    players_x[self.playerNum - 1] = self.x
    players_y[self.playerNum - 1] = self.y

    if force_hit_now and not self.hasHit and not hitbox_properties[-(self.playerNum)]
    == None:
        force_hit_now = False
        #print("Has Forced a hit")
        other = self.accessOtherPlayer[self.playerNum - 1]
        other.maxHitstun = hitbox_properties[-(self.playerNum)][2]
        self.start_hitstun()

    self.check_correct_side()
    self.check_block()
    self.check_is_hurt()

```



```

self.check_is_hitting()
self.hitstun_movement()
self.air()
self.dash()

if self.readLog:
    self.buttoncheck = self.log[self.logFrame]
    self.logFrame += 1
    if self.logFrame >= self.logLength:
        self.logFrame = 0
elif self.isCpu:
    self.buttoncheck = [False,False,False,False,False,False,False]
else:
    self.buttoncheck =
[keyboard.is_pressed(self.controls[0]),keyboard.is_pressed(self.controls[1]),
    keyboard.is_pressed(self.controls[2]),
keyboard.is_pressed(self.controls[3]),
    keyboard.is_pressed(self.controls[4]),
keyboard.is_pressed(self.controls[5]),
    keyboard.is_pressed(self.controls[6])
]

if self.isCpu:
    self.runCpuQueue()

#Logs player 1 stuff
if self.playerNum == 1:
    try:
        f = open("log.txt","a")
        f.write(str(self.buttoncheck))
        f.write("\n")
        f.close()
    except Exception as exc:
        print(exc)
        sys.exit()

#Move specific code
self.attackLw()
self.heavyN()
self.specialLw()
self.specialS()
self.specialN()
self.throw_logic()

self.animate()

if not self.isCpu:

```

```

        if self.buttoncheck[0] and not self.buttoncheck[1]:
            self.left()
        elif self.animListID == get_anim_ID("WalkF") and (self.is_left != False) or
(self.animListID == get_anim_ID("WalkB") and (self.is_left != True )):
            if not self.isHitstun:
                self.set_new_anim_by_ID() #Resets you to standing when left no longer
pressed

        if self.buttoncheck[0] and not self.buttoncheck[1]:
            self.leftPressed = True
        else:
            self.leftPressed = False

        if self.buttoncheck[1] and not self.buttoncheck[0]:
            self.right()
        elif (self.animListID == get_anim_ID("WalkF") and (self.is_left != True)) or
(self.animListID == get_anim_ID("WalkB") and (self.is_left != False )):
            self.set_new_anim_by_ID() #Resets you to standing when right no longer
pressed

        if self.buttoncheck[1] and not self.buttoncheck[0]:
            self.rightPressed = True #Set so it can be used to check if the button has just
been pressed
        else:
            self.rightPressed = False

        if self.buttoncheck[3] and not self.buttoncheck[2]:
            self.down()
        elif self.isCrouch:
            self.uncrouch()
        if (self.buttoncheck[2] and not self.buttoncheck[3]) or self.jumpBuffer > 0:
            self.up()

        if (self.buttoncheck[4] and self.buttoncheck[5] and not self.buttoncheck[6]) and
not(self.buttoncheck[0] or self.buttoncheck[1]):
            self.throw()

        if (self.buttoncheck[4] and not self.buttoncheck[5] and not self.buttoncheck[6]
and not self.attackPressed) or self.attackBuffer > 0:
            self.attackPressed = True
            self.attack()

        if (self.buttoncheck[6] and not self.buttoncheck[4] and not self.buttoncheck[5]
and not self.specialPressed) or self.specialBuffer > 0:
            self.specialPressed = True
            self.special()

```

```

        if (self.buttoncheck[5] and not self.buttoncheck[4] and not self.buttoncheck[6]
and not self.heavyPressed) or self.heavyBuffer > 0:
            self.heavyPressed = True
            self.heavy()

    if self.attackBuffer > 0:
        self.attackBuffer -= 1

    if self.specialBuffer > 0:
        self.specialBuffer -= 1

    if self.jumpBuffer > 0:
        self.jumpBuffer -= 1

    if self.heavyBuffer > 0:
        self.heavyBuffer -= 1

    if self.start_blocking_now > 0:
        self.start_blocking_now -= 1

    if self.hitstunFrames > 0:
        #print("still in hitstun")
        self.hitstunFrames -= 1

    if self.backdashTimer > 0:
        self.backdashTimer -= 1
    if self.forwarddashTimer > 0:
        self.forwarddashTimer -= 1

    if self.training_settings[4] != 0 and self.playerNum == 2:
        if (self.isHitstun and self.training_settings[3] == 0) or self.isBlockstun:
            self.isLeaveBlockstun = True
        elif self.isLeaveBlockstun and self.animListID in ACTIONABLE_LIST :
            if self.training_settings[4] == 1 and not self.isJump:
                #print("force jab")
                self.set_new_anim_by_ID(get_anim_ID("Attack"))
                self.isLeaveBlockstun = False
            elif self.training_settings[4] == 2 and not self.isJump:
                #print("force jab")
                self.set_new_anim_by_ID(get_anim_ID("AttackLw"))
                self.isLeaveBlockstun = False
            elif self.training_settings[4] == 3 and not self.isJump:
                #print("force jab")
                self.set_new_anim_by_ID(get_anim_ID("Heavy"))
                self.isLeaveBlockstun = False
            elif self.training_settings[4] == 4 and not self.isJump:

```

```

        #print("force jab")
        self.set_new_anim_by_ID(get_anim_ID("HeavyLw"))
        self.isLeaveBlockstun = False
    elif self.training_settings[4] == 5 and not self.isJump:
        #print("force dp")
        self.set_new_anim_by_ID(get_anim_ID("SpecialLw"))
        self.isLeaveBlockstun = False
    elif self.training_settings[4] == 6 and not self.isJump:
        #print("force dp")
        self.set_new_anim_by_ID(get_anim_ID("SpecialN"))
        self.isLeaveBlockstun = False
    elif self.training_settings[4] == 7 and not self.isJump:
        #print("force dp")
        self.set_new_anim_by_ID(get_anim_ID("SpecialS"))
        self.isLeaveBlockstun = False
    elif self.training_settings[4] == 8 and not self.isJump:
        #print("force backdash")
        self.set_new_anim_by_ID(get_anim_ID("BackDash"))
        self.isLeaveBlockstun = False
        if self.is_left:
            self.moveXThisFrame = stats.BDASH_SPEED[self.char_id]
        else:
            self.moveXThisFrame = -stats.BDASH_SPEED[self.char_id]
    if self.isCpu:
        if self.isBlockstun:
            self.isLeaveBlockstun = True
        elif self.isLeaveBlockstun and self.animListID in ACTIONABLE_LIST and not
self.isJump:
            rng = random.randint(0, 100)
            if rng < 35:
                self.set_new_anim_by_ID(cpu_data.JAB[self.char_id])
            elif rng < 45:
                self.set_new_anim_by_ID(cpu_data.DP[self.char_id])
            elif rng < 55:
                self.set_new_anim_by_ID(cpu_data.DP2[self.char_id])
            elif rng < 60:
                self.set_new_anim_by_ID(cpu_data.GRAB)
            elif rng < 62:
                self.set_new_anim_by_ID(get_anim_ID("BackDash"))
                if self.is_left:
                    self.moveXThisFrame = stats.BDASH_SPEED[self.char_id]
                else:
                    self.moveXThisFrame = -stats.BDASH_SPEED[self.char_id]
            self.isLeaveBlockstun = False

    if self.hitstunFrames <= 0:
        self.isHitstun = False

```

```

        self.isBlockstun = False
        if self.y < 5:
            self.y = 0

    if not self.isHitstun:
        self.maxHitstun = False

    if not self.buttoncheck[4]:
        self.attackPressed = False

    if not self.buttoncheck[5]:
        self.heavyPressed = False

    if not self.buttoncheck[6]:
        self.specialPressed = False

    self.set_pushback()

    if True:#self.moveXThisFrame != 0 or self.moveYThisFrame != 0:
        x = self.x
        y = self.y
        newXVal = x
        newYVal = y
        #print(f"{newXVal}, {newYVal}")

    #Forces players to stay within fighting area
    if abs(x+self.moveXThisFrame) < BOX_SIZE[0]-4:
        newXVal = x+self.moveXThisFrame

    newYVal = y+self.moveYThisFrame

    #Pushbox
    left_pushbox = (char_pos[-(self.playerNum)][0] - PUSHBOXES[0]) - self.x <
105 and self.is_left == False
    right_pushbox = self.x - (char_pos[-(self.playerNum)][0] + PUSHBOXES[0]) <
105 and self.is_left
    top_pushbox = self.y - (char_pos[-(self.playerNum)][1] + PUSHBOXES[2]) <
55
    bottom_pushbox = (char_pos[-(self.playerNum)][1] + PUSHBOXES[1]) -
self.y < 105
    #if top_pushbox and bottom_pushbox:
        #print(f"same height: [{(char_pos[-(self.playerNum)][0] - PUSHBOXES[0]) -
self.x}, {self.x - (char_pos[-(self.playerNum)][0] + PUSHBOXES[0])}]")
        #print(f"X Pos for other one: {char_pos[-(self.playerNum)][0]}")
        if (left_pushbox or right_pushbox) and (top_pushbox and bottom_pushbox)
and not self.disableJostle:

```

```

        #print("pushing!")
        if ((self.moveXThisFrame > 0 and left_pushbox) or (self.moveXThisFrame
< 0 and right_pushbox)):
            newXVal = x

            if (abs(char_pos[0][0]- char_pos[1][0]) < 104):
                push_off = PUSHING_FORCE
                if not self.is_left:
                    push_off *= -1
                newXVal = x
                newXVal += push_off

            self.x = newXVal
            self.y = newYVal

            if self.x > BOX_SIZE[0]-4:
                self.x = BOX_SIZE[0]-4
            if self.x < -BOX_SIZE[0] +4:
                self.x = -BOX_SIZE[0] +4
            if self.y < 0:
                self.y = 0
            self.turtle.setpos(self.x, self.y)
            self.lastmoveX = newXVal - x
            self.lastmoveY = newYVal - y
            #print(f"{self.lastmoveX}, {self.lastmoveY}")
            self.moveXThisFrame = 0.0
            self.moveYThisFrame = 0.0

        #self.update_hit_hurt()

        char_pos[self.playerNum - 1][0] = self.x
        char_pos[self.playerNum - 1][1] = self.y
        #print(char_pos)

class battleUI:
    def __init__(self, screen, p1Turtle, p2Turtle):
        self.screen = screen
        self.p1 = p1Turtle
        self.p2 = p2Turtle
        self.p1HP = turtle.Turtle()
        self.p2HP = turtle.Turtle()
        self.youWin = turtle.Turtle()
        self.p1HpPos = [-200,150]
        self.p2HpPos = [200,150]
        self.p1HP.penup()

```

```
self.p2HP.penup()
self.youWin.penup()
screen.tracer(0)
self.youWin.hideturtle()
self.p1HP.hideturtle()
self.p2HP.hideturtle()
self.p1HP.goto(self.p1HpPos[0], self.p1HpPos[1])
self.p2HP.goto(self.p2HpPos[0], self.p2HpPos[1])
self.youWin.goto(0, 0)
screen.update()
screen.tracer(10)
```

```
self.hp_bar_length = 300
self.hp_start_x = 50
self.hp_start_y = 175
self.hp_bar_thickness = 2*SCALE
```

```
def clear(self): #Clears the screen
    self.p1HP.clear()
    self.p2HP.clear()
    self.youWin.clear()
```

```
def update_healthbar(self): #Updates health bar based on their health remaining
    p1_hp_length = (self.p1.hp/stats.HEALTH[self.p1.char_id]) * self.hp_bar_length
    p2_hp_length = (self.p2.hp/stats.HEALTH[self.p2.char_id]) * self.hp_bar_length
    self.screen.tracer(0)
    self.p1HP.clear()
    self.p1HP.penup()
    self.p1HP.goto(-self.hp_start_x, self.hp_start_y - self.hp_bar_thickness)
    if self.p1.hp > 0:
        self.p1HP.fillcolor("cyan")
        self.p1HP.begin_fill()
        self.p1HP.goto(-self.hp_start_x, self.hp_start_y + self.hp_bar_thickness)
        self.p1HP.goto(-(self.hp_start_x + p1_hp_length), self.hp_start_y +
self.hp_bar_thickness)
        self.p1HP.goto(-(self.hp_start_x + p1_hp_length), self.hp_start_y -
self.hp_bar_thickness)
        self.p1HP.goto(-(self.hp_start_x), self.hp_start_y - self.hp_bar_thickness)
        self.p1HP.end_fill()
    self.p1HP.goto(-(self.hp_start_x + (self.hp_bar_length * 0.5)), self.hp_start_y)
    self.p1HP.shape("sprites/hpbar.gif")
    self.p1HP.stamp()

    self.p2HP.clear()
    self.p2HP.penup()
    if self.p2.hp > 0:
        self.p2HP.goto(self.hp_start_x, self.hp_start_y)
        self.p2HP.fillcolor("red")
```

```

        self.p2HP.begin_fill()
        self.p2HP.goto(self.hp_start_x, self.hp_start_y + self.hp_bar_thickness)
        self.p2HP.goto((self.hp_start_x + p2_hp_length), self.hp_start_y +
self.hp_bar_thickness)
        self.p2HP.goto((self.hp_start_x + p2_hp_length), self.hp_start_y -
self.hp_bar_thickness)
        self.p2HP.goto((self.hp_start_x), self.hp_start_y - self.hp_bar_thickness)
        self.p2HP.end_fill()
        self.p2HP.goto((self.hp_start_x + (self.hp_bar_length * 0.5)), self.hp_start_y)
        self.p2HP.shape("sprites/hpbar.gif")
        self.p2HP.stamp()

```

```

self.screen.update()
self.screen.tracer(10)

```

```

def update(self):
    self.update_healthbar()
    if self.p1.hp <= 0 or self.p2.hp <= 0:
        player = 1
        if self.p2.hp > self.p1.hp:
            player = 2
        self.youWin.shape(f"sprites/p{player}_win.gif") #Hooray
        self.youWin.stamp()

```

```

chars = []
cpus = False

```

```

def rematch(x,y): #Another on click method for turtle screen awesome
    global chars
    if abs(x) > 25:
        return
    if y > 13 and y < 40:
        menu.run(True,chars,cpus)
    if y > -60 and y < -33:
        menu.run()

```

```

def pause_update(pause_turtle, controls, training_settings:list):
    global chars
    shape = pause_turtle.shape()
    up = [controls[0][2], controls[1][2]]
    down = [controls[0][3], controls[1][3]]
    accept = [controls[0][4], controls[1][4]]
    if not training_settings[0]:
        if shape in ["menu/pause_1.gif", "menu/pause_2.gif", "menu/pause_3.gif"]:
            if keyboard.is_pressed(up[0]) or keyboard.is_pressed(up[1]):
                if shape == "menu/pause_2.gif":
                    pause_turtle.shape("menu/pause_1.gif")
                    time.sleep(0.2)

```



```

        if shape == "menu/pause_3.gif":
            pause_turtle.shape("menu/pause_2.gif")
            time.sleep(0.2)
    if keyboard.is_pressed(down[0]) or keyboard.is_pressed(down[1]):
        if shape == "menu/pause_1.gif":
            pause_turtle.shape("menu/pause_2.gif")
            time.sleep(0.2)
        if shape == "menu/pause_2.gif":
            pause_turtle.shape("menu/pause_3.gif")
            time.sleep(0.2)
    if keyboard.is_pressed(accept[0]) or keyboard.is_pressed(accept[1]):
        if shape == "menu/pause_1.gif":
            pause_turtle.shape("menu/movelist.gif")
        if shape == "menu/pause_2.gif":
            return 2
        if shape == "menu/pause_3.gif":
            menu.run()
    else:
        if shape in ["menu/training_pause_1.gif", "menu/training_pause_2.gif",
"menu/training_pause_3.gif"]:
            if keyboard.is_pressed(up[0]) or keyboard.is_pressed(up[1]):
                if shape == "menu/training_pause_2.gif":
                    pause_turtle.shape("menu/training_pause_1.gif")
                    time.sleep(0.2)
                if shape == "menu/training_pause_3.gif":
                    pause_turtle.shape("menu/training_pause_2.gif")
                    time.sleep(0.2)
            if keyboard.is_pressed(down[0]) or keyboard.is_pressed(down[1]):
                if shape == "menu/training_pause_1.gif":
                    pause_turtle.shape("menu/training_pause_2.gif")
                    time.sleep(0.2)
                if shape == "menu/training_pause_2.gif":
                    pause_turtle.shape("menu/training_pause_3.gif")
                    time.sleep(0.2)
            if keyboard.is_pressed(accept[0]) or keyboard.is_pressed(accept[1]):
                if shape == "menu/training_pause_1.gif":
                    pause_turtle.shape("menu/training_menu.gif")
                    return 1
                if shape == "menu/training_pause_2.gif":
                    return 2
                if shape == "menu/training_pause_3.gif":
                    menu.run()

class setting_icon:
    def __init__(self, pos:int, list_update:list, list_num:int, screen):
        self.pos = pos
        self.turtle = turtle.Turtle()
        self.turtle.penup()

```

```

        self.turtle.goto(0, pos)
        self.list_update = list_update
        self.list_num = list_num
        self.turtle.shape(self.list_update[list_num])
        self.screen = screen

    def onclick_update(self, x, y):
        self.list_num += 1
        if self.list_num >= len(self.list_update):
            self.list_num = 0
        self.turtle.shape(self.list_update[self.list_num])
        self.screen.update()
        #print("update")

class save_icon:
    def __init__(self, pos:int, turtle_list:list, screen, characters:list):
        self.pos = pos
        self.turtle = turtle.Turtle()
        self.turtle.penup()
        self.turtle.goto(0, pos)
        self.turtle_list = turtle_list
        self.turtle.shape("menu/training_save.gif")
        self.screen = screen
        self.characters = characters

    def onclick_update(self, x, y):
        new_training_settings = [True]
        new_training_settings.append(bool(self.turtle_list[0].list_num))
        new_training_settings.append(bool(self.turtle_list[1].list_num))
        new_training_settings.append(int(self.turtle_list[2].list_num))
        new_training_settings.append(int(self.turtle_list[3].list_num))
        #print(new_training_settings)
        run(new_training_settings, self.characters)

def run(training_settings=[False,False,False,0,0], character=[0,0], cpu=False):
    global chars
    global cpus
    chars = character
    cpus = cpu
    controlsList = get_controls_from_txt()
    turtle.TurtleScreen._RUNNING=True
    screen = turtle.Screen()
    screen.setup(SCREEN_WIDTH, SCREEN_HEIGHT)
    screen.title("Shotos")
    screen.bgcolor("white")
    screen.clearscreen()
    #Ifs in training mode, Enable hitboxes, Enable Frame by Frame, Blocking Mode,

```

Reversal]

```
#screen.delay(17)
f = open("log.txt","w")
f.write("")
f.close()
for i in ["sprites", "reverse_sprites"]:
    for root, dirs, files in os.walk(i):
        #print(files)
        for filename in files:
            if filename.endswith(".gif"):
                #print(filename)
                #Handles scaling up and stuff
                larger = PhotoImage(file=f"{i}/{filename}").zoom(SCALE, SCALE)
                screen.addshape(f"{i}/{filename}", Shape("image", larger))
for root, dirs, files in os.walk("menu"):
    for filename in files:
        if filename.endswith(".gif"):
            screen.addshape(f"menu/{filename}")
```

screen.tracer(0)

if training_settings[1] and training_settings[0]: #Hitbox/hurtbox drawing!

```
p1_hurtbox_draw = turtle.Turtle()
p1_hurtbox_draw.color("green")
p1_hurtbox_draw.hideturtle()
p1_hurtbox_draw2 = turtle.Turtle()
p1_hurtbox_draw2.color("green")
p1_hurtbox_draw2.hideturtle()
p1_hitbox_draw = turtle.Turtle()
p1_hitbox_draw.color("red")
p1_hitbox_draw.hideturtle()
p2_hurtbox_draw = turtle.Turtle()
p2_hurtbox_draw.color("green")
p2_hurtbox_draw.hideturtle()
p2_hurtbox_draw2 = turtle.Turtle()
p2_hurtbox_draw2.color("green")
p2_hurtbox_draw2.hideturtle()
p2_hitbox_draw = turtle.Turtle()
p2_hitbox_draw.color("red")
p2_hitbox_draw.hideturtle()
```

#Initialising players

```
p1 = player(
    1,
    -150, 0,
    #["a", "d", "w", "s", "f", "g", "h"],
    controlsList[0],
    False,
```

```

        training_settings,
        character[0]
    )
    p2 = player(
        2,
        150, 0,
        #["left", "right", "up", "down", ".", "/", "shift"],
        controlsList[1],
        True,
        training_settings,
        character[1]
    )
    if cpu:
        p2.isCpu = True

    accessOtherPlayer = [p2, p1]
    p1.update_other_list(accessOtherPlayer)
    p2.update_other_list(accessOtherPlayer)

    youwin = False

    ui = battleUI(screen, p1, p2)

    ground = turtle.Turtle()
    ground.penup()
    ground.pensize(5)
    ground.goto(-1000,-125)
    ground.pendown()
    ground.goto(1000,-125)
    ground.penup()

    if training_settings[2]:
        frame_advance = turtle.Turtle()
        frame_advance.penup()
        frame_advance.goto(0,-150)
        frame_advance.shape("menu/frame_advance.gif")

    pause_ui = turtle.Turtle()
    pause_ui.penup()
    pause_ui.goto(-4,4)
    pause_ui.hideturtle()
    if not training_settings[0]:
        pause_ui.shape("menu/pause_1.gif")
    else:
        pause_ui.shape("menu/training_pause_1.gif")

    ui.update()

```

```

p1.update()
p2.update()
p1.update()
p2.update()
screen.tracer(10)
screen.update()
paused = False
pause_release = False

prev_delay = 0.0
p1HPRestore = 0
p2HPRestore = 0
while youwin == False:
    try:
        if prev_delay > FRAME_LENGTH*2:
            pass
            #print("Frame Skip! Performance aint looking good")

        start = time.time()
        turn_order = bool(random.getrandbits(1))
        if not paused and (((training_settings[2] and keyboard.is_pressed("space"))
or (not training_settings[2]))):
            pause_ui.hideturtle()
            if training_settings[0]:
                pause_ui.shape("menu/training_pause_1.gif")
            else:
                pause_ui.shape("menu/pause_1.gif")
            if not keyboard.is_pressed("escape"):
                pause_release = True
            if keyboard.is_pressed("escape") and pause_release:
                pause_release = False
                paused = True
            ui.update()
            if turn_order:
                p1.update()
                p2.update()
            else:
                p2.update()
                p1.update()
            if training_settings[0] and training_settings[1]:
                draw_boxes(screen, p1_hurtbox_draw, p1_hurtbox_draw2,
p1_hitbox_draw, p2_hurtbox_draw, p2_hurtbox_draw2, p2_hitbox_draw)
            elif paused:
                pause_ui.showturtle()
                ui.clear()
                if training_settings[0] and training_settings[1]:
                    p1_hurtbox_draw.clear()
                    p1_hurtbox_draw2.clear()

```

```

        p1_hitbox_draw.clear()
        p2_hurtbox_draw.clear()
        p2_hurtbox_draw2.clear()
        p2_hitbox_draw.clear()
    screen.update()
    if not keyboard.is_pressed("escape"):
        pause_release = True
    if keyboard.is_pressed("escape") and pause_release:
        pause_release = False
        paused = False
    pause_val = pause_update(pause_ui, controlsList, training_settings)
    if pause_val == 2:
        paused = False
    if pause_val == 1:
        screen.clearscreen()
        screen.tracer(0)
        block = ["menu/block_0.gif", "menu/block_1.gif", "menu/block_2.gif"]
        hitbox = ["menu/hitbox_false.gif", "menu/hitbox_true.gif"]
        frame = ["menu/frame_false.gif", "menu/frame_true.gif"]
        reversal = [
            "menu/reversal_0.gif", "menu/reversal_1.gif", "menu/reversal_2.gif",
"menu/reversal_3.gif", "menu/reversal_4.gif", "menu/reversal_5.gif",
            "menu/reversal_6.gif", "menu/reversal_7.gif", "menu/reversal_8.gif"
        ]
        bg = turtle.Turtle()
        bg.shape("menu/training_menu.gif")
        setting_hitbox = setting_icon(90, hitbox, int(training_settings[1]), screen)
        setting_frame = setting_icon(30, frame, int(training_settings[2]), screen)
        setting_block = setting_icon(-30, block, int(training_settings[3]), screen)
        setting_reversal = setting_icon(-90, reversal, int(training_settings[4]),
screen)
        save = save_icon(-150,
[setting_hitbox,setting_frame,setting_block,setting_reversal], screen, character)
        screen.update()
        screen.tracer(10)
        setting_hitbox.turtle.onclick(setting_hitbox.onclick_update)
        setting_frame.turtle.onclick(setting_frame.onclick_update)
        setting_block.turtle.onclick(setting_block.onclick_update)
        setting_reversal.turtle.onclick(setting_reversal.onclick_update)
        save.turtle.onclick(save.onclick_update)
        screen.mainloop()
    elif training_settings[2] and not keyboard.is_pressed("space"):
        pause_ui.hideturtle()
        if training_settings[0]:
            pause_ui.shape("menu/training_pause_1.gif")
        else:
            pause_ui.shape("menu/pause_1.gif")
        if not keyboard.is_pressed("escape"):

```

```

        pause_release = True
    if keyboard.is_pressed("escape") and pause_release:
        pause_release = False
        paused = True
    ui.update()
    end = time.time()
    new_delay = (end-start)*10**3
    #print(f"Time taken: {new_delay}ms, normalise: {FRAME_LENGTH -
(new_delay)}ms")
    if (new_delay/1000) < FRAME_LENGTH/1000:
        time.sleep(FRAME_LENGTH/1000 - (new_delay/1000))
    prev_delay = new_delay

    p1.hp -= p1.decreaseHp
    p2.hp -= p2.decreaseHp
    if training_settings[0] and (p1.hp < 1 or p1HPRestore >=
HP_RESTORE_DURATION):
        p1.hp = stats.HEALTH[p1.char_id]
        p1HPRestore = 0
    if training_settings[0] and (p2.hp < 1 or p2HPRestore >=
HP_RESTORE_DURATION):
        p2.hp = stats.HEALTH[p2.char_id]
        p2HPRestore = 0
    if p1.decreaseHp == 0:
        p1HPRestore += 1
    else:
        p1HPRestore = 0
    if p2.decreaseHp == 0:
        p2HPRestore += 1
    else:
        p2HPRestore = 0

    p1.decreaseHp = 0
    p2.decreaseHp = 0
    if p1.hp <= 0 or p2.hp <= 0:
        youwin = True
except Exception as exc:
    print(exc)
    sys.exit()

ui.update()
screen.update()
time.sleep(2)
screen.tracer(0)
resetti = turtle.Turtle()
resetti.shape("menu/rematch.gif")
screen.update()
screen.onscreenclick(rematch)

```

```
screen.mainloop()
```

build.py

```
from distutils.core import setup
import py2exe
import shutil
import os

def empty_folder(folder):
    for filename in os.listdir(folder):
        file_path = os.path.join(folder, filename)
        try:
            if os.path.isfile(file_path) or os.path.islink(file_path):
                os.unlink(file_path)
            elif os.path.isdir(file_path):
                shutil.rmtree(file_path)
        except Exception as e:
            print('Failed to delete %s. Reason: %s' % (file_path, e))

empty_folder(r'dist') #Empties the folder so we dont carry anything from prev builds
#Moving all the assets!
shutil.copytree(r"sprites", r"dist\sprites")
shutil.copytree(r"reverse_sprites", r"dist\reverse_sprites")
shutil.copytree(r"menu", r"dist\menu")
shutil.copytree(r"text", r"dist\text")
shutil.copy(r"default_controls.txt", r"dist\controls.txt")

setup( #Fancy py2exe compiler thing
    options = {'py2exe': {'bundle_files': 1, 'compressed': True}},
    windows = [{ 'script': "main.py"}],
    zipfile = None,
)
```

image_reverser.py

```
#uses PIL to create a reversed image of all sprites in sprites folder
from PIL import Image, ImageTk
import os

def reverse():
    for root, dirs, files in os.walk("sprites"):
        #print(files)
        for filename in files:
            if filename.endswith(".png"):
                pil_image = Image.open(f"sprites/{filename}")
```



```
new_file = f"sprites/{filename}"
new_file = new_file.replace(".png", ".gif")
pil_image = pil_image.save(new_file)
for root, dirs, files in os.walk("sprites"):
    #print(files)
    for filename in files:
        if filename.endswith(".gif"):
            pil_image = Image.open(f"sprites/{filename}")
            pil_image_flip = pil_image.transpose(Image.Transpose.FLIP_LEFT_RIGHT)
            pil_image_flip = pil_image_flip.save(f"reverse_sprites/{filename}")
```